

INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE
MONTERREY
CAMPUS CIUDAD DE MÉXICO

School of Engineering and Science



**TECNOLÓGICO
DE MONTERREY®**

**View planning for three-dimensional environment
reconstruction using the Next Best View method**

*A thesis submitted in partial fulfilment of the requirements
for the degree of Master of Science in Engineering*

Author: [Everardo SHAIN
RUVALCABA](#)

Advisor: Dr. Efraín LÓPEZ DAMIAN

Thesis Committee:	Dr. Hugo Gustavo GONZÁLEZ HERNÁNDEZ	President
	Dr. Alfredo SANTANA DÍAZ	Secretary
	Dr. Efraín LÓPEZ DAMIAN	Spokesperson

Tlalpan, Ciudad de México
3rd December, 2024

Dedication

I dedicate this work to my parents and my friends who have been there for me through my ups and downs, keeping me motivated and giving me the strength to keep doing my best.

Acknowledgements

I would like to thank my parents Everardo Shain Salgado and Rosalía Angélica Ruvalcaba Armas for the support through all my life and for bringing me the opportunity to have a great education both at home and at excellent and prestigious institutions.

A big appreciation to my thesis advisor Dr. Efraín López Damian for the guidance during these two years. Thanks to Dr. Alfredo Santana Díaz and Dr. Hugo Gustavo González Hernández for their valuable comments as committee members.

Thanks to all my professors from all my educational levels for preparing me on the best way possible for this postgraduate stage.

Finally, I would like to express my gratitude to the institutions ITESM and CONAHCYT for making this whole project possible with their huge support.

Abstract

View planning for three-dimensional environment reconstruction using the Next Best View method

by Everardo SHAIN RUVALCABA

This study was made with the purpose of understanding the impact of the objective function and optimization methods on the Next Best View problem, which consists in finding the next position that the sensor or camera needs to take to scan an object or scenery in its totality. A simulated 5-Degree-of-Freedom mobile robot with a mounted simulated range sensor was used on a Virtual Reality Modeling Language environment, and the space discretization was made using a voxel map. For the objective function, two main factors were included: an area factor to make sure that the image taken by the sensor provides the best possible information, and a motion factor made up of distance and energy sub-factors to reduce the resources used by the robot, making multiple experiments on a laboratory scene to determine their best arrangement on the final objective function. Global optimization tasks such as a backstepping technique to escape local minima and a dynamic change in the objective function were implemented. The retrieval of the scene was made on an iterative process, with each iteration needing an optimization process for which three different methods were tested: Nelder-Mead, an Evolution Strategy, and Simulated Annealing. A set of experiments comparing the three methods in computational time and retrieval efficiency were made on three different environments with increasing difficulty to test their repeatability, with them being a laboratory model, a room with a cube and a pyramid inside it, and a study room with multiple furniture and windows.

Contents

Declaration of Authorship	ii
Dedication	iii
Acknowledgements	iv
Abstract	v
List of Figures	ix
List of Tables	xi
Abbreviations	xiii
Symbols	xiv
1 Introduction	1
1.1 Background	1
1.2 Literature Review	3
1.2.1 Robots	3
1.2.2 Space discretization	5
1.2.3 Objective function	6
1.2.4 Optimization method	7
1.3 Research Question and Hypothesis	8
1.4 Objectives	8
1.5 Main contributions	9
1.6 Methodology	9
2 Initial considerations	13
2.1 Robot	13
2.1.1 Robot dimensions	14
2.1.2 Implementation of orientation icosahedron	15
2.1.3 Sensor	17
2.2 Space discretization	19
2.2.1 Voxel map labeling	19

3	Objective function	20
3.1	Area factor	20
3.2	Motion factors	26
3.2.1	Distance factor	28
3.2.2	Energy factor	32
3.3	Quality factor	35
4	Global optimization	38
4.1	Backstepping	38
4.2	Dynamic change in objective function	40
4.2.1	Change in area factor	41
5	Local optimization	43
5.1	Mathematical model	43
5.1.1	Variables	43
5.1.2	Objective function	46
5.1.3	Constraints	46
5.2	Linear programming analysis	47
5.2.1	Proportionality	47
5.2.2	Additivity	48
5.2.3	Divisibility	49
5.2.4	Certainty	49
5.3	Proposed methods	49
5.3.1	Nelder-Mead algorithm	52
5.3.2	(1+1)-ES	54
5.3.3	Tabu search	56
5.3.4	Simulated annealing	58
6	Results and Discussion	61
6.1	Models	61
6.1.1	Room with objects model	61
6.1.2	Study room	63
6.1.3	Laboratory model	64
6.1.4	Laboratory sub-model	65
6.2	Setup parameters	67
6.3	Objective function experiments	67
6.3.1	Area factor experiments	68
6.3.2	Motion factor experiments	71
6.3.3	Quality factor experiments	75
6.4	Global optimization experiments	79
6.4.1	Backstepping experiments	79
6.4.2	Dynamic change in objective function experiments	82
6.5	Local optimization experiments	85
6.5.1	Nelder-Mead	86
6.5.2	(1+1)-ES	88
6.5.3	Simulated annealing	95
6.6	Final comparisons	96

6.6.1	Room with objects model comparisons	98
6.6.2	Study room model comparisons	101
6.6.3	Laboratory model comparisons	105
7	Conclusions and Future Work	109
7.1	Conclusions	109
7.2	Future Work	111
A	Articles Published	112
	Bibliography	114

List of Figures

1.1	NBV process.	2
1.2	Voxel map and octree.	5
1.3	General methodology.	9
1.4	Phase 1 diagram.	10
1.5	Phase 2 diagram.	11
1.6	Phase 3 diagram.	12
2.1	Proposed robot configuration.	14
2.2	Robot dimensions.	15
2.3	Safe cylinder.	15
2.4	Orientation icosahedron.	16
2.5	Sensor maximum and minimum distance.	17
2.6	Sensor view angle.	17
2.7	Sensor projected view.	18
3.1	Area factor function.	24
3.2	Distance and energy factors on proposed robot.	28
3.3	Comparison of distance factors.	31
3.4	Comparison of energy factors.	35
4.1	Global optimization process.	39
4.2	Area factor with and without dynamic change	42
5.1	Optimization diagram.	45
6.1	Room with objects model.	62
6.2	Room with objects model map.	62
6.3	Study room model.	63
6.4	Study room model map.	64
6.5	Laboratory model.	64
6.6	Laboratory model map.	65
6.7	Laboratory sub-model.	66
6.8	Laboratory sub-model map.	66
6.9	Area factor experiment retrieved models.	70
6.10	α vs $\%retrieved$	71
6.11	Motion factor experiment retrieved models greater than filter.	75
6.12	Motion factor experiment retrieved models smaller than filter.	76
6.13	Motion factor function.	77
6.14	Quality factor experiment retrieved models.	80

6.15	Backstepping experiment retrieved models.	83
6.16	Dynamic change in objective function experiment retrieved models.	85
6.17	Dynamic change in objective function zoomed comparison.	85
6.18	Nelder-Mead experiment retrieved models.	87
6.19	Nelder-Mead experiment time comparison.	88
6.20	(1+1)-ES $\sigma_{initial}$ experiment retrieved models.	90
6.21	(1+1)-ES gen_{max} experiment retrieved models.	92
6.22	(1+1)-ES gen_{max} experiment time comparison.	93
6.23	(1+1)-ES with tabu search experiment retrieved models.	94
6.24	Simulated annealing experiment retrieved models.	97
6.25	Room with objects model experiment retrieved models.	99
6.26	Room with objects model experiment graphs.	100
6.27	Study room model experiment retrieved models.	103
6.28	Study room model experiment graphs.	104
6.29	Laboratory model experiment retrieved models.	106
6.30	Laboratory model experiment graphs.	108
A.1	Accepted status of the article.	112
A.2	Production state of the article.	112
A.3	First page of the article.	113

List of Tables

2.1	Pan-tilt angles of orientation icosahedron.	16
5.1	Assumptions checklist.	50
6.1	Information from room with objects model.	62
6.2	Information from study room model.	63
6.3	Information from laboratory model.	64
6.4	Information from laboratory sub-model	65
6.5	Setup parameters.	67
6.6	Area factor experiment simulation parameters.	68
6.7	Area factor experiment variable parameters.	69
6.8	Area factor experiment results.	69
6.9	Motion factor experiment simulation parameters.	71
6.10	Motion factor experiment variable parameters.	73
6.11	Motion factor experiment results.	73
6.12	Motion factor experiment results after filter.	74
6.13	Quality factor experiment simulation parameters.	77
6.14	Quality factor experiment variable parameters.	78
6.15	Quality factor experiment results.	78
6.16	Quality factor experiment results after filter.	79
6.17	Backstepping experiment simulation parameters.	81
6.18	Backstepping experiment variable parameters.	82
6.19	Backstepping experiment results.	82
6.20	Dynamic change in objective function experiment simulation parameters.	84
6.21	Dynamic change in objective function experiment results.	84
6.22	Local optimization method simulation parameters.	86
6.23	Nelder-Mead experiment results.	86
6.24	Nelder-Mead experiment ranked results.	88
6.25	(1+1)-ES $\sigma_{initial}$ experiment variable parameters.	89
6.26	(1+1)-ES $\sigma_{initial}$ experiment results.	89
6.27	(1+1)-ES $\sigma_{initial}$ experiment ranked results.	89
6.28	(1+1)-ES stopping mechanism analysis.	91
6.29	(1+1)-ES gen_{max} experiment variable parameters.	91
6.30	(1+1)-ES gen_{max} experiment results.	91
6.31	(1+1)-ES gen_{max} experiment ranked results.	92
6.32	(1+1)-ES with tabu search experiment variable parameters.	93
6.33	(1+1)-ES with tabu search experiment results.	93
6.34	(1+1)-ES with tabu search experiment ranked results.	95

6.35 Simulated annealing experiment variable parameters.	95
6.36 Simulated annealing experiment results.	96
6.37 Simulated annealing experiment ranked results.	96
6.38 Room with objects model experiment simulation parameters.	98
6.39 Room with objects model experiment results.	99
6.40 Room with objects model experiment ranked results.	101
6.41 Study room model experiment simulation parameters.	102
6.42 Study room model experiment results.	102
6.43 Study room model experiment ranked results.	104
6.44 Laboratory model experiment simulation parameters.	105
6.45 Laboratory model experiment results.	106
6.46 Laboratory model experiment ranked results.	107

Abbreviations

NBV	N ext B est V iew
CAD	C omputer- A ided D esign
DOF	D egrees O f F reedom
PSS	P ositional S pace S urface
PSD	P ositional S pace D irections
UAV	U nmanned A erial V ehicle
RNE	R andom- S ampling- B ased N ext- B est V iew E xploration
RH-NBVP	R eceding H orizon- N ext B est V iew P laning
RRG	R apidly- E xploring R andom G raph
RRT	R apidly- E xploring R andom T ree
ARTS	A ccelerated R ay- T racing S ystem
ES	E volution S trategy
VRML	V irtual R eality M odeling L anguaje

Symbols

v	face of icosahedron
$height_{min}$	minimum height of the sensor
$height_{max}$	maximum height of the sensor
$safe_{dist}$	sensor safe distance
VM	voxel map
vx	x index of voxel map
vy	y index of voxel map
vz	z index of voxel map
$V_{unmarked}$	unmarked voxel
V_{empty}	empty voxel
$V_{occupied}$	occupied voxel
V_{border}	border voxel
$V_{occluded}$	empty voxel
$V_{occp\text{plane}}$	occp\text{plane} voxel
f	objective function
$f_{principal}$	principal factor of objective function
n_f	number of factors in objective function
n_{sf}	number of secondary factors in objective function
f_{area}	area factor
a_{ov}	proportion of overlapped area
a_{op}	proportion of occ\text{plane} area
a_{us}	proportion of unseen area
x	percentage of occ\text{plane} voxels
y	percentage of occupied voxels
z	percentage of unmarked voxels

f_{op}	specific part of area factor regarding occplane voxels
f_{ov}	specific part of area factor regarding occupied voxels
f_{uv}	specific part of area factor regarding unmarked voxels
f_{der}	derivative of proposed area factor
$f_{navigation}$	navigation factor
$dist_{3d}$	three dimensional distance
d_x	distance around X axis
d_y	distance around Y axis
d_z	distance around Z axis
$dist_{2d}$	two dimensional distance
NBV_x	X axis position from the next best view
$Sensor_x$	current X axis position from the sensor
NBV_y	Y axis position from the next best view
$Sensor_y$	current Y axis position from the sensor
d_{maxEnv}	maximum distance regarding the environment size
$size_x$	size around the X axis of the environment
$size_y$	size around the Y axis of the environment
d_{norm}	normalized two dimensional distance
d_{max}	maximum distance
f_{dist}	distance factor
f_{dist1}	first variant of distance factor
f_{dist2}	second variant of distance factor
e	energy
NBV_z	Z axis position from the next best view
$Sensor_z$	current Z axis position from the sensor
NBV_{θ}	tilt angle from the next best view
$Sensor_{\theta}$	current tilt angle from the sensor
NBV_{ψ}	pan angle from the next best view
$Sensor_{\psi}$	current pan angle from the sensor
dy_{norm}	normalized distance around Y axis
f_{energ1}	first variant of energy factor
f_{energ2}	second variant of energy factor
$mean_{quality}$	measurement variable for quality factor

$nw_{occupied}$	total number of occupied voxels in the entire voxel map
$f_{quality}$	quality factor
$nv_{occupied}$	total number of occupied voxels in the candidate view
$nv_{ocplane}$	total number of occplane voxels in the candidate view
f_{area2}	area factor after change of function
$ndim$	number of variables to optimize
p_i	i-th vertex of the simplex on Nelder-Mead algorithm
$eval()$	exhaustive search evaluation
p_o	centroid of simplex on Nelder-Mead algorithm
p_r	reflected point of simplex on Nelder-Mead algorithm
p_e	expanded point of simplex on Nelder-Mead algorithm
p_c	contracted point of simplex on Nelder-Mead algorithm
$f()$	objective function evaluation
gen	number of generations on (1+1)-ES algorithm
gen_{max}	maximum number of generations on (1+1)-ES algorithm
p_{parent}	parent from (1+1)-ES algorithm
$Sensor_v$	current icosahedron face v from the sensor
$p_{offspring}$	offspring from (1+1)-ES algorithm
σ	strength of mutation on (1+1)-ES algorithm
$random_{gauss}$	random number following a Gaussian distribution with mean 0 and standard deviation of 1
m	successful mutations on (1+1)-ES algorithm
c	defined parameter on (1+1)-ES algorithm
$\sigma_{initial}$	initial value of σ on (1+1)-ES algorithm
$tabu_{parent}$	parent from (1+1)-ES algorithm with tabu search
$tabu_{offspring}$	offspring from (1+1)-ES algorithm with tabu search
$tabu_{list}$	tabu list from (1+1)-ES algorithm with tabu search
$tabu_{span}$	banned generations for all tabu elements from (1+1)-ES algorithm with tabu search
$point_{ref}$	reference point for simulated annealing algorithm
$point_{new}$	new point for simulated annealing algorithm
$point_{opt}$	optimal point for simulated annealing algorithm
T_0	initial temperature for simulated annealing algorithm
T_k	current temperature for simulated annealing algorithm
k	current iteration for simulated annealing algorithm

L	maximum iterations for simulated annealing algorithm
$acceptance_{prob}$	acceptance probability for simulated annealing algorithm
$cool_{ratio}$	cooling ratio for simulated annealing algorithm
$\%retrieved$	total percentage retrieved from the original model
$triangles_{retr}$	number of retrieved triangles
$triangles_{total}$	total number of triangles in the VRML file
f_{motion}	total motion factor
$ranking_{incr}$	increasing ranking
$ranking_{decr}$	decreasing ranking
$value_{rank}$	value that will be ranked
$value_{max}$	maximum value within the rank
$value_{min}$	minimum value within the rank
β	optimal percentage of occplane voxels
γ	optimal percentage of occupied voxels
δ	optimal percentage of unmarked voxels
α	optimal percentage of overlapped area
ρ	minimum value of navigation factor
Ω	normalized orthodromic distance
Δd	orthodromic distance
ρ_{dist}	minimum value for distance factor
ψ	pan angle
θ	tilt angle
ψ_{norm}	normalized pan angle
θ_{norm}	normalized tilt angle
ρ_{energ}	minimum value for energy factor
ω_i	angle formed between the surface normal of the occupied voxel i and the sensor ray
ϵ_i	angle formed between the surface normal of the occplane voxel i and the sensor ray

Chapter 1

Introduction

This chapter introduces the concept of Next Best View by explaining its purpose, applications and provides a general representation of the process itself. Literature review is presented by including previous works that dealt with this topic highlighting their key contributions such as their objective functions, optimization methods and types of robots used. A research question is raised as well as its corresponding hypothesis in order to set the objectives of this work. Finally, the main contributions are shown and the methodology for all the project is proposed.

1.1 Background

Getting true digital representations of a physical object or environment can be a challenge when it comes to autonomous methods. Nowadays, CAD technologies are raising on importance among worldwide fabricators, so many objects produced in the industry are brought to reality by converting a 3D digital object to a physical piece. However, there is the opposite situation in which the object exists without a CAD model like the ones made by artisans, or even pieces that indeed were fabricated using digital models or blueprints that were kept by the fabricator, so it is not possible to access them. If we want to obtain the digital object, we need to have some scanning strategy to save time and resources that would be bigger if no strategy was used resulting in a random exhaustive scanning. This strategy is called Next Best View, which is defined as the process to select the view that would provide more information on the object or environment following some criteria, and repeating this process until the scan is complete. A visual representation of this process is shown in [Figure 1.1](#).

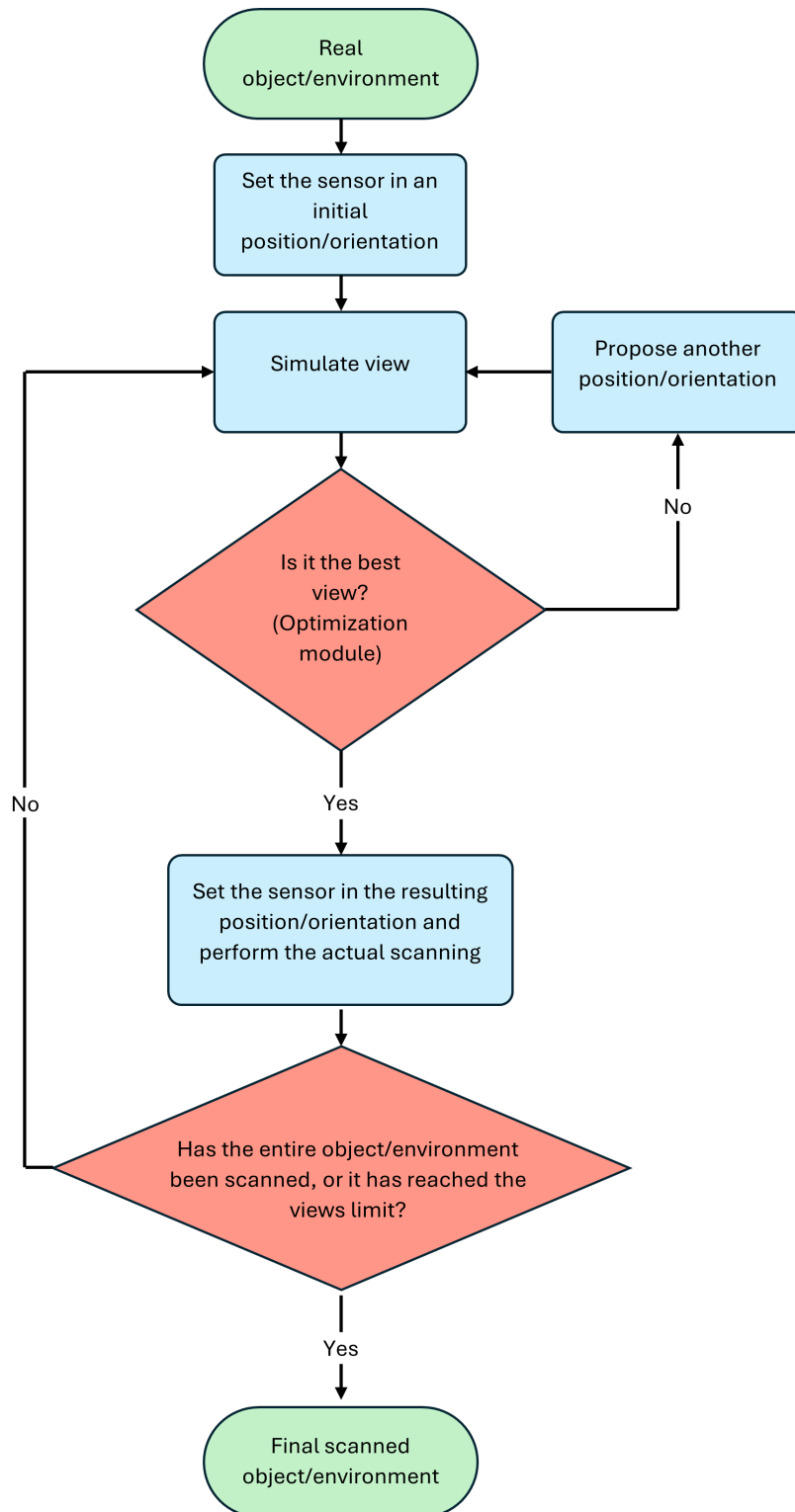


FIGURE 1.1: NBV process.

Some applications where this technique can be used are object digitization for reverse engineering such as a core die of the inlet of a diesel engine [1]. It can even be used for non-engineering applications such as criminology providing a reconstruction of a crime scene [2]. A medical

approach can also be set in which a whole scene is scanned, although instead of objects or terrain, a set of chronic wounds are reconstructed with the human body being the environment [3]. For industrial applications involving robotics, the NBV is used for environment exploration on autonomous mobile robots required to find and retrieve task-related objects to accomplish their tasks [4], and a more concrete example can be found on industrial facilities in which robots perform an inspection of the facility so that workers can avoid hazardous places when performing activities such as removing toxic substances, and repairing damaged machinery [5].

NBV can be considered as an optimization problem as we seek to find the best view given the available conditions; therefore, two key elements are the objective function and the optimization method used. The objective function sets the conditions and criteria to define how good a view, while the optimization method is an algorithm to reach the best view according to the requirements of the objective function. Both have a huge importance when designing the solution as the first defines the nature of the problem and the second provides the solution.

In this project we will use the NBV approach in order to perform a simulation of modeling an environment such as a the inside of a room, having multiple proposed models each having a different configuration of the room. The sensor will be inside the room and will be able to move freely around the available space.

1.2 Literature Review

1.2.1 Robots

One important element on the Next Best View problem is the type of robot used, as it will determine the degrees of freedom of the problem. The robot proposed in [6] and [7] is a simulated 5 DOF mobile robot with a range sensor, with the first two DOF being the displacement of the mobile base on X and Z axes, a third degree of freedom with a vertical bar that lifts the sensor around the Y axis, and the last two DOF being the pan and tilt angles of the range sensor. This 5 DOF simulated robot has the advantage that it reduces the problem on one degree of freedom by not considering the roll angle, making the optimization process easier and less time-consuming and without compromising a lot the range of possible views, even if the roll angle could provide more views if the horizontal and vertical apertures of the sensor were different as it could use the biggest aperture on any of the pan-tilt angles, it is still the less

essential degree of freedom. Another advantage is that this simulated robot could be easily replaced by an actual robot when applying NBV on real world scenarios as most of the mobile robots available that have pan-tilt heads omit roll angle.

We can even have less DOF as proposed on [8] where there are only 4 DOF. In order to reach such DOF, a Cyberware PS cylindrical scanner and a turntable are used. They divide their positional space into two: PSS and PSD. For the PSS is discretized into cells forming a cylinder and has 2 DOF while the PSD is a unique polar system with another 2 DOF adding up to the total of 4 DOF. As it has only 4 variables representing its positional space the computational time of the optimization process would be less than the previous robot and still not compromise the range of possible views due to the configuration of those 4 DOF as they are adapted on a new coordinate system that adapts to those specific DOF. However, we have the disadvantage that it is clearly made for object scanning meaning that the sensor is outside the object and the approach of this project is to scan an environment with the sensor being inside it.

Another approach can be seen on [9] and [10] where 6 DOF manipulators are used. The advantage of these manipulators is that they incorporate all the possible DOF meaning that there are no possible views that are ignored. However, one main disadvantage is that the displacement range of the sensor is limited to the kinematics of the manipulator as due to it being fixed to the ground and only its links have movement as it is intended for object scanning instead of an environment, plus having a more complex path planning that is specific to each model link configuration and dimensions, with [9] using an ABB Industrial Robot 2000 and [10] using a modular manipulator built of powercube modules by Schunk.

There are more extreme proposals such as the one on [11] where a humanoid robot is used. The first degree of freedom is the position of the free-floating body, the second being its orientation, and the last DOF represent each joint of the humanoid robot. They propose a HRP-2 robot that has 30 DOF (Head: 2 DoF; Arm: 6 DoF x 2; Hand: 1 DoF x 2; Waist: 2 DoF; Leg: 6 DoF x 2) [12], which adds up to a total of 32 DOF for the problem. Although it has the advantage that it can be used in both object and environment recognition as it can move around an object or inside a building, it is clear that it has a big amount of DOF which makes it heavy time-consuming and the path planning is even more complex than the manipulators from [9] and [10], and even UAVs (unmanned aerial vehicles) according to [2].

Not all mobile robots need to be ground vehicles, we can also have UAVs as proposed on [13], [14] and [15]. This UAVs have 6 DOF being able to move and rotate around all X,Y,Z axes.

Their main advantages are their big freedom to move around the environment and depending on their size they could enter small areas that could be almost impossible with a ground vehicle. However, the main disadvantage is their difficulty on path planning as well as their stability control.

1.2.2 Space discretization

The next important factor to take into consideration is the way in which the workspace is discretized. We understand for discretization the division of a continuous space into a discrete representation, and an easy to understand example is the division of a two-dimensional picture into small pixels. For this specific problem we have a three-dimensional space, meaning that we need three-dimensional discrete elements that are called voxels. There are two main methods to divide the space into voxels: voxel map and octree.

The first approach, the voxel map, is the simplest one. The 3D space is divided into tiny labeled cubes with all of them having the same size which is better called resolution. This approach has been widely used by many authors such as [3], [6], [11], [13], [16–23].

An octree is similar to the voxel map in terms of it being composed by labeled cubes. The main difference is that the cubes inside the octree have different resolution within each other, this is done by dividing some of the big cubes into smaller cubes while keeping other big cubes in their original size. Some works that use this octree approach are [24–27]. The work on [28] and [29] uses an octomap which is an octree with probabilistic occupancy estimation.

A visual representation of both voxel map and octree approaches can be seen on Figure 1.2 inspired by [30].

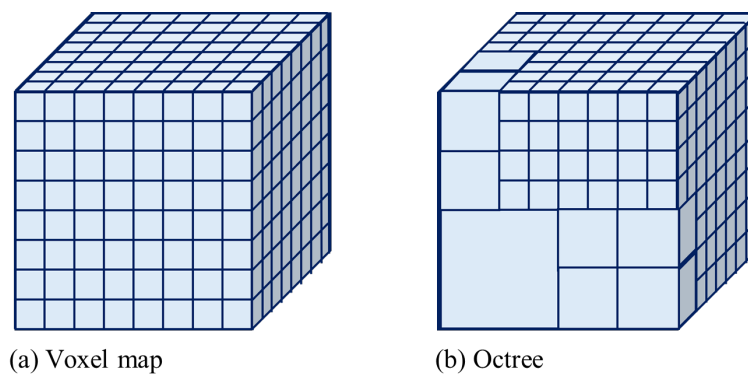


FIGURE 1.2: Voxel map and octree.

The main advantage of using the octree approach instead of a voxel map is that we are more protected against imperfect sensor readings as it increases the resolution on specific areas of the 3D space.

A hybrid method is proposed on [31] by initially using a voxel map, and after labeling all of the voxels only the ones that are occupied will have their resolution increased using octrees. They refer to this approach as semioctree and its advantage is that it increases the model resolution while still keeping it homogeneous.

These proposed discretization methods are three-dimensional although we can have 2D discretization as shown on [32]. They do not divide the 3D space into any discrete element, instead they perform the discretization after taking the 2D image of the object and divide it into pixels. Then, they build local feature histograms with probabilistic recognition based on the available photos. This has the inconvenient that a data base containing huge amounts of views of multiple objects is needed, making it use a lot of resources and forcing a prior knowledge of the model making it less generalized.

1.2.3 Objective function

The first step of the optimization process is the declaration of an objective function which will be used for setting up a criteria for deciding which view is better, and this function can have multiple factors.

The most essential is the area factor that analyzes the proportions of the view depending on the labels of the voxels from the voxel map or octree. It is possible to have the objective function with only this area factor as shown on [6].

Another factor that can be used is the quality one. The work by [6] besides from showing a case where only the area factor was used, it also included a second scenario where they add a quality factor. The proposal by [18] has also both factors although they refer to the area factor as visibility factor but is equivalent. The advantage of having this quality factor is that we are more protected from misleading sensing measurements as the rays from the sensor are more perpendicular to the surface although it can considerably increase the number of views.

Other works add an extra factor for distance. The objective function of [7] uses one factor that combines area and quality into a single equation and adds the distance factor considering

3D Euclidean distance. A similar approach can be seen on [33] where they use area, quality and distance factors. However, instead of using 3D Euclidean distance they use the normalized orthodromic distance. Finally, on [28] they consider an area factor, a distance factor using Euclidean distance and a positioning factor to make sure that the space is free for the robot to be there. The advantage from the distance factor is that it optimizes the movement of the robot which reduces the amount of power needed and from an economic point of view it saves resources.

1.2.4 Optimization method

Optimization method refers to the algorithm used for finding the best candidate view based on the objective function. Testing all the possible views would ensure that the absolute best is selected. However, this would be extremely time-consuming.

The work by [34] compares two RNE algorithms which propose random views systematically.

- The first is the RRT algorithm in which random samples are generated according to a tree and nodes structure. According to the authors, this first method was inspired by the RH-NBVP algorithm proposed by [15] which is also used on [35] and further improved on [36] due to its high computation time. This RRT algorithm is also used on [37], where a two-phase approach is proposed with the first one being the exploration stage using RRT and the second being a relocation stage.
- The second algorithm on their comparison is RRG which takes as a base the RRT algorithm but replaces the tree with a graph allowing an advanced connectivity between the nodes as well as making it asymptotically optimal according to [38]. This RRG algorithm is also used on [39].

Another approach is using the two-phase optimization method by [6], with the first being for the position of the robot using the Nelder-Mead simplex algorithm and the second being an exhaustive search on the pan-tilt space for the orientation of the sensor. The advantage of this approach over the RNE algorithms is that only the initial simplex is random, all the other steps do not include any type of randomization making it more repeatable.

A regression approach can be seen on [23] where they use a convolutional neural network algorithm to perform the regression for the position of the robot while the orientation is computed

using geometric reasoning. The main disadvantage of this approach is that a training for the convolutional neural network is needed and there may be the chance that such training needs to change according to the environment or objects used.

1.3 Research Question and Hypothesis

What is the impact of changing the objective function and the optimization method on the NBV problematic?

We will have three hypotheses: first that by changing the objective function with the combination of additional factors instead of just the principal area factor we can upgrade the results of the retrieved model; then that the changes in the parameters when tuning each optimization method have an impact on their results; and finally that we can obtain significant differences among our proposed methods.

1.4 Objectives

The general objective of this work is to retrieve an environment by providing a multi-factor objective function focused on both obtaining new information of the model and minimizing resources, with the implementation of a optimization method that reaches an optimal solution within an adequate computational time.

- Build a principal area factor aiming at retrieving the model.
- Formulate a distance factor to reduce the total distance covered by the robot.
- Add an energy factor focused on optimizing the movements of the sensor.
- Consider a quality factor to improve the voxels mean quality.
- Find the best arrangement of this factors into a single objective function.
- Provide an algorithm that finds an optimal solution and minimizes computational time.

1.5 Main contributions

The first main contribution of this work is the definition of a new dynamically changing objective function for the NBV problematic adapted to a 5-DOF mobile robot. We implement an area factor which has been used before, but the main novelty in our function is the way we deal with the motion of the robot by dividing it into two factors: distance factor for the 2D movement of the robot on the ground, and an energy factor for the angular and vertical movement of the sensor within the robot. We then contribute on the optimization methods by modifying a previously studied Nelder-Mead with exhaustive search algorithm to now omit the exhaustive search part and keep a pure Nelder-Mead method. We also test the objective function with an evolution strategy called (1+1)-ES where the "(1+1)" in its name makes reference to the fact that it only produces one offspring from one parent on every iteration and ES is an acronym for Evolution Strategy, and finally a simulated annealing algorithm, both unused on the NBV problematic to the best of our knowledge. For all of these algorithms we contribute by providing their parameter values after their corresponding tuning to be used on other future similar works. We also develop a backstepping technique to avoid getting trapped at local minima.

1.6 Methodology

The methodology of this work is composed of three phases plus some initial considerations as seen on Figure 1.3.

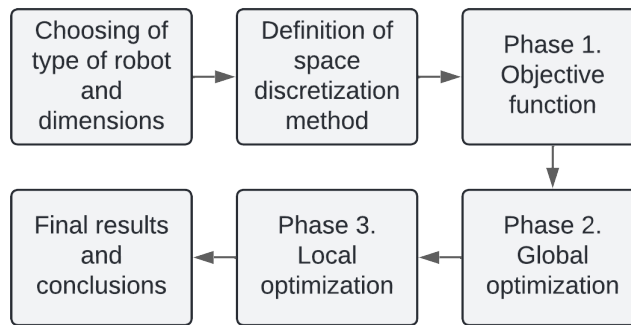


FIGURE 1.3: General methodology.

A diagram of the first phase corresponding to the objective function can be seen on Figure 1.4, where we divide it into three main sections: area, motion (distance and energy) and quality. For every factor we perform their formulation, experimental design and testing. Notice how they follow an order meaning that for testing the quality factor we need to have previously

tested motion and area factors, this was done in this specific way because if all the factors were introduced on a single experimental design we would end up with a huge number of combinations as every one of them has parameters that need to be experimented with. This objective function will be tested using the Nelder-Mead with exhaustive search optimization method and with a laboratory model as the environment in which the robot will move, where the final result of this phase is an objective function with the best tested arrangement of the defined factors.

Phase 1. Objective function

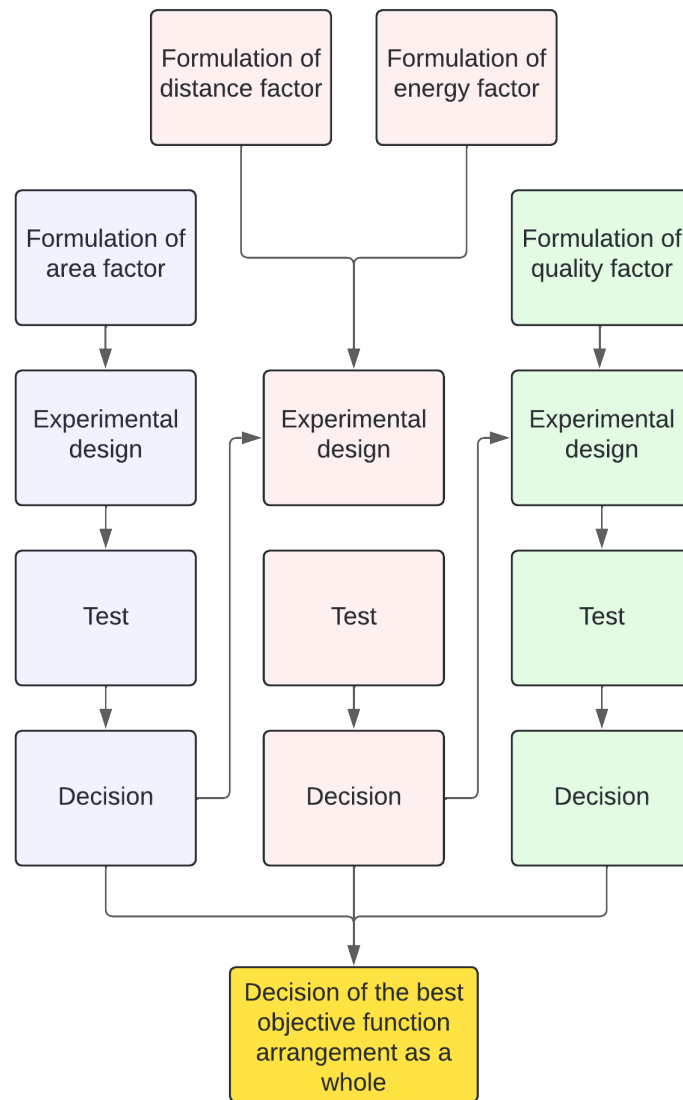


FIGURE 1.4: Phase 1 diagram.

Phase 2 defines two tasks to be performed on the global optimization process that are intended to improve our results by helping the robot to avoid local minima, and can be seen on Figure 1.5. This new tasks are tested using the Nelder-Mead with exhaustive search method inside a laboratory environment as the previous phase and with its resulting objective function. We first

define and test a backstepping technique and once we reaches its best parameters a dynamic change in the objective function is proposed and tested. The final result of this phase is are the best parameters for both of this tasks.

Phase 2. Global optimization

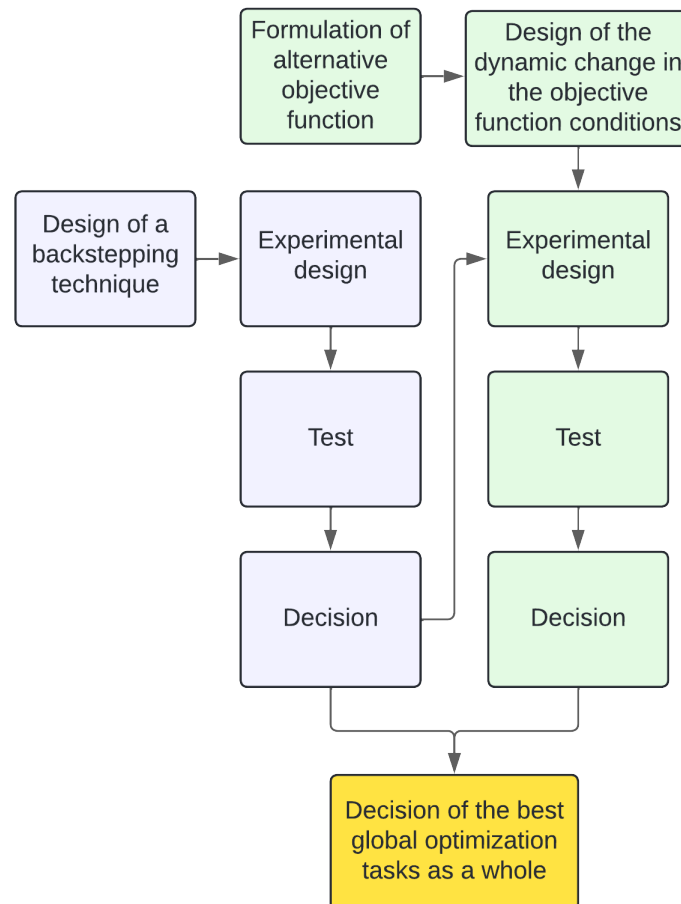


FIGURE 1.5: Phase 2 diagram.

Phase 3 is our last section and can be seen on Figure 1.6. The past phases used only the Nelder-Mead with exhaustive search as their optimization method, but in this phase more options are explored and tested, while keeping the objective function from the first phase and the global optimization tasks from the second phase. The laboratory environment will still be used for each individual test within each method, and for the final comparison between the best versions of the methods more test environments will be used to test the repeatability of this work.

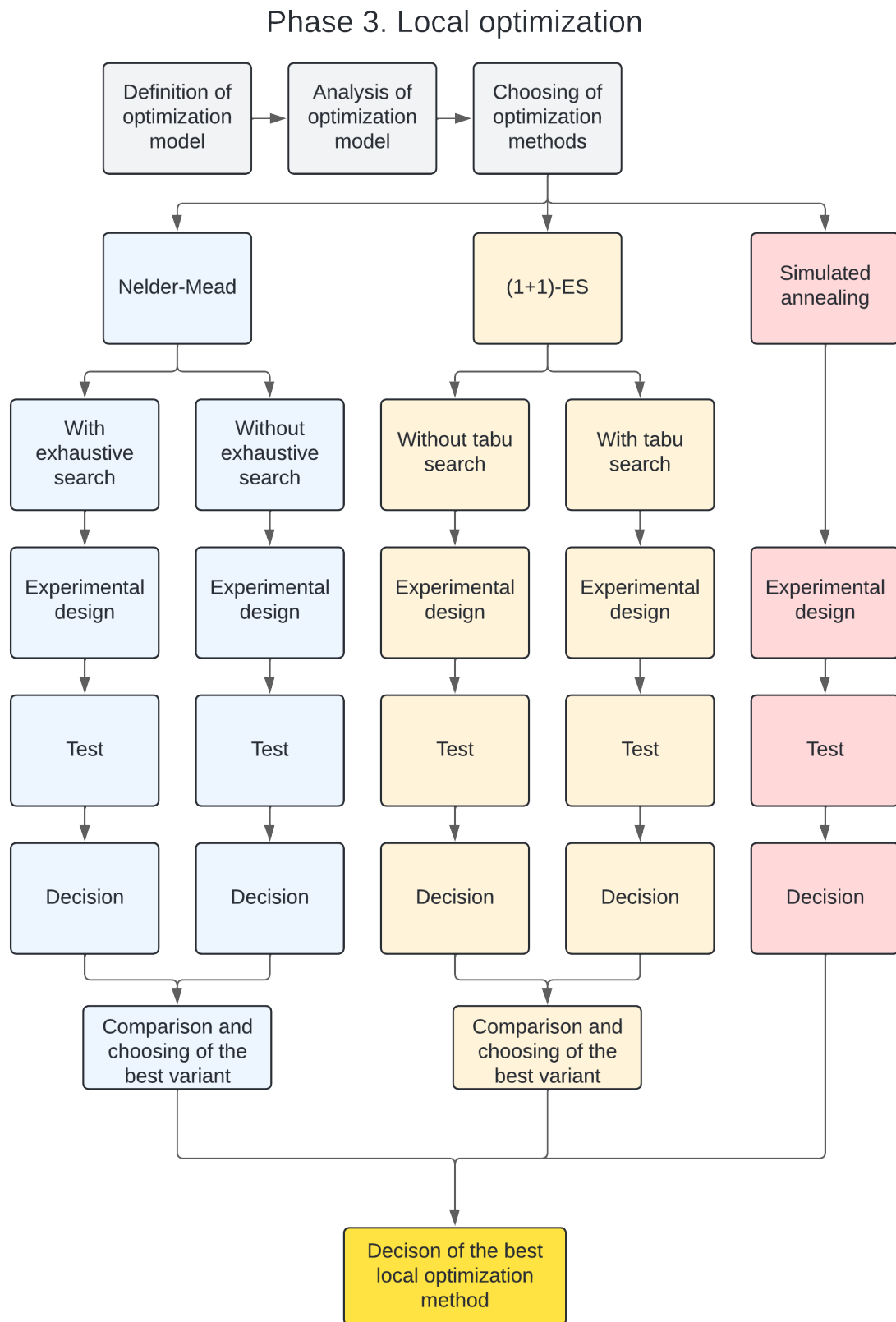


FIGURE 1.6: Phase 3 diagram.

Chapter 2

Initial considerations

This chapter establishes the initial considerations to be included in this project such as the type of robot and space discretization method used according to the options proposed on section 1.2. The objective function and optimization method will be presented and developed individually on further chapters.

2.1 Robot

According to the information presented on section 1.2, the most adequate robots for environment scanning are UAVs, humanoid and 5 DOF mobile robots. We rule out the consideration of using a humanoid robot due to its high number of DOF which unnecessarily increases the difficulty and time-consumption of the problem as we do not have the need of using such robot as it was the case on [11], we are interested in navigation for mobile robots.

From the remaining proposed options, we will choose a 5 DOF mobile robot instead of UAVs as it reduces the problem on one non-essential degree of freedom as well as having an easier control of its movement if the physical implementation is needed on future work. As we mentioned before, this 5 DOF mobile robot configuration is not exclusive to any brand and can be expressed as a general model which could even be home-made. A proposal can be seen on Figure 2.1.

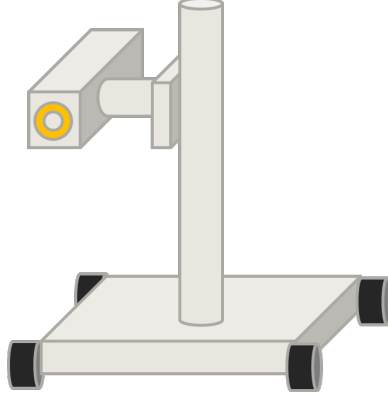


FIGURE 2.1: Proposed robot configuration.

2.1.1 Robot dimensions

Although it may seem trivial, the robot dimensions are crucial to its behaviour. As we are dealing with recovery of an environment, the robot used should be suitable for the given conditions. For example, if we are trying to retrieve a room which has a height of 2 meters, the robot should not exceed that height as for obvious reasons it would not even fit inside the room.

Even if we are working with different models of robots with different complexity, for our simulation we just need to indicate three dimensions which will generalize the robot. These three dimensions are:

- Minimum sensor height $height_{min}$
- Maximum sensor height $height_{max}$
- Sensor safe distance $safe_{dist}$

A graphical representation of these three measurements can be seen on Figure 2.2, where we notice that the minimum sensor height is the lowest it can move and maximum sensor height is the highest point it can get, while the sensor safe distance represents half of the robot width, and as we have only one measurement variable but two axis, it will correspond to the highest of the X or Z axes robot widths. One recommendation is to exaggerate a little this sensor safe distance by increasing it, as it will be used to form a cylinder around the robot used for collision checking, so it is better to have a tolerance around the robot by increasing this value. This sensor safe cylinder will be able to be turned on or off during the simulation, and is represented on Figure 2.3, where we can see that it has a radius equal to the sensor safe distance, and a height equal to the maximum sensor height without taking into consideration the minimum

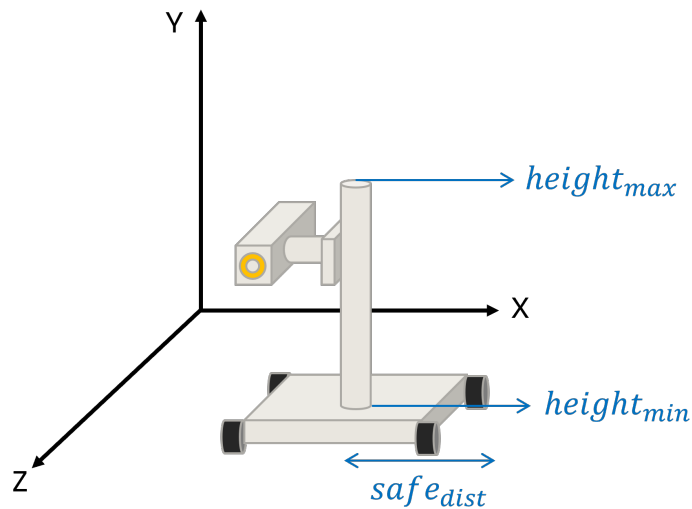


FIGURE 2.2: Robot dimensions.

sensor height as it goes from the ground because the base also needs to be considered inside the cylinder.

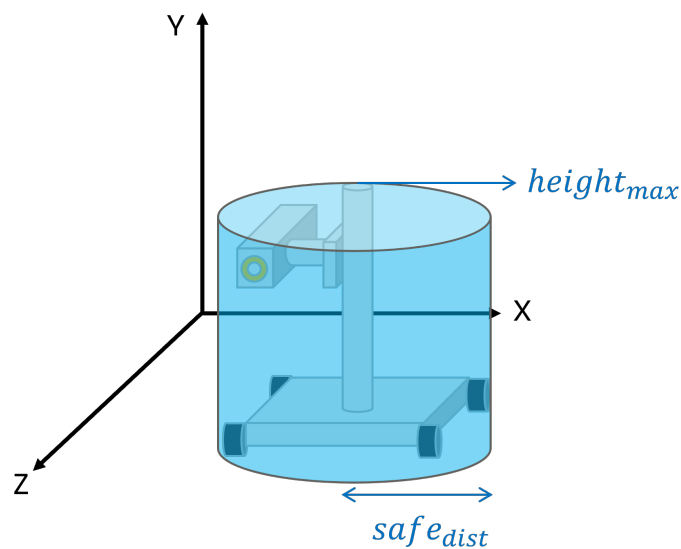


FIGURE 2.3: Safe cylinder.

2.1.2 Implementation of orientation icosahedron

Although we have 3 DOF for traslation and 2 DOF for orientation adding up to a total of 5, this only represent the physical DOF of the robot. If we made no further modification then the DOF of the optimization model would also be 5. However, we can add an orientation icosahedron as shown on [6] and [33]. This reduces the pan-tilt angles into just one variable v that goes from 0 to 19 and represents the face of the orientation icosahedron with each of them having an

associated set of pan-tilt angles; therefore, we reduce one degree of freedom of the optimization model and end up with a total of 4 DOF. A visual representation can be seen on Figure 2.4.

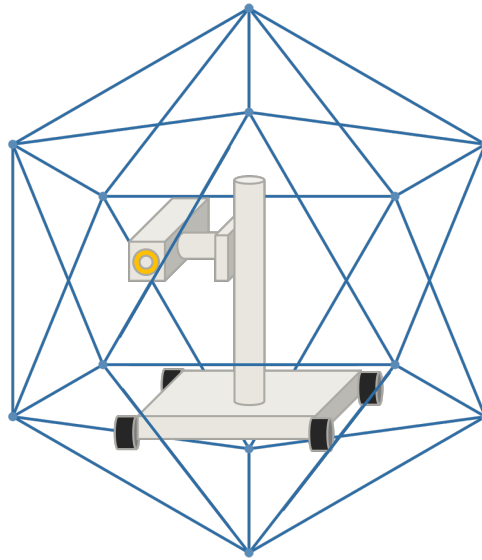


FIGURE 2.4: Orientation icosahedron.

The ray of the sensor on the face v of the icosahedron is projected on the normal vector of that face. The list of pan-tilt angles per each value of v are presented on Table 2.1.

TABLE 2.1: Pan-tilt angles of orientation icosahedron.

Icosahedron face v	Pan angle (degrees)	Tilt angle (degrees)
0	0	69.094843
1	-45	54.735611
2	-90	20.905158
3	90	20.905158
4	45	54.735611
5	69.094843	90
6	110.905158	90
7	135	54.735611
8	180	69.094843
9	180	110.905158
10	135	125.264391
11	90	159.094844
12	-90	159.094844
13	-45	125.264391
14	0	110.905158
15	45	125.264391
16	-69.094843	90
17	-110.905158	90
18	-135	54.735611
19	-135	125.264391

2.1.3 Sensor

The sensor used will be a simulated range sensor developed by [6] and used previously on [6, 7, 20, 23, 28, 29, 33]. It uses a 3D Bresenham algorithm [40] with also implementing the ARTS method from [41] to simulate ray tracing, and we can see that this type of sensor is widely used because it is a general representation and its characteristics can be assigned through parameters during simulation.

The first parameters are related to the range in which the sensor can detect the surface, and those are the maximum and minimum distance shown on Figure 2.5.

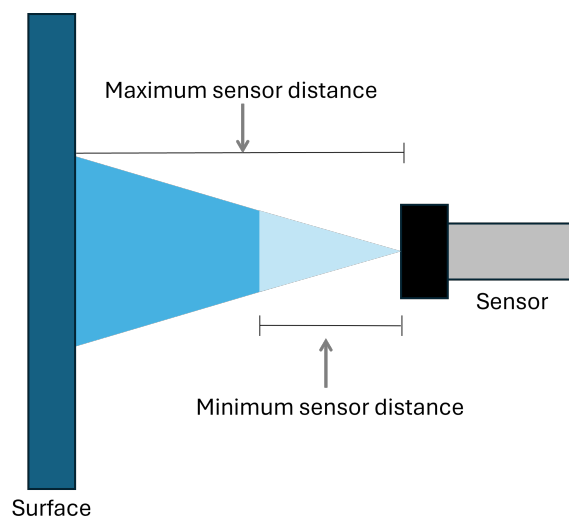


FIGURE 2.5: Sensor maximum and minimum distance.

The surface normal will not always be parallel to the ray of the sensor, there can be an inclination between those two so we define a maximum sensor view angle represented on Figure 2.7

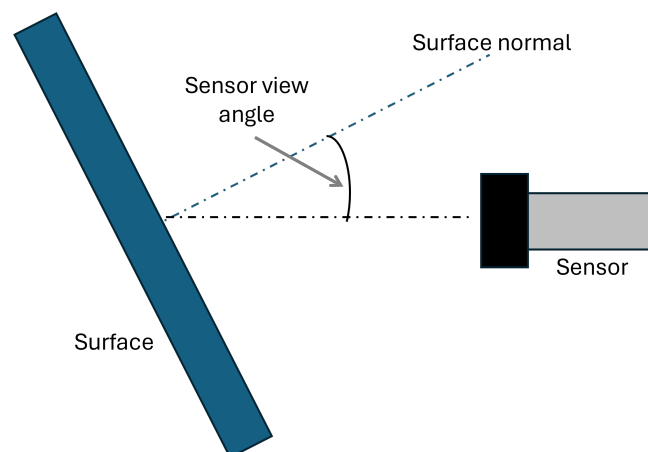


FIGURE 2.6: Sensor view angle.

Next, we have the parameters corresponding to the section of the surface that can be sensed as seen on Figure 2.6. Horizontal and vertical aperture angles are defined and this are projected creating a 2D image which is compounded of points and has horizontal and vertical dimensions.

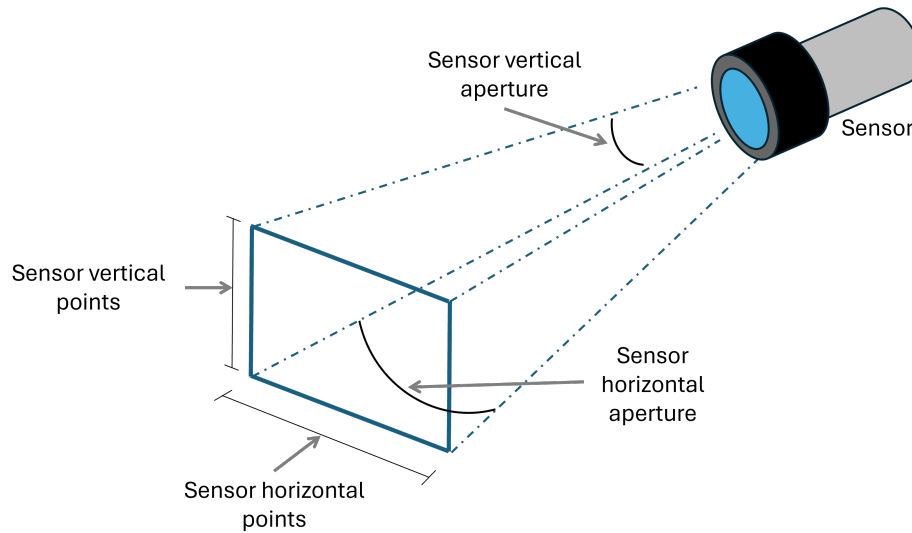


FIGURE 2.7: Sensor projected view.

Sensors in real life have slight errors that need to be considered on this simulated sensor as it is intended to imitate a real sensor. To cause this slight imperfections we add Gaussian noise with a mean of 0 and a standard deviation of 1, with an amplitude of our last parameter that represents the sensor noise deviation.

As a summary, all the sensor parameters are:

- Sensor view angle
- Minimum sensor distance
- Maximum sensor distance
- Sensor noise deviation
- Sensor horizontal aperture
- Sensor vertical aperture
- Sensor horizontal points
- Sensor vertical points

2.2 Space discretization

The two main 3D space discretization methods presented on section 1.2 were voxel map and octree. The main reason of using an octree was to reduce the probability of a misreading from the sensor, but as we are working in a simulation where we do not have the imperfections from the actual sensor there is no need to upgrade from voxel map to octree; therefore, we will use voxel map for space discretization.

2.2.1 Voxel map labeling

The traced rays of the sensor will pass through the voxels on its path and will determine their type according to the following classification:

- Unmarked voxel ($V_{unmarked}$): a voxel that has not yet been explored by the ray tracing, it is unknown if it has points of the environment inside it or not.
- Empty voxel (V_{empty}): a voxel that has been explored by the ray tracing and has no points of the environment inside it making it empty.
- Occupied voxel ($V_{occupied}$): a voxel that has been explored by the ray tracing, has points of the environment inside and is not adjacent to an empty voxel.
- Border voxel (V_{border}): a voxel that has been explored by the ray tracing, has points of the environment inside and is adjacent to an empty voxel.
- Occluded voxel ($V_{occluded}$): a voxel that is not adjacent to an empty voxel and is occluded by an occupied or border voxel making it unfeasible for the ray tracing, it is unknown if it has points of the environment inside it or not.
- Occplane voxel ($V_{ocplane}$): a voxel that is adjacent to an empty voxel and is occluded by an occupied or border voxel making it unfeasible for the ray tracing, it is unknown if it has points of the environment inside it or not.

The voxel map VM will have vx , vy and vz as indexes for all its voxels, and its update will be done after each view is taken.

Chapter 3

Objective function

This chapter introduces the concept of a objective function and establishes how it can be implemented in this particular problem, defining the different factors it will have and their individual mathematical modeling.

An objective function is a mathematical expression that is dependent on variables that characterize a phenomenon, and that its value depends on the desired behavior indicated while designing the function, with aims to have the highest result at the most desired values of its variables. A objective function can be made from multiple factors n_f that have the same importance as:

$$f = \sum_{i=1}^{n_f} f_i \quad (3.1)$$

or having a principal factor $f_{principal}$ and multiple secondary factors n_{sf} that have less importance than the principal one, as:

$$f = f_{principal} \left(\sum_{i=1}^{n_{sf}} f_i \right) \quad (3.2)$$

3.1 Area factor

In this section we propose a objective function only considering the area factor $f = f_{area}$. For now, the interest is to understand the impact of this area factor and tune its parameters to obtain better results. Here, the objective function will be

$$f = f_{area} \quad (3.3)$$

The first step is to select a objective function based on previous works as a base, with the objective function mentioned on [6] being:

$$f_{\text{area}} = (5a_{ov}^3 - 10.5a_{ov}^2 + 6a_{ov}) (1 - \frac{1}{2} |a_{op} - a_{us}|) \quad (3.4)$$

where a_{ov} is the proportion of overlapped area, a_{op} the proportion of occplane area and a_{us} the proportion of unseen area.

The one on [20] can be summarized as:

$$f_{\text{area}} = f(x, \beta) + f(y, \gamma) \quad (3.5)$$

with β being the optimal percentage of occplane voxels, x the percentage of occplane voxels, γ the optimal percentage of occupied voxels and y the percentage of occupied voxels, formed by:

$$f(x, \beta) = \begin{cases} f_1(x, \beta), & x \leq \beta \\ f_2(x, \beta), & x > \beta \end{cases} \quad (3.6)$$

$$f(y, \gamma) = \begin{cases} f_1(y, \gamma), & y \leq \gamma \\ f_2(y, \gamma), & y > \gamma \end{cases}$$

where

$$f_1(x, \beta) = -\frac{2}{\beta^3}x^3 + \frac{3}{\beta^2}x^2$$

$$f_2(x, \beta) = -\frac{2}{(\beta-1)^3}x^3 + \frac{3(\beta+1)}{(\beta-1)^3}x^2 - \frac{6\beta}{(\beta-1)^3}x + \frac{3\beta-1}{(\beta-1)^3} \quad (3.7)$$

$$f_1(y, \gamma) = -\frac{2}{\gamma^3}y^3 + \frac{3}{\gamma^2}y^2$$

$$f_2(y, \gamma) = -\frac{2}{(\gamma-1)^3}y^3 + \frac{3(\gamma+1)}{(\gamma-1)^3}y^2 - \frac{6\gamma}{(\gamma-1)^3}y + \frac{3\gamma-1}{(\gamma-1)^3}$$

A general approach shown on [7] being:

$$f_{\text{area}} = \begin{cases} f_{ov} + f_{op}, & f_{op} \geq f_{uv} \\ f_{ov} + f_{uv}, & f_{op} < f_{uv} \end{cases} \quad (3.8)$$

where

$$\begin{aligned}
 f_{op} &= \frac{-2x^3+3\beta x^2-3\beta+2}{\beta^3-3\beta+2} \\
 f_{ov} &= \frac{-2y^3+3\gamma y^2-3\gamma+2}{\gamma^3-3\gamma+2} \\
 f_{uv} &= \frac{-2z^3+3\delta z^2-3\delta+2}{\delta^3-3\delta+2}
 \end{aligned} \tag{3.9}$$

with f_{op} being the specific part of the function regarding occplane voxels, β being the optimal percentage of occplane voxels, x the percentage of occplane voxels, f_{ov} being the specific part of the function regarding occupied voxels γ the optimal percentage of occupied voxels, y the percentage of occupied voxels, f_{uv} being the specific part of the function regarding unmarked voxels δ the optimal percentage of unmarked voxels and z the percentage of unmarked voxels.

Finally, another two proposals shown also on [7] had already implicit quality factors that were omitted for this section as the goal is to compare only area factors, so those functions without quality factors are:

$$\begin{aligned}
 f_{\text{utility1}} &= (-1.420455x^3 + 0.426136x^2 + 0.994318) \\
 &\quad + (-2.155789y^3 + 1.212632y^2 + 0.943158) \\
 &\quad + (-2.155789z^3 + 1.212632z^2 + 0.943158) \\
 &\quad + (-100w^2 + 10w + 0.75)
 \end{aligned} \tag{3.10}$$

and

$$\begin{aligned}
 f_{\text{utility2}} &= (-2.31482x^3 + 1.38889x^2 + 0.925926) \\
 &\quad + (-4.80769y^3 + 4.32692y^2 + 0.480769) \\
 &\quad + (-4.80769z^3 + 4.32692z^2 + 0.480769)
 \end{aligned} \tag{3.11}$$

with x , y , z and w being the ratios of occupied, occplane, unmarked and border voxels in the view. The main difference is that the first utility function f_{utility1} takes into account border voxels and in the second utility function f_{utility2} that term is removed because they represent a very small percentage of the totals.

A comparison was made using the different objective functions shown before, noticing the following key features:

- The one shown in Equation 3.4 has a more simple and manipulable structure than the ones in Equations 3.5 - 3.7 and Equations 3.8, 3.9, that both have a separate function

for each area percentage variable, and although the two from Equations 3.10, 3.11 do not consider explicitly a function for each area percentage variable but instead they are integrated into a main function that adds them up, they are still difficult to manipulate as each area percentage variable is a cubic equation by themselves.

- The ones in Equations 3.5 - 3.7 and Equations 3.8, 3.9 are already in a general form expressed by variables.
- The one in Equations 3.5 - 3.7 is a piecewise function in each of its sections, so two different expressions need to be considered for each section.
- The function in Equations 3.8, 3.9 changes depending on the occluded plane and occupied voxels percentages, it has a different expression when one of them is bigger than the other, making it piecewise but only in its final form and not in every section like Equations 3.5 - 3.7.
- The function in Equations 3.5 - 3.7 considers only occupied and occplane area, while the ones on Equation 3.4, Equations 3.8, 3.9 and Equations 3.10, 3.11 also consider unseen area, with Equation 3.10 being the only one to consider border boxels.

It was decided that the one that will be used as a base for the new proposed function will be the expression shown in Equation 3.4 because as seen on the previous comparison, it is the one which is more manipulable and we can transform it into a function that depends on a parameter α representing the optimal percentage of overlapped area. The main contributions of this section are transforming that objective function into an expression dependent on that parameter α and finding its best value.

For transforming the function to a general expression, a deep understanding of its behavior is needed, which can be obtained with the characteristics provided on [6]:

- A unique maximum when $a_{ov} = 0.4$ and $a_{op} = a_{us}$, with $a_{ov} + a_{op} + a_{us} = 1$.
- A local minimum at $a_{ov} = 1$ with a value greater than zero.
- Zero at $a_{ov} = 0$.
- Decrease as a_{op} differs from a_{us} .

This conditions are set to favour the following behaviour:

1. Providing overlap with previously acquired views for fine registration of the data.
2. Eliminating occlusion plane areas.
3. Observing new unseen areas.
4. Favouring at the same time the sensing of occlusion plane and unseen areas.

We can plot the resulting area factor function on a three-dimensional plot shown on Figure 3.1, where we represent the result of the area factor as a fourth dimension labeled by colors, where the maximum value of f_{area} is located at the yellowest coordinate.

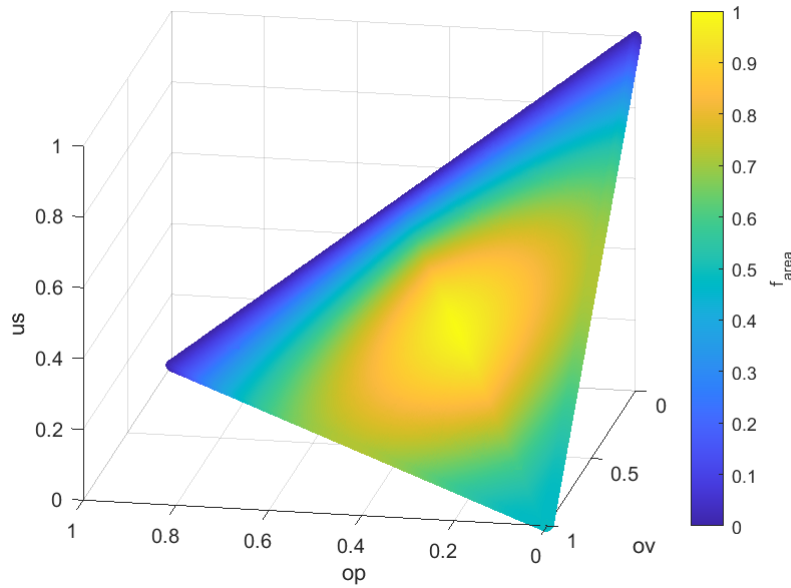


FIGURE 3.1: Area factor function.

We want to keep those behaviours but also have some control of the function by adding parameters to the optimal percentages of overlapped, unseen and ocplane area. The first characteristic is the only one that needs to be changed, and it can be done by just replacing the 40% of the overlapped area by a variable that will be called α , which represents the optimal percentage of overlapped area (represented as a decimal number from 0 to 1), as the unseen and ocplane areas are dependent of overlapped area by the restrictions $a_{ov} + a_{op} + a_{us} = 1$ and $a_{op} = a_{us}$,

having the unique maximum at:

$$a_{ov} = \alpha$$

$$a_{op} = \frac{1-\alpha}{2} \quad (3.12)$$

$$a_{us} = \frac{1-\alpha}{2}$$

The first step would be to identify the critical points of the function, being $a_{ov} = 1$ a minimum and $a_{ov} = \alpha$ a maximum. It is known that the critical points of a function are the roots of its derivative, so we can form a polynomial expression with these two points, being $(a_{ov} - 1)(a_{ov} - \alpha)$ which can be expressed as an auxiliary function f_{der} :

$$f_{der} = a_{ov}^2 - (\alpha + 1)a_{ov} + \alpha \quad (3.13)$$

that represents the derivative of the actual function, so the next step is to integrate it:

$$\int f_{der} = \frac{a_{ov}^3}{3} - \frac{(\alpha + 1)a_{ov}^2}{2} + a_{ov}\alpha + c \quad (3.14)$$

As one of the restraints is to have a value of zero at $a_{ov} = 0$, the value of c needs to be zero, otherwise that restraint would not be fulfilled.

$$f_{der} = \frac{2a_{ov}^3 - 3(\alpha + 1)a_{ov}^2 + 6a_{ov}\alpha}{6} \quad (3.15)$$

Evaluating this function at $a_{ov} = \alpha$ the maximum would be obtained, being:

$$\frac{3\alpha^2 - \alpha^3}{6} \quad (3.16)$$

Evaluating this function at $a_{ov} = 1$ the local minimum would be obtained, being:

$$\frac{3\alpha - 1}{6} \quad (3.17)$$

One of the characteristics needed is to have a local minimum greater than zero, so we set Equation 3.17 greater than zero to obtain the minimum value to fulfill that condition:

$$\frac{3\alpha - 1}{6} > 0 \quad (3.18)$$

obtaining $\alpha \leq \frac{1}{3}$ as the range of values for alpha that do not fulfill this restraint. It will be

established that alpha needs to be greater than this value. It is proposed that the maximum will have a value of 1 to normalize it, so the function is divided by Equation 3.16 to change its scale, obtaining:

$$\frac{2a_{ov}^3 - 3(\alpha + 1)a_{ov}^2 + 6a_{ov}\alpha}{3\alpha^2 - \alpha^3} \quad (3.19)$$

Testing this function with $\alpha = 0.4$ as originally proposed on [6], we obtain a maximum of 1 at $a_{ov} = 0.4$ and a local minimum of 0.4807 at $a_{ov} = 1$, with a global minimum of 0 at $a_{ov} = 0$. The last conditions that need to be fulfilled are adding $a_{op} = a_{us}$ to the global maximum, and decreasing as a_{op} differs from a_{us} , which both can be implemented keeping the factor $(1 - \frac{1}{2}|a_{op} - a_{us}|)$ used in [6], making the final area-based objective function:

$$f_{area} = \left(\frac{2a_{ov}^3 - 3(\alpha + 1)a_{ov}^2 + 6a_{ov}\alpha}{3\alpha^2 - \alpha^3} \right) \left(1 - \frac{1}{2}|a_{op} - a_{us}| \right), \quad \alpha \in \left(\frac{1}{3}, 1 \right] \quad (3.20)$$

With a_{ov} , a_{op} and a_{us} being calculated as:

$$a_{ov} = \frac{n_{occupiednotborder} + n_{border}}{n_{total}} \quad (3.21)$$

$$a_{op} = \frac{n_{ocplane}}{n_{total}} \quad (3.22)$$

$$a_{us} = \frac{n_{unmarked} + n_{occluded}}{n_{total}} \quad (3.23)$$

where n_{total} represents the total number of voxels in the current image, $n_{occupiednotborder}$ the number of occupied voxels, n_{border} the number of border voxels, $n_{ocplane}$ the number of occ-plane voxels, $n_{unmarked}$ the number of unmarked voxels and $n_{occluded}$ the number of occluded voxels. The reason why on Equation 3.21 both $n_{occupiednotborder}$ and n_{border} are together is because although they are considered as different types of voxels inside the program due to their differences on their labeling as one is a variation of the other, for the purpose of this function they can be taken into account as just one category because we do not have any specific term containing border voxels, there is no point in keeping them separated. The same applies to Equation 3.23 where $n_{unmarked}$ and $n_{occluded}$ are both considered as unseen area.

3.2 Motion factors

If we only considered the area factor proposed on the previous section, it will be expected to have the best performance regarding the recovered percentage of the scene as it would have just one

priority being the voxels information of the picture. However, if we intend to implement this on a real robot we will not have infinite resources, that is why motion factors are needed. Previous works like [33] managed to deal with this problem by adding a navigation factor $f_{navigation}$ that considered the normalized orthodromic distance. They define orthonomic distance as the shortest distance between two points on a sphere, and normalize it by dividing it by the maximum distance which is the one from two opposite points and can be seen as the diameter of the sphere.

$$f_{navigation} = (\rho - 1)\Omega^2 + 1 \quad (3.24)$$

where ρ is a parameter to indicate which will be the minimum of the function, and Ω is the normalized orthodromic distance calculated as:

$$\Omega = \frac{\Delta d}{diameter} \quad (3.25)$$

where Δd is the orthonomic distance. However, this approach can not be possible in this project because we are dealing with environment scanning win which the sensor is inside the environment, and the work by [33] is about object scanning in which the sensor is outside the object and can move around it with a spherical range, that is why orthonomic distance is used.

Another approach can be seen on [7] where although they do not consider a motion factor inside the objective function, they still calculate a distance in order to graph it. That distance is calculated as:

$$dist_{3d} = \sqrt{d_x^2 + d_y^2 + d_z^2} \quad (3.26)$$

where d_x is the distance around the X axis, d_y is the distance around the Y axis and d_z is the distance around the Z axis. The three of them are expressed in meters as well as the main distance $dist_{3d}$. What this distance represents is the movement of the sensor in all three dimensions, however, this may overgeneralize the movement of the sensor as it may not have the same impact to move the entire robot on X and Z axis or to move just the sensor around the link on the Y axis, we can go further and separate this into two factors:

- Distance factor: Considers only the movement that the base of the robot would need to make
 - Distance around X axis
 - Distance around Z axis

- Energy factor: Considers only the movement that the sensor would need to make around the robot itself
 - Distance around Y axis
 - Pan angle (angle around Y axis)
 - Tilt angle (angle around X axis)

This does not only help to separate the distance from the sensor and the robot base, but also to include the pan-tilt angles into the new energy factor. On Figure 3.2 we include visual representation of this two factors.

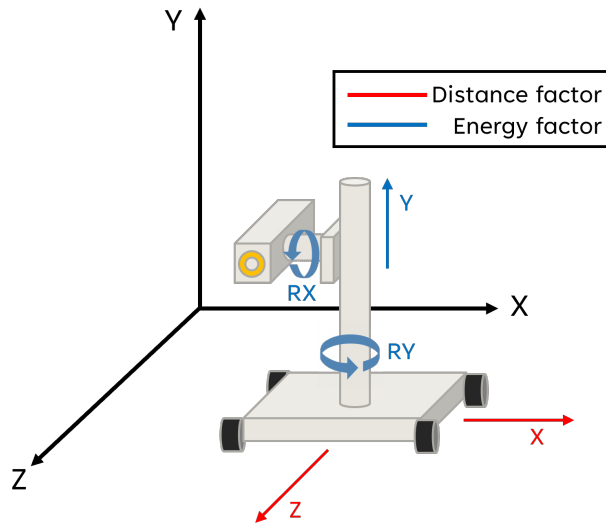


FIGURE 3.2: Distance and energy factors on proposed robot.

3.2.1 Distance factor

This distance factor gives priority to views in which the displacement of the base of the robot is smaller. First we need to have a measurable variable that will be used inside the factor itself. This variable needs to be the distance in X and Z axis as mentioned before, and we can use as a base Equation 3.26 and adjust it to our needs by removing the d_y term:

$$dist_{2d} = \sqrt{d_x^2 + d_z^2} \quad (3.27)$$

where d_x is the distance around the X axis and d_z is the distance around the Z axis. Both of them are expressed in meters as well as the main distance $dist_{2d}$. Distances around each axis

need to be calculated as they are not explicit inside the program, it can be done by subtracting the current position to the NBV position:

$$d_x = NBV_x - Sensor_x$$

$$d_z = NBV_z - Sensor_z$$
(3.28)

where NBV_x is the X axis position from the next best view, NBV_z the Z axis position from the next best view, $Sensor_x$ the current X axis position from the sensor and $Sensor_z$ the current Z axis position from the sensor

Finally, once we have this distance, it needs to be normalized because we need to have our objective function between 0 and 1 always, and for that to happen each factor needs to be on the same range, so we need a maximum distance and a minimum distance. The minimum distance should be 0, as there can be occasions in which the base of the robot does not move and is valid as the sensor can still move around the robot itself. However, the maximum distance is hard to define, as not all the environments have the same size. One solution that was considered was to read the parameters from the file of the desired model and obtain a maximum distance d_{maxEnv} regarding the environment as:

$$d_{maxEnv} = \sqrt{size_x^2 + size_z^2}$$
(3.29)

where $size_x$ is the size around the X axis of the environment and $size_z$ the size around the Z axis. Both of them are expressed in meters as well as d_{maxEnv} . However, this proposal had two main disadvantages:

- This would only apply when simulating where we have access to all the parameters of the file, but if we wanted to take this further to a real environment with a real robot, we would not have all this information available.
- At bigger environments the maximum distance would be bigger, and it could be high enough to reduce the impact of this distance factor, as it would take bigger differences within distances to really produce an effect. For example, if the maximum distance was 50 meters, and there are two possible views one where it would move 1 meter and the other 5 meters, when dividing these two by the maximum distance we would not have a noticeable effect, but at practice the difference between 1 meter and 5 meters is important.

For this reasons, it was decided that the maximum distance needs to be a constant value d_{max} expressed in meters regardless of the environment size, and it is considered that $d_{max} = 10m$ is a great value that adjusts to a great range of environment sizes without affecting its impact on the objective function. The normalized distance d_{norm} will be dimensionless, and be calculated as:

$$d_{norm} = \frac{dist_{2d}}{d_{max}} \quad (3.30)$$

Once we have the normalized distance, now the function for the factor needs to be created. One consideration that needs to be made is that as we defined d_{max} as a constant value 10 that does not depend on the environment size, it can happen that on some cases $dist_{2d}$ is bigger than d_{max} so d_{norm} ends up being greater than 1, for this reason our function will only be valid for $d_{norm} \leq 1$, and for $d_{norm} > 1$ it will be 0 as we do not want movements greater than d_{max} .

We can use as a base Equation 3.24 and adapt it, that although it was made considering the normalized orthodromic distance of Equation 3.25, we can relate it to our distance d_{norm} . The main characteristics defined on [33] are:

1. The candidate view with the shortest distance must have the maximum value
2. The candidate view with the largest distance must have a minimum value given by ρ
3. A candidate view with a larger distance than other must have a smaller value.
4. At short distances, the value of the function must decrease more slowly than at large distances.

The first three constraints seem to work with our desired behaviour, so we will keep them. However, the last constraint may have a similar effect to the one described before where some distances that are not big enough would not have an impact on the function. To deal with this, we can propose a linear behaviour instead, which can be done by removing the squaring of the distance. We will keep both versions and the general distance factor f_{dist} will have these two variants, we will make their corresponding tests later to see which one has a better behaviour. As for the minimum value ρ , they established that it needs to be between 0 and 1, but if it will be the smallest value of our factor it would not make sense to have it big. We propose a value of 0.1 to still keep a wide range of the factor while still avoiding the possibility to get 0. We will rename it to ρ_{dist} to avoid nomenclature problems, so it will be $\rho_{dist} = 0.1$.

The first option for the distance factor f_{dist1} , which takes into account the second-degree polynomial for bigger decrease at higher distance values, will be:

$$f_{dist1} = \begin{cases} (\rho_{dist} - 1)d_{norm}^2 + 1, & d_{norm} \leq 1 \\ 0, & d_{norm} > 1 \end{cases}, \quad \rho_{dist} = 0.1 \quad (3.31)$$

The second option for the distance factor f_{dist2} , which takes into account the first-degree polynomial for linear change, will be:

$$f_{dist2} = \begin{cases} (\rho_{dist} - 1)d_{norm} + 1, & d_{norm} \leq 1 \\ 0, & d_{norm} > 1 \end{cases}, \quad \rho_{dist} = 0.1 \quad (3.32)$$

On Figure 3.3 we can see a graphical representation of both variants of the distance factor f_{dist} and we can identify the following characteristics:

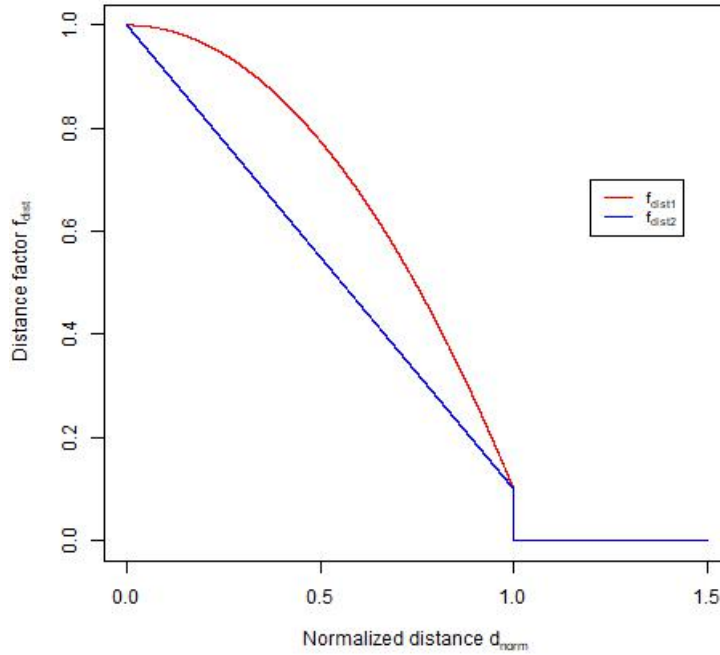


FIGURE 3.3: Comparison of distance factors.

- Both have their maximum $f_{dist} = 1$ when $d_{norm} = 0$.
- Both have their local minimum $f_{dist} = 0.1$ when $d_{norm} = 1$.
- Both have their global minimum $f_{dist} = 0$ when $d_{norm} > 1$.

- f_{dist1} has a structure of a second-degree polynomial structure.
- f_{dist2} has a structure of a first-degree polynomial structure.
- f_{dist1} has higher values than f_{dist2} .

On Chapter 6 we will test these two variants and determine which one gives us better results.

3.2.2 Energy factor

This energy factor gives priority to views in which the movement of the sensor around the robot itself is smaller. First we need to have a measurable variable that will be used inside the factor itself. This variable needs to be the distance around Y axis d_y expressed in meters, as well as the pan angle ψ which is the angle around the Y axis and tilt angle θ which is the angle around X axis, both expressed in degrees. As we are combining linear and angular movements, we can not directly apply a direct formula like the Euclidean distance formula of Equation 3.26 or Equation 3.27. However, we can propose a symbolic phenomenon that we will call "energy" represented by the term e , and although the term energy is already an existing quantitative property on physics, we will use it to describe the movement of our sensor around the robot, as this movement is not a concrete distance but still consumes energy from the robot, whether it is electrical, pneumatic or hydraulic depending on the robot. Each of the three variables used for measuring the energy need to be normalized for two main reasons:

- We need to have our objective function between 0 and 1 always, and for that to happen each factor needs to be on the same range, so for the energy factor to go from 0 to 1 we could calculate it without normalizing every variable, and once we have the complete energy normalize it, but in this case we do not have the same range on every variable, we can not define a generalized normalization.
- As we are mixing linear and angular movement, we have different units for our variables, with d_y being measured in meters and the angles ψ and θ in degrees. We want to normalize each variable to make them dimensionless so that we can mix them without any problem and our final variable e will also be dimensionless.

First, each variable without normalization needs to be calculated as they are not explicit inside the program, it can be done as:

$$\begin{aligned} d_y &= NBV_y - Sensor_y \\ \theta &= NBV_\theta - Sensor_\theta \\ \psi &= NBV_\psi - Sensor_\psi \end{aligned} \tag{3.33}$$

where NBV_y is the Y axis position from the next best view, NBV_ψ the pan angle from the next best view, NBV_θ the tilt angle from the next best view, $Sensor_y$ the current Y axis position from the sensor, $Sensor_\psi$ the current pan angle from the sensor and $Sensor_\theta$ the current tilt angle from the sensor.

For normalizing each variable, we need their individual minimum and maximum distance and a minimum distance. The minimum distance around the Y axis should be the minimum height of the sensor $height_{min}$, while the higher limit should be the maximum height of the sensor $height_{max}$, both of them in meters. The minimum pan angle should be -180° while the maximum should be 180° , however, both of them should be considered as maximum because we will be interested in the absolute value as both positive and negative movements consume the same energy, so the minimum should be 0° and the maximum 180° . This same analysis is applied to the tilt angle that will have the same range from 0° to 180° .

For normalizing d_y we just need to divide it by its maximum possible value, and it will be a dimensionless variable:

$$dy_{norm} = \frac{d_y}{height_{max}} \tag{3.34}$$

For normalizing ψ and θ we first need to divide them by their maximum possible value, and then obtain their absolute value due to the situation of both positive and negative rotations making the same impact to the energy. Both normalized variables will be dimensionless:

$$\begin{aligned} \psi_{norm} &= \left| \frac{\psi}{180^\circ} \right| \\ \theta_{norm} &= \left| \frac{\theta}{180^\circ} \right| \end{aligned} \tag{3.35}$$

Once we have each normalized variable, we can create our energy e . We can make an average of the three variables as all of them are dimensionless and have the same importance.

$$e = \frac{dy_{norm} + \psi_{norm} + \theta_{norm}}{3} \quad (3.36)$$

Now we need to define the function for the energy factor.

We can use as a base Equations 3.31, 3.32 that although they were related to distance, we can apply their same behaviour but now with energy. Their main characteristics were:

1. The candidate view with the smallest independent variable must have the maximum value
2. The candidate view with the largest independent variable must have a minimum value given by ρ
3. A candidate view with a larger independent variable than other must have a smaller value.
4. On Equation 3.31, at short independent variable values, the value of the function must decrease more slowly than at large independent variable values.
5. On Equation 3.32, the relation within the dependent and independent variable should be linear.
6. The function should be piecewise, with the main expression being only used when the independent variable is smaller or equal than 1, otherwise the function should be 0.

The first 5 constraints seem to work with our desired behaviour, so we will keep them. However, the last constraint is no longer necessary as our normalized variable e is completely fixed between 0 and 1 due to fixed physical characteristics of the sensor, so we will not have a case in which the energy is greater than 1, which was possible in the distance d_{norm} . As for the minimum value ρ , we proposed a value of 0.1 for ρ_{dist} to still keep a wide range of the factor while still avoiding the possibility to get 0. We will keep the same value for energy creating a new variable ρ_{energ} to avoid nomenclature problems, so it will be $\rho_{energ} = 0.1$.

The first option for the energy factor f_{energ1} , which takes into account the second-degree polynomial for bigger decrease at higher energy values, will be:

$$f_{energ1} = (\rho_{energ} - 1)e^2 + 1, \quad \rho_{energ} = 0.1 \quad (3.37)$$

The second option for the energy factor f_{energ2} , which takes into account the first-degree polynomial for linear change, will be:

$$f_{energ2} = (\rho_{energ} - 1)e + 1, \quad \rho_{energ} = 0.1 \quad (3.38)$$

On Figure 3.4 we can see a graphical representation of both variants of the distance factor f_{dist} and we can identify the same characteristics as on the distance factor but now on energy terms:

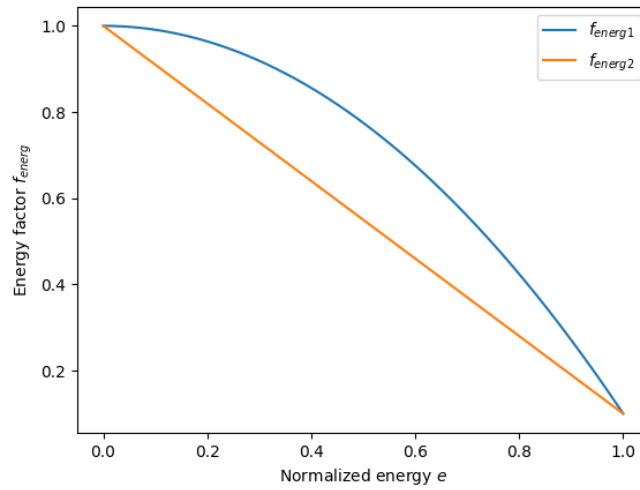


FIGURE 3.4: Comparison of energy factors.

- Both have their maximum $f_{energ} = 1$ when $e = 0$.
- Both have their global minimum $f_{energ} = 0.1$ when $e = 1$.
- f_{energ1} has a structure of a second-degree polynomial structure.
- f_{energ2} has a structure of a first-degree polynomial structure.
- f_{energ1} has higher values than f_{energ2} .

On Chapter 6 we will test these two variants and determine which one gives us better results.

3.3 Quality factor

The reason of including a quality factor is to make sure that the resulting environment is as similar as the original with the most details possible, as we are dividing the scenery into voxels.

A problem that could be presented is a bad recovery of information by the sensor when it has less contact area, this is when the rays are not as perpendicular to the surface. As we are working with a simulated sensor, it may not be a direct problem to the data that we obtain during the simulations because the sensor will work on ideal conditions, but the aim of this project is to lay the foundations for future physical implementations, so we still need to consider quality related problems.

For measuring the quality of the retrieved scenery, we will define a variable $mean_{quality}$ which represents the quality mean of all occupied voxels [33] that is computed as:

$$mean_{quality} = \frac{\sum_{i=1, nw_{occupied}} \cos(\omega_i)}{nw_{occupied}} \quad (3.39)$$

where ω_i is the angle formed between the surface normal of the occupied voxel i and the sensor ray, and $nw_{occupied}$ the total number of occupied voxels in the entire voxel map.

For the quality factor we need to maximize $mean_{quality}$, so the factor will be directly the quality value, however, we are still missing a step which is computing the quality of each view, as our $mean_{quality}$ is from the entire voxel map and all candidate views would get the same value on the quality factor if we used this variable.

The quality factor $f_{quality}$ will be just the quality of the candidate view and will be computed as:

$$f_{quality} = \frac{\sum_{i=1, nv_{occupied}} \cos(\omega_i)}{nv_{occupied}} \quad (3.40)$$

where $nv_{occupied}$ the total number of occupied voxels in the candidate view. This function was previously implemented on [33].

Equations 3.10, 3.11 proposed on [7] had already implicit quality factors that were omitted when presented, however, in this section we will show them as the goal is to compare quality factors, so those functions with quality factors are:

$$\begin{aligned} f_{utility1q} = & (-1.420455x^3 + 0.426136x^2 + 0.994318) * \frac{\sum_{i=1, nv_{ocplane}} \cos(\epsilon_i)}{nv_{ocplane}} \\ & + (-2.155789y^3 + 1.212632y^2 + 0.943158) * \frac{\sum_{i=1, nv_{occupied}} \cos(\omega_i)}{nv_{occupied}} \\ & + (-2.155789z^3 + 1.212632z^2 + 0.943158) \\ & + (-100w^2 + 10w + 0.75) \end{aligned} \quad (3.41)$$

and

$$\begin{aligned}
 f_{\text{utility}2q} = & (-2.31482x^3 + 1.38889x^2 + 0.925926) * \frac{\sum_{i=1, nv_{\text{ocplane}}} \cos(\epsilon_i)}{nv_{\text{ocplane}}} \\
 & + (-4.80769y^3 + 4.32692y^2 + 0.480769) * \frac{\sum_{i=1, nv_{\text{occupied}}} \cos(\omega_i)}{nv_{\text{occupied}}} \quad (3.42) \\
 & + (-4.80769z^3 + 4.32692z^2 + 0.480769)
 \end{aligned}$$

where ϵ_i is the angle formed between the surface normal of the occplane voxel i and the sensor ray, and nv_{ocplane} the total number of occplane voxels in the candidate view.

We can see that both of them use an extra term $\frac{\sum_{i=1, nv_{\text{ocplane}}} \cos(\epsilon_i)}{nv_{\text{ocplane}}}$ corresponding to the mean quality of occplane voxels within the candidate view, this was done because they defined separate terms for the function, each corresponding to a voxel type, and they defined a quality factor for each voxel type. We want to implement the quality factor outside the area factor so we can not attack the problem like Equations 3.41, 3.42, plus we do not consider calculating the mean quality of occplane voxels as a crucial step because our measurement variable $mean_{\text{quality}}$ only considers occupied voxels, we stay with our f_{quality} from Equation 3.40.

Chapter 4

Global optimization

By global optimization we refer to the whole process seen on Figure 4.1 to recover the environment. On the other hand, local optimization will refer to the algorithm used within each iteration and is the one that will find the optimal value of the objective function every time a view is taken, although this local optimization topic will be covered separately on a future chapter.

In this chapter we will present some tasks that we can give to our global optimization process in order to obtain better results. These tasks are designed with the purpose of helping the robot get out of local minima.

4.1 Backstepping

The way our robot finds the best candidate view is through local maximization of the objective function. We implement local maximization instead of global because an environment can be of any size, including huge models, and if we decided to use global maximization we would end up with an exaggerated number of possible combinations of coordinates for the sensor. Local optimization has the advantage that the combinations of coordinates that will be tested are reduced significantly as it considers only the ones within a specific range from the sensor. However, this also causes huge problems as the robot can get stuck on a part of the map which has a higher objective value than all the positions within the range of local optimization, meaning that there is nothing else to scan or upgrade, however other parts of the map still need to be scanned but are out of range.

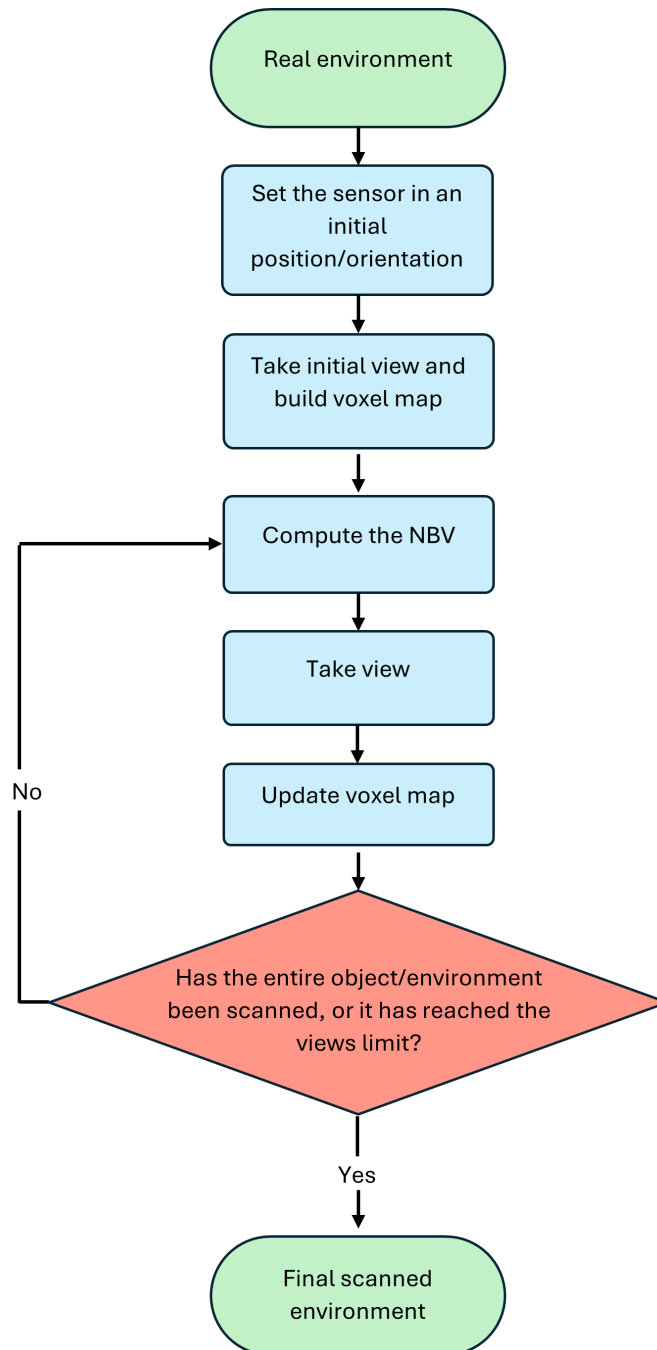


FIGURE 4.1: Global optimization process.

To solve this problem we define a backstepping technique which will move the robot out of the place where it got stuck. In order to trigger the backstepping the following conditions need to be met:

- The robot does not move within the environment, meaning that $d_{norm} = 0$.
- The sensor does not move within the robot, meaning that $e = 0$.

- The total percentage retrieved from the original model is less than 100, meaning that we need to keep improving the retrieved model.

The proposed algorithm will perform the following tasks:

- Check the distance that the robot would need to move from the current view n to the previous one $n - 1$.
- If that distance is bigger than a defined minimum backstepping distance, move to that position.
- If that distance is not bigger, repeat the analysis on the next in line previous view $n - 2$ and so on.

The reason why we analyze the views in descending order is to optimize time during the simulation, as it would cost more time to check all the previous views, here when it finds a suitable position stops analyzing the rest of the views. A minimum backstepping distance is set so that the robot does not return to a very close position, because the reason why we define this task is to get out of a certain area, it would be pointless to barely move. However, we will still test it without this minimum backstepping distance when we change the values of this variable on Chapter 6.

Another restriction that will be added will be to make a register of all the positions in which it needed to perform backstepping, in order to avoid this positions for future backsteppings. We will test it with and without this restriction on Chapter 6.

4.2 Dynamic change in objective function

Although we previously defined all the factors of the objective function on Chapter 3, it does not mean that we have to use the same function during the entire simulation. We could go further and define a task that could send an instruction to the program to change the composition of an specific factor or even change the whole arrangement of this factors inside the general objective function, all of this being done automatically during simulation without any input of the user.

4.2.1 Change in area factor

Despite of defining an algorithm that could move the robot to a previous position when it could no longer find a different local maximum in section 4.1, it still has its limitations due to the implementation of the valid positions array, as it may reach a point in which it no longer finds a different local maximum and needs backstepping but all the feasible previous positions are registered as not valid because they previously experienced backstepping, so the robot would end up stuck without any movement or possibility to perform backstepping. This situation could be when there are few unmarked voxels left and what really needs to be taken into account are the occplane voxels. This task will be activated when both of these conditions are fulfilled:

- The robot needs to perform backstepping but is unable to.
- The percentage of unmarked voxels in the entire voxel map is less than 1%.

We could only set the first condition, but the second one helps us to be sure that in fact there are few unmarked voxels left and what really needs to be taken into account are the occplane voxels.

The actions that this task will do are:

- Replace the area factor f_{area} with a new area factor f_{area2} .
- Reset the valid positions array, as we no longer have the same criteria on the objective function so previous positions that were set to not valid by the previous criteria may be valid now.

As the changes to the area factor will be done only to the desired proportion of unseen area a_{us} and desired proportion of occplane area a_{op} , we can keep the original expresion of f_{area} regarding the desired proportion of overlapped area a_{ov} which was on Equation 3.19. Then, we can multiply this by another factor, which was previously $(1 - \frac{1}{2} |a_{op} - a_{us}|)$ on f_{area} because it favoured both unseen and occplane area equally. But now we are only interested into occplane area and we can define this factor just as a_{op} so that the area factor f_{area2} grows as the occplane area proportion is bigger.

The final expression of the new area factor will be Equation 4.1.

$$f_{area2} = \left(\frac{2a_{ov}^3 - 3(\alpha + 1)a_{ov}^2 + 6a_{ov}\alpha}{3\alpha^2 - \alpha^3} \right) (a_{op}), \quad \alpha \in \left(\frac{1}{3}, 1 \right] \quad (4.1)$$

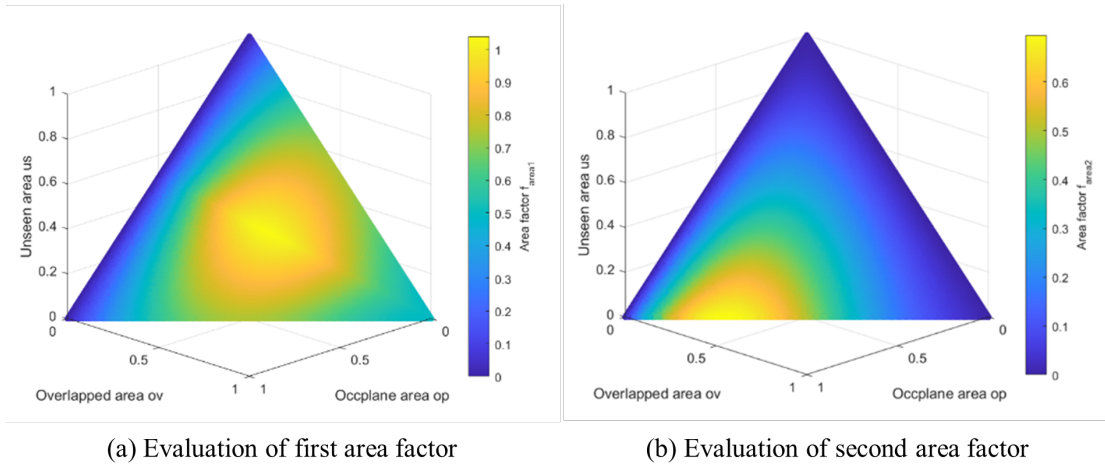


FIGURE 4.2: Area factor with and without dynamic change.

We can plot both this function among the previous one before the dynamic change on a three-dimensional plot shown on Figure 3.1, where we used $\alpha = 0.4$ as an example because having a 5-dimensional graph is not good for visualization. We represent the result of the area factor as a fourth dimension labeled by colors, where their maximum is located at the yellowest coordinate on both cases and we notice that the maximum of the new function is lower on the axis corresponding to us as it does not consider it inside the function.

Chapter 5

Local optimization

In this chapter the mathematical model of the problem will be presented, an analysis of the type of problem we are dealing with will be made and some optimization methods will be proposed to later be tested on Chapter 6.

5.1 Mathematical model

The first step in order to get into the optimization process is to represent the problem in terms of equations and form a mathematical model which includes the objective function, constraints and variables.

5.1.1 Variables

This variables correspond to a set of decisions that need to be made to have the best possible performance, which were developed on Chapter 3. These variables were related to area, energy, distance and quality factors. Bellow we can see a list of all of the factors with their variables, all of which are included directly on the objective function and some of them are intended to be maximized, minimized or even set to a certain value according to the analysis done in Chapter 3:

- Area factor: a_{ov} , a_{op} , a_{us} .
- Energy factor: e .

- Distance factor: d_{norm} .
- Quality factor: $\omega_i, nv_{occupied}$.

This specific problem deals with another type of variables which are not related directly to the objective function but instead they involve the movement of the sensor. For a better understanding of this two types of variables, it is better to explain first the relation between both and how the sensor behaves with the objective function.

The objective function was designed to indicate which is the best candidate view in terms of motion, quality and area factors as mentioned above. However, even if the variables to maximize are from the candidate views, what we need to tell the sensor is how to obtain that best candidate view. As the objective function is intended to be designed to follow certain behaviours, we already know which are the parameters for the best view, and if we only focused on obtaining the optimal solution of the function we would end up with the ideal results we already know. Instead of taking this approach, what we need to do is test the possible views that can be obtained by the sensor in its current location and determine which of them is the best solution, even if the value of the objective function is not 1 which would represent the optimal solution.

With this explanation we get to the conclusion that this other type of variables is sort of an index of each candidate view and can be expressed as:

- Position: NBV_x, NBV_y, NBV_z .
- Orientation: v .

The reason why these variables are not included directly on the function is because there is no mathematical relation between the sensor coordinates and the resulting view, it depends on the environment at that position and it is unpredictable as this solution is intended to work on multiple maps. This already gives us the clue that we need to use simulation-based methods as the only way we can obtain the view parameters of a certain sensor position is through simulation.

For a better understanding, a diagram of this process is included below on Figure 5.1.

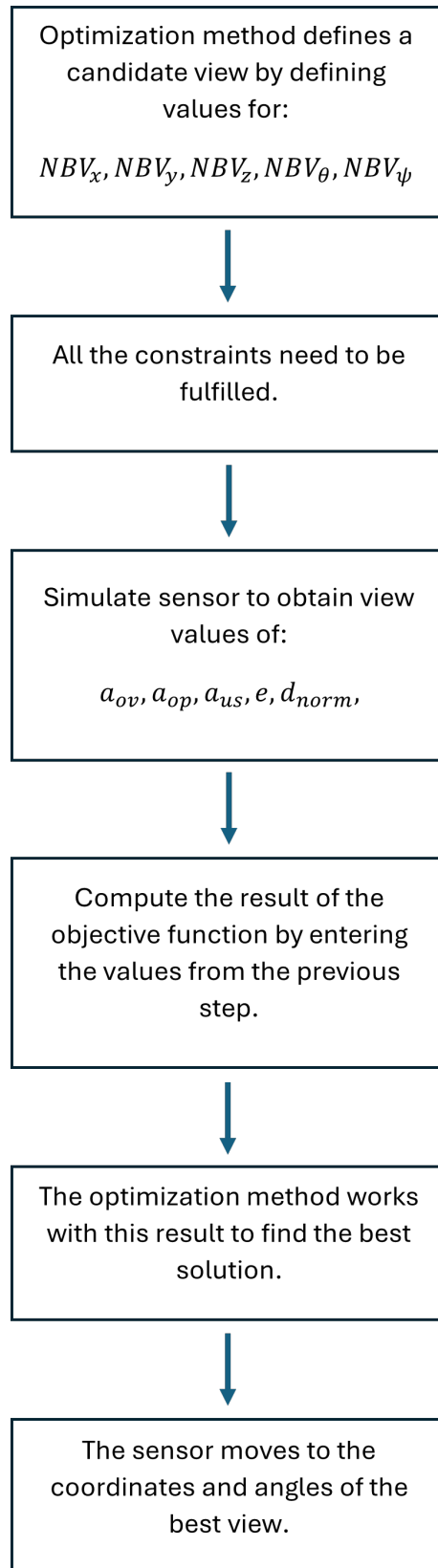


FIGURE 5.1: Optimization diagram.

5.1.2 Objective function

The objective function will be built by combining the individual factors presented on Chapter 3 which have the following expressions:

- Area factor: Equation 3.20 or Equation 4.1.
- Distance factor: Equation 3.31 or Equation 3.32.
- Energy factor: Equation 3.37 or Equation 3.38.
- Quality factor: Equation 3.40.

The final objective function is yet to be obtained as numerous experiments need to be conducted on Chapter 6 to determine the arrangement of these factors among the final function. Even if we do not have this final objective function, we can still analyze each individual factor to propose the optimization methods to be used, which will be done later on this chapter.

5.1.3 Constraints

As mentioned in subsection 5.1.1, we have two types of variables. The constraints of the optimization model are defined by the second type of variables (NBV_x, NBV_y, NBV_z, v) and are defined as:

- The position of the sensor around the Y axis needs to be inside the minimum and maximum permitted height.

$$NBV_y \geq height_{min} \tag{5.1}$$

$$NBV_y \leq height_{max}$$

- The position of the sensor around X and Z axes needs to be inside the world map limits.

$$NBV_x \geq 0$$

$$NBV_x \leq size_x$$

(5.2)

$$NBV_z \geq 0$$

$$NBV_z \leq size_z$$

- The face of the icosahedron needs to be between 0 and 19.

$$v \geq 0$$

(5.3)

$$v \leq 19$$

- The volume inside the safe cylinder from Figure 2.3 needs to be full of empty voxels.

$$\begin{aligned} VM(vx, vy, vz) = V_{empty} \quad \forall \quad & NBV_x - safe_{dist} \leq vx \leq NBV_x + safe_{dist}, \\ & 0 \leq vy \leq height_{max}, \\ & NBV_z - safe_{dist} \leq vz \leq NBV_z + safe_{dist} \end{aligned} \quad (5.4)$$

5.2 Linear programming analysis

After the mathematical model is obtained, our next step is to determine which type of problem we are dealing with, and the first alternative is linear programming.

Although we do not have our final objective function, we can still check the four assumptions of linear programming stated on [42] on each of the defined factors and make an analysis on how each possible combination of those factors could impact the assumptions on the final objective function.

5.2.1 Proportionality

This first assumption studies each variable among the objective function. A change in each variable should produce a proportional change into the result of the objective function, meaning

that the only possible exponent each variable must have is 1.

- Area factor: this factor is special as it has two possible equations that may be used together. We are certain that Equation 3.20 will be used and Equation 4.1 may be used depending on the possible implementation of the dynamic change in the objective function. On both cases we have the terms $2a_{ov}^3$ and $3(\alpha + 1)a_{ov}^2$ which immediately rules out this assumption whether the dynamic change is implemented or not.
- Distance factor: we also have two possible options. Equation 3.31 has the term $(\rho_{dist} - 1)d_{norm}^2$ which does not fulfill this assumption, but Equation 3.32 changes that term to $(\rho_{dist} - 1)d_{norm}$ which would fulfill this assumption. However, the function is piecewise, which invalidates this assumption.
- Energy factor: this factor is similar to the previous one. We have two options, Equation 3.37 has the term $(\rho_{energ} - 1)e^2$ which does not fulfill this assumption, but Equation 3.38 changes that term to $(\rho_{energ} - 1)e$ which would fulfill this assumption. However, the function is piecewise, which invalidates this assumption.
- Quality factor: we only have the possibility of Equation 3.40, which does not have any exponent different than 1 but it has the term $\cos(\omega_i)$ that makes this factor non-proportional.

5.2.2 Additivity

This assumption studies the interaction between the variables. Its purpose is to avoid cross-product terms, meaning that the variables cannot be multiplied or divided with each other.

- Area factor: we have a common term $\left(\frac{2a_{ov}^3 - 3(\alpha + 1)a_{ov}^2 + 6a_{ov}\alpha}{3\alpha^2 - \alpha^3}\right)$ on both Equation 3.20 and Equation 4.1 that fulfils this assumption. However, that term is multiplied with $(1 - \frac{1}{2}|a_{op} - a_{us}|)$ in the first case and with a_{op} on the second scenario, causing this assumption to fail on both functions.
- Distance factor: this factor only includes one variable d_{norm} , there is no possibility that it can have a cross-product term, this assumption is fulfilled.
- Energy factor: this factor only includes one variable e , there is no possibility that it can have a cross-product term, this assumption is fulfilled.

- Quality factor: we have the term $nv_{occupied} \cos(\omega_i)$ that includes the product of two variables, it fails to fulfill this assumption.

5.2.3 Divisibility

This assumption is valid when each variable is not restricted to integer values.

- Area factor: all of our three variables a_{ov} , a_{op} and a_{us} are proportions that go from 0 to 1, they are continuous, this assumption is fulfilled.
- Distance factor: our variable d_{norm} is a proportion that goes from 0 to 1, it is continuous, this assumption is fulfilled.
- Energy factor: our variable e is a proportion that goes from 0 to 1, it is continuous, this assumption is fulfilled.
- Quality factor: the first variable ω_i is an angle that can have decimal values. However, $nv_{occupied}$ has only integer values, this assumption is not fulfilled.

5.2.4 Certainty

This last assumption takes into consideration that the parameters of the objective function need to be known. Among each factor all the coefficients are known because they were built by mathematical operations. However, we are still missing the coefficients and arrangement of this factors among the final objective function, but on the first section of Chapter 6 experiments will be made in order to determine this coefficients so when it is time to test the optimization methods we will already have those parameters and this assumption will be fulfilled.

5.3 Proposed methods

In order to make a decision of the nature of this problem, all possible combinations for linearity are presented bellow on Table 5.1.

A classification of optimization methods is proposed on [43]. The first criterion is the certainty assumption, it is a deterministic problem when it is fulfilled and probabilistic when not. As it

TABLE 5.1: Assumptions checklist.

Factor		Proportionality	Additivity	Divisibility	Certainty
Area	Equation 3.20	No	No	Yes	Yes
	Equation 4.1	No	No	Yes	Yes
Distance	Equation 3.31	No	Yes	Yes	Yes
	Equation 3.32	No	Yes	Yes	Yes
Energy	Equation 3.37	No	Yes	Yes	Yes
	Equation 3.38	No	Yes	Yes	Yes
Quality	Equation 3.40	No	No	No	Yes

is fulfilled in all the scenarios, we need a deterministic method. For deterministic methods we have linear and nonlinear, and in order to be linear at least the proportionality and additivity assumptions need to be fulfilled, as we could still work on some way with linear methods if divisibility was rejected. However, even if the additivity assumption is fulfilled in some scenarios, proportionality is rejected on every single one of them, making this a nonlinear problem.

The next classification for non-linear problems is to be convex or non-convex. The requirements for a problem to be convex are to have a convex set of variables and convex functions according to [44]. Clearly our model is not convex as the set of feasible variables has the topology of the environment and therefore it does not contain the line segment between any two points in the set, plus we do not have the certainty that there are no multiple local optima which are not allowed on convex models. Although this classification does not strictly indicate us which methods to use as many algorithms can be used for both convex and non-convex problems, it helps to understand better the nature of our model and if we find any exclusive method for convex optimization we will not consider it.

First, we have the conventional methods which are direct search and gradient-based [45]. We do not want to have gradient-based methods as we do not have a clear mathematical relationship between the variables from the sensor and the ones used on the objective function as it is done through sensing simulation, we need methods that only use the evaluation of the objective function, making direct search the best approach. The Nelder-Mead algorithm used by [6] and [7] developed by [46] corresponds to local search [47–49], it will be our first proposed method.

Aside from conventional methods, we have metaheuristics which are methods that try to imitate nature behaviour. A general classification of this metaheuristics is presented on [50] where the main criterion is the number of initial points considered. Single-solution based metaheuristics use one initial point and move away from it trying to upgrade the solution; on the other hand,

population-based metaheuristics build a set of initial points. As the already proposed Nelder-Mead algorithm has an initial simplex built with multiple points, we will have a bigger focus on methods with a single initial point to have bigger difference among the proposed methods.

There is a wide variety of single-solution based metaheuristics, we will propose tabu search as it has the particularity that it is inspired by local search methods but adds memory structures and helps to add variety within our points, therefore we can have an interesting comparison with the Nelder-Mead algorithm which is a local search conventional method without memory.

Even if evolutionary algorithms are considered population-based metaheuristics, there is one method called (1+1)-ES that uses a single initial point as a parent and produces a single offspring on each iteration [51]. We will also use this algorithm as it has a single initial point, plus we will have a new approach on our set of methods as this one has a random strategy unlike our other proposals. Lastly, we can have a different approach by using a simulated annealing which is a probabilistic method.

Below we can see all three proposed methods with their main differences:

1. Nelder-Mead algorithm:

- Conventional method
- Local search without memory

2. (1+1)-ES:

- Metaheuristic
- Random strategy

3. Tabu search:

- Metaheuristic
- Local search with memory

4. Simulated annealing:

- Metaheuristic
- Probabilistic method

We will treat all of these methods as minimizations, therefore we will multiply by -1 our objective function to turn it into a minimization as it is originally a maximization, and we will assign a value of infinity to the evaluations in which the constraints were not fulfilled. On all of them we will keep the current position and use exhaustive search for orientation in case that the optimal point has an evaluation value of infinity when it does not fulfill the position constraints meaning that the robot can not move but can change orientation, which happens on the first iterations as we do not have enough information of the environment and we are unable to correctly check the safe radio constraint.

5.3.1 Nelder-Mead algorithm

This method creates an initial simplex with one more dimension than the number of variables used, and performs certain operations (reflection, expansion, contraction and reduction) on the simplex until it reaches a tolerance value indicating that it reached an optimal value. The Nelder-Mead method has some variations across the literature with some including inside and outside contraction [52]. However, the algorithm used on [6] and [7] which is the one we will use is based on the algorithm from [53] and only includes inside contraction.

The number of vertexes of the simplex will be $ndim + 1$ where $ndim$ is the number of variables of the optimization model. $ndim$ will be 3 as they use this algorithm only for the position coordinates due to the implementation of an exhaustive search for the orientation, which must not be confused with the exhaustive search proposed for every method as that one was used only when no optimal solution could be found, whereas this one will be used every time we want to evaluate a point. Each point p of the simplex will have those 3 coordinates (NBV_x , NBV_y and NBV_z). A function $eval(p)$ will be implemented as the exhaustive search that will evaluate the objective function on all 20 values of v within the position of p to return the best of those evaluations.

The steps will have slight modifications from the Nelder-Mead algorithm of [53] and use the base code provided on [54]:

1. Build the initial simplex with points $p_1, \dots, p_{ndim+1}$. The current position of the sensor ($Sensor_x$, $Sensor_y$ and $Sensor_z$) will be the coordinates for p_1 , while the rest of the points will be random feasible coordinates. Go to step 2.

2. Order the simplex points from best to worst, meaning that: $eval(p_1) \leq eval(p_2) \leq \dots \leq eval(p_{ndim}) \leq eval(p_{ndim+1})$. Go to step 3.
3. Obtain the centroid p_0 of points $p_1, \dots, p_{ndim}$. Go to step 4.
4. Reflection:
 - Get the reflected point $p_r = p_0 - (p_{ndim+1} - p_0)$.
 - If $eval(p_1) \leq eval(p_r) < eval(p_{ndim})$, then replace p_{ndim+1} by p_r , go to step 8.
 - Else, go to step 5.
5. Expansion:
 - If $eval(p_r) < eval(p_1)$, then get the expanded point $p_e = p_0 - 2(p_{ndim+1} - p_0)$.
 - If $eval(p_e) < eval(p_r)$, then replace p_{ndim+1} by p_e , go to step 8.
 - Else, replace p_{ndim+1} by p_r , go to step 8.
 - Else, go to step 6.
6. Contraction:
 - Get the contracted point $p_c = p_0 + 0.5(p_{ndim+1} - p_0)$.
 - If $eval(p_c) < eval(p_{ndim+1})$, then replace p_{ndim+1} by p_c , go to step 8.
 - Else, go to step 7.
7. Reduction:
 - For $i = 2, \dots, ndim + 1$, compute $p_i = p_1 + 0.5(p_i - p_1)$. Go to step 8.
8. Stopping mechanism:
 - Compute $nelder_{range} = \frac{2|p_{ndim+1} - p_1|}{|p_{ndim+1}| + |p_1|}$ and $nelder_{iter} = nelder_{iter} + 1$.
 - If $nelder_{iter} \geq 1500$ or $nelder_{range} < 0.0001$, then optimal point is p_1 , end algorithm.
 - Else, go to step 2.

As mentioned above, this steps include the exhaustive search of the orientation with the $eval(p)$ function. We have the hypothesis that the exhaustive search consumes a lot of computational time so we will propose another option in which no exhaustive search is included and all the optimization is done with the Nelder-Mead algorithm.

As we now include the variable v on the algorithm, $ndim$ will change from 3 to 4 and each point p of the simplex will have those 4 coordinates (NBV_x , NBV_y , NBV_z and v). Instead of using $eval(p)$ we will use $f(p)$ that evaluates the objective function on the given coordinates from p . One important thing to take into consideration is the fact that v is an integer value, and if we decided to modify the algorithm operations to limit that variable for only integer values, we would have problems on the stopping mechanism as the tolerance value is a small number less than 1 but the minimum change in v would be 1. We propose to keep all the points as they are even if v is not integer and make the tolerance verification with those values, then we make the rounding of v inside $f(p)$ and store that rounded value into a variable dir so when we reach the optimal point we will tell the sensor to move on dir which is integer instead of v that is decimal. This does not represent any crucial problem as it ensures that the evaluation of the objective function of the optimal point indeed corresponds to its real orientation which is the integer value of the icosahedron face, it serves as a coding of the variable v .

We will test this two approaches on the next chapter and compare the final results to see if this new approach indeed reduces the computational time and if the final result is not jeopardised due to this modifications.

5.3.2 (1+1)-ES

This optimization algorithm is based on the nature of evolution, and the "(1+1)" in its name makes reference to the fact that it only produces one offspring from one parent on every iteration. It takes an initial parent that produces an offspring slightly different than its parent and whoever has a better value according to the objective function is chosen as the parent of the next generation and repeats until the number of generations gen is equal to the parameter gen_{max} which is the maximum generations allowed.

The steps are inspired on the (1+1)-ES algorithm of [55] and will be:

1. Initialization:

- Build the initial parent p_{parent} . The current position and orientation of the sensor ($Sensor_x$, $Sensor_y$, $Sensor_z$, $Sensor_v$) will be the coordinates for p_{parent} . Go to step 2.
- Set the generation counter counter as $g = 1$.

2. Mutation:

- Get all four coordinates for the new offspring: $p_{offspring}[i] = p_{parent}[i] + \sigma * random_{gauss}$ where i is the i th coordinate of the point, σ is a variable parameter that represents the strength of the mutation and $random_{gauss}$ is a random number following a Gaussian distribution with mean 0 and standard deviation of 1.
- Round the generated coordinate associated to v as it needs to be integer: $p_{offspring}[4] = round(p_{offspring}[4])$.
- Go to step 3.

3. Selection:

- If $f(p_{offspring}) < f(p_{parent})$, then replace parent by $p_{parent} = p_{offspring}$, increase the number of successful mutations m by $m = m + 1$.
- Go to step 4.

4. σ update:

- Check only every 20 generations.
- If $\frac{m}{gen} > 0.2$, then $\sigma = \frac{\sigma}{c}$ where c is a defined parameter.
- Else if $\frac{m}{gen} < 0.2$, then $\sigma = \sigma * c$.
- Go to step 5.

5. Stopping mechanism:

- Compute $gen = gen + 1$
- If $gen \geq gen_{max}$, optimal point is p_{parent} , end algorithm.
- Else, go to step 2.

As this algorithm has not been used on the NBV problematic to the best of our knowledge unlike the Nelder-Mead method, we are not certain on which parameter values to use. For gen_{max} we will use initially 500 as it is a big number and although it may increase the computational time, we will analyze the results on the next chapter and develop a method to obtain a smaller gen_{max} based on the results obtained with $gen_{max} = 500$. For the parameter c we will use 0.817 as it has been proven to be an adequate value for the algorithm regardless of its application [56] and has been developed theoretically on [57]. Finally, we need to have an additional parameter $\sigma_{initial}$ which is the initial value of σ , we will test multiple values on the next chapter to determine which one is the best for our model.

5.3.3 Tabu search

This method uses memory to avoid recently viewed points and it can be applied on the whole candidate or just some of their attributes. We will implement it just on the variable v as it is integer and easier to have record of, plus we consider that the change in orientation is the one that provides the biggest diversification. Tabu search is more of a technique than a method itself, therefore it needs another algorithm as a base that in our case will be the (1+1)-ES explained above.

This method can work by proposing multiple candidate points and selecting the one that has the best function evaluation and is not part of the tabu list as shown on [58]. However, this would represent a significant increase on computational time as multiple function evaluations would need to be done on each iteration. We will make an adaptation so that only one function evaluation is done on each iteration by adding the elements from the tabu list as constraints on the creation of the candidate point. This may cause the algorithm to repeat several times the creation of the point until it meets the constraint, but the impact on the computational time will not be as significant as if multiple function evaluations were made as they are the most time-consuming operation of our program.

We can summarize the entire algorithm as the following steps inspired by the ones from the previous (1+1)-ES method with a few modifications:

1. Initialization:

- Build the initial parent $tabu_{parent}$. The current position and orientation of the sensor ($Sensor_x, Sensor_y, Sensor_z, Sensor_v$) will be the coordinates for $tabu_{parent}$.
- Initialize on zero all elements of $tabu_{list}$ that will be a vector of 20 elements (all possible values of v) and is the number of generations that each value is banned. Go to step 2.
- Set the element $tabu_{parent}[4]$ on $tabu_{list}$ as $tabu_{span}$ which is a parameter that represents the number of banned generations for all tabu elements.
- Set the generation counter counter as $g = 1$.

2. Mutation:

- Get the first three coordinates for the new offspring: $tabu_{offspring}[i] = tabu_{parent}[i] + \sigma * random_{gauss}$ where i is the i th coordinate of the point and goes from 1 to 3, σ is

a variable parameter that represents the strength of the mutation and $random_{gauss}$ is a random number following a Gaussian distribution with mean 0 and standard deviation of 1.

- For the coordinate associated to v we generate a random integer from 0 to 19: $tabu_{offspring}[4] = rand() \% 20$.
- If the element $tabu_{offspring}[4]$ on $tabu_{list}$ is greater than zero it is banned for this generation, repeat the creation of $tabu_{offspring}[4]$ until it is admissible.
- Go to step 3.

3. Tabu list update:

- Reduce by 1 the banned generations from all the elements from $tabu_{list}$ that are different than 0.
- Set the element $tabu_{offspring}[4]$ on $tabu_{list}$ as $tabu_{span}$.
- Go to step 4.

4. Selection:

- If $f(tabu_{offspring}) < f(tabu_{parent})$, then replace parent by $tabu_{parent} = tabu_{offspring}$, increase the number of successful mutations m by $m = m + 1$.
- Go to step 5.

5. σ update:

- Check only every 20 generations.
- If $\frac{m}{gen} > 0.2$, then $\sigma = \frac{\sigma}{c}$ where c is a defined parameter.
- Else if $\frac{m}{gen} < 0.2$, then $\sigma = \sigma * c$.
- Go to step 6.

6. Stopping mechanism:

- Compute $gen = gen + 1$
- If $gen \geq gen_{max}$, optimal point is $tabu_{parent}$, end algorithm.
- Else, go to step 2.

The reason why we consider random numbers from 0 to 19 on $tabu_{offspring}[4]$ is because it is the coordinate corresponding to v which is our tabu element, and the reason why tabu search

is used is to provide variety. If we had kept its computation as $tabu_{offspring}[i] = tabu_{parent}[i] + \sigma * random_{gauss}$ like the other coordinates we would end up with very few value options at each iteration as it is constrained to be an integer number, and this lack of options would cause the tabu list constraint to jam the algorithm. If the range of v was bigger we would have needed a coding for the variable or additional operations, but we consider that having 20 possible values for the variable is acceptable for it to be implemented on a tabu list.

We end up with $tabu_{span}$ as the only parameter that will need to be set on a certain value. Multiple experiments will be done on the next chapter to obtain that value.

5.3.4 Simulated annealing

This last method is a probabilistic metaheuristic that helps to avoid local minima as stated on [59] by sometimes accepting bad solutions as the new point giving the algorithm the opportunity to explore new areas. This is done by providing a high probability for the first iterations and decreasing it for bigger iteration values. Bellow we can see the steps for this algorithm inspired by the work of [60]:

1. Initialization:

- Build the initial reference point $point_{ref}$. The current position and orientation of the sensor ($Sensor_x, Sensor_y, Sensor_z, Sensor_v$) will be the coordinates for $point_{ref}$.
- Build the initial optimal point $point_{opt}$. The current position and orientation of the sensor ($Sensor_x, Sensor_y, Sensor_z, Sensor_v$) will be the coordinates for $point_{opt}$.
- Set the temperature $T_k = T_0$ where T_0 is a parameter for the initial temperature.
- Set the iteration counter as $k = 1$.

2. Neighbour point generation:

- Get the first three coordinates for the new point $point_{new}$ within a neighbourhood of 3 from the current reference point $point_{ref}$: $point_{new}[i] = point_{ref}[i] + rand(-3, 3)$ where i is the i th coordinate of the point and goes from 1 to 3 and $rand(-3, 3)$ is a function that returns a random float number from -3 to 3 .
- Get the fourth coordinate for the new point $point_{new}$ within a neighbourhood of 4 from the current reference point $point_{ref}$: $point_{new}[4] = point_{ref}[4] + randint(-4, 4)$

where $randint(-4, 4)$ is a function that returns a random integer number from -4 to 4 .

- Go to step 3.

3. Evaluation:

- Compute $f(point_{ref})$ and $f(point_{new})$.
- Compute $acceptance_{prob} = e^{\frac{-(f(point_{new}) - f(point_{ref}))}{T_k}}$.
- Go to step 4.

4. Selection:

- If $f(point_{new}) < f(point_{opt})$, then replace optimal point by $point_{opt} = point_{new}$.
- If $f(point_{new}) < f(point_{ref})$, then replace reference point by $point_{ref} = point_{new}$.
- Else if $rand(0, 1) < acceptance_{prob}$, where $rand(0, 1)$ is a function that returns a random float number between 0 and 1, then replace reference point by $point_{ref} = point_{new}$.
- Go to step 5.

5. Cooling:

- Check only every $\frac{L}{25}$ iterations to have a total of 25 updates, where L is a parameter that represents the maximum iterations.
- Update temperature by $T_k = T_k * cool_{ratio}$ where $cool_{ratio}$ is a parameter for the cooling factor.
- Go to step 6.

6. Stopping mechanism:

- Compute $k = k + 1$
- If $k \geq L$, optimal point is $point_{opt}$, end algorithm.
- Else, go to step 2.

We end up with 3 parameters: $cool_{ratio}$, T_0 and L . We can tune T_0 by defining our desired initial probability of acceptance for the worst case scenario that represents the biggest possible change in the function evaluations which for our problem is 1. We choose the initial acceptance probability of 95% considering that this for the worst case scenario as with smaller changes than

1 we would have bigger probabilities which is desired for the first iterations of this method. This gives us the analysis of Equation 5.5 where we compute the value $T_0 = 19.5$.

$$0.95 = e^{\frac{-1}{T_0}}$$

$$T_0 = \frac{-1}{\ln(0.95)} \quad (5.5)$$

$$T_0 = 19.5$$

For $cool_{ratio}$, we can tune it by defining our desired final probability of acceptance for the worst case scenario that represents the biggest possible change in the function evaluations which for our problem is 1. We choose the final acceptance probability of 5% so that we still have a small probability on the last iteration. This gives us the analysis of Equation 5.6 where we obtain the value $cool_{ratio} = 0.85$ and we confirm that it is a feasible value as it should be between 0 and 1 as stated by [61].

$$T_f = \frac{-1}{\ln(0.05)}$$

$$T_f = 0.3333$$

$$T_f = T_0 * cool_{ratio}^{25} \quad (5.6)$$

$$cool_{ratio} = \sqrt[25]{\frac{0.3333}{19.5}}$$

$$cool_{ratio} = 0.85$$

Now we only have L as an unknown parameter for which we will make an experimental analysis on the next chapter to determine its best value.

Chapter 6

Results and Discussion

In this chapter all results are presented in the same order as Chapters 3 - 5 were developed following the methodology in order to notice the evolution of the project, how each modification on each section changed the results to end up with the final results.

6.1 Models

In this section the tested environments are presented, including photos from multiple angles plus their dimensions which will be useful for determining how much of the model was retrieved. This models are VRML files and are virtual representations scenarios that could exist on real life chosen so that they had an increasing difficulty among them. They are built with triangular meshes and as a single surface instead of an actual solid model as we only want to include the surface that would be sensed by the robot, otherwise, all the fillings of the solid would be unreachable to the sensor and our model would have a lot of unseen area.

6.1.1 Room with objects model

This first model is a simple room with one wall that divides it partially into two sections. To increase a little the complexity we decided to add a cube on one section and one pyramid on the other section. A window hole was added to see the interior due to the limitations of our simulator not giving us the ability to look inside the model.

The model information is presented on Table 6.1 where we can see its dimensions and total surface area to be retrieved.

TABLE 6.1: Information from room with objects model.

Parameter	Value
Total surface area	246.634048 m^2
Dimensions	6 m x 2.7 m x 12 m

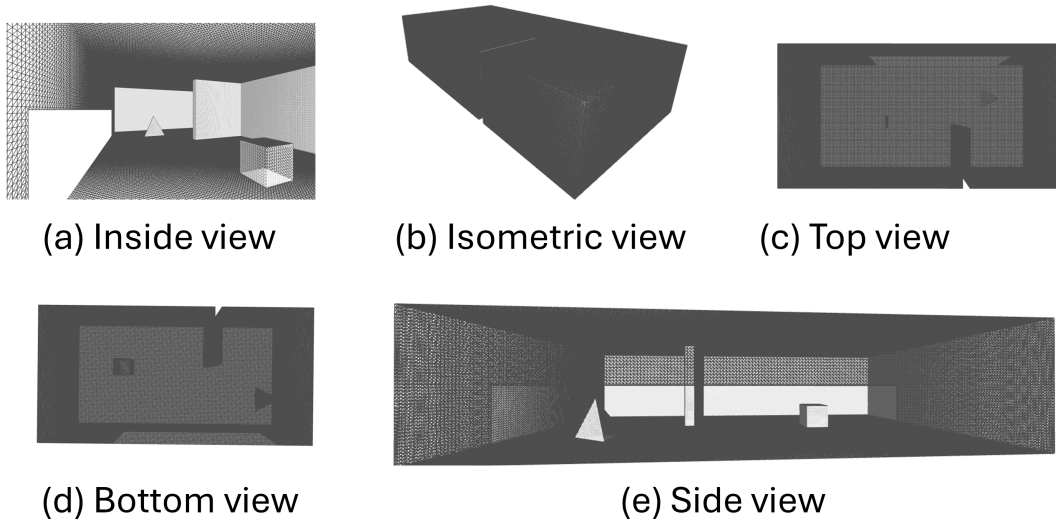


FIGURE 6.1: Room with objects model.

Multiple angles of this model can be seen on Figure 6.1 including an inside view. Figure 6.2 shows the map of this model where the wall is highlighted and the cube and pyramid are dimmed.

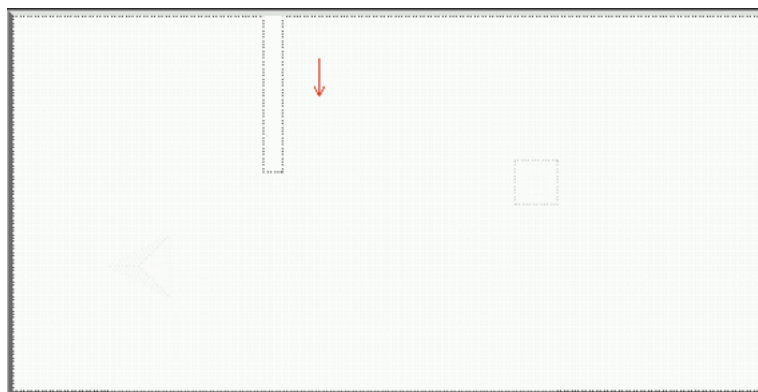


FIGURE 6.2: Room with objects model map.

6.1.2 Study room

This model represents a study room with windows, a chimney on a corner and multiple furniture. We consider this model to have an intermediate complexity due to its general shape and the furniture introduced, specially the sofa. The reason why it is intermediate and not extremely complex is because the desks from the furniture are oversimplified into rectangular prisms. As we have more interior detail it was decided to built the model without a roof to be able to see the results on a better way.

The model information is presented on Table 6.2 where we can see its dimensions and total surface area to be retrieved.

TABLE 6.2: Information from study room model.

Parameter	Value
Total surface area	170.448196 m^2
Dimensions	$6 \text{ m} \times 2.7 \text{ m} \times 12.096 \text{ m}$

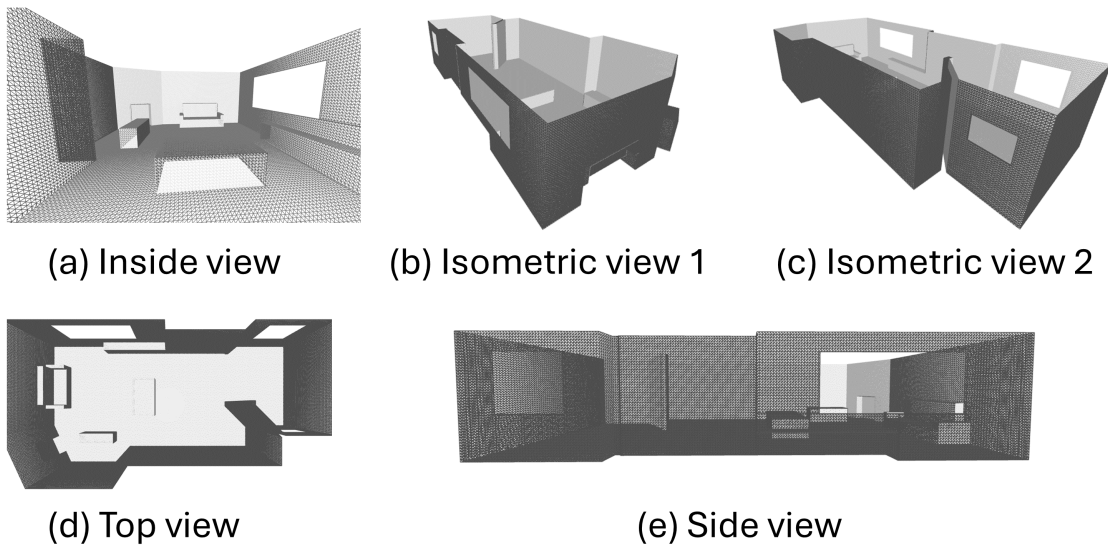


FIGURE 6.3: Study room model.

Multiple angles of this model can be seen on Figure 6.3 including an inside view on Figure 6.3(a) where we see with more detail the sofa and the simplified desks. Figure 6.4 shows the map of this model with the furniture dimmed and the walls highlighted.

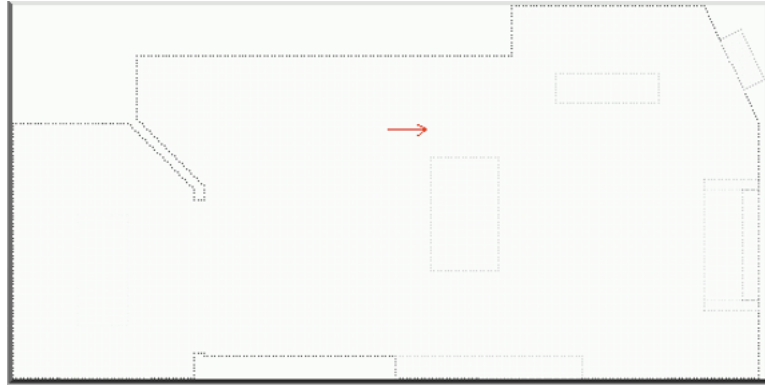


FIGURE 6.4: Study room model map.

6.1.3 Laboratory model

This model developed by [62] consists of a large building containing two sections, one containing a dome plus three columns and two furniture, while the other section is more simple containing just one furniture and one column although it has more narrow areas. Both sections are joined with a corridor in the middle of the model.

The model information is presented on Table 6.3 where we can see its dimensions and total surface area to be retrieved.

TABLE 6.3: Information from laboratory model.

Parameter	Value
Total surface area	443.205414 m^2
Dimensions	7 m x 5.3 m x 20 m

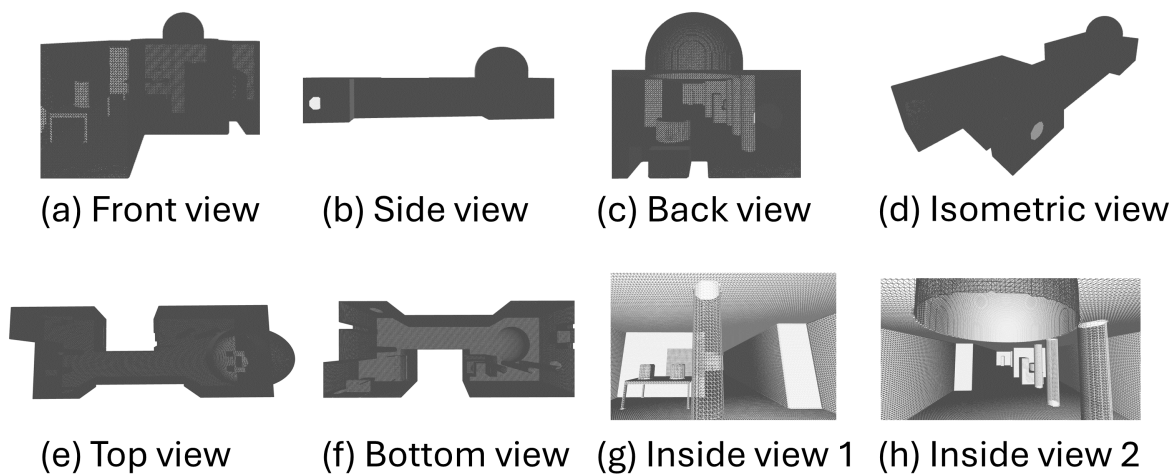


FIGURE 6.5: Laboratory model.

Multiple angles of this model can be seen on Figure 6.5 including two inside views. Figure 6.5(g) corresponds to the section where one column and one furniture can be seen, while Figure 6.5(h) corresponds to the other section where the dome, three columns and two furniture can be seen. Figure 6.6 shows the map of this model for better a better representation of the internal details, as we could not delete the roof like the previous model because this one is an already existing model provided by [62].

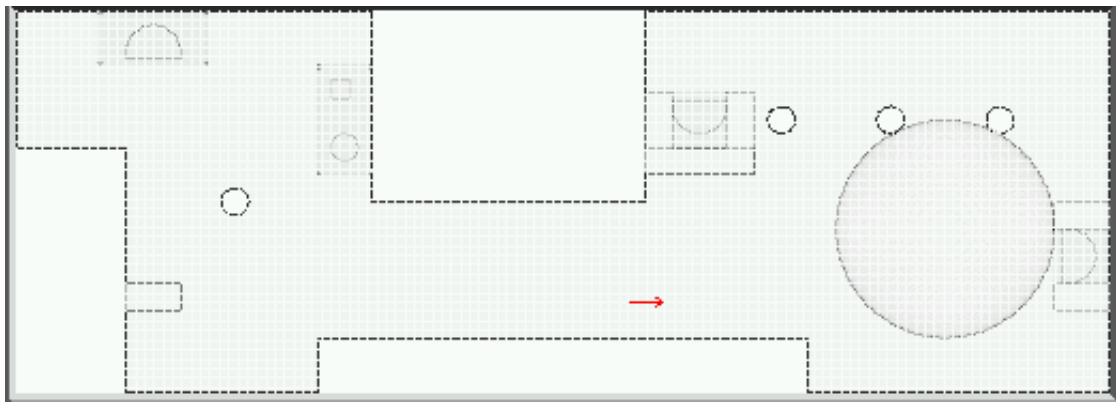


FIGURE 6.6: Laboratory model map.

We consider this model as the most complex one. Therefore, it will be our base model for the initial experiments as we consider that if the algorithm is able to retrieve this model it will recover all the other models. Still, on the final section once all algorithms have been tuned correctly we will test them on all the models.

6.1.4 Laboratory sub-model

For some experiments we will need a more contained environment in order to perform specific tasks, meaning that the robot should have less space to move into. For solving this requirement, a sub-model is defined by sectioning the original laboratory model by half, focusing only on the simplest section containing just one furniture and one column with more narrow areas.

The model information is presented on Table 6.4.

TABLE 6.4: Information from laboratory sub-model

Parameter	Value
Total surface area	248.038544 m^2
Dimensions	7 m x 2.7 m x 15.636 m

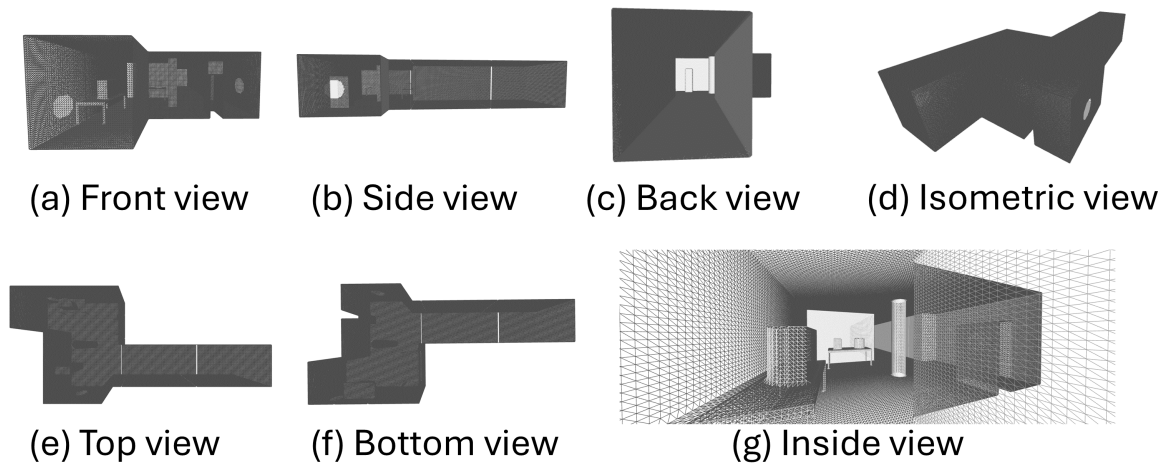


FIGURE 6.7: Laboratory sub-model.

Multiple angles of this model can be seen on Figure 6.7 including one inside view on Figure 6.7(g) containing the column and a better view of the furniture than on Figure 6.5(g). Also, on Figure 6.5(c) it can be seen that the section where it is cut from the main laboratory model is opened without any wall, as it is meant to be a truthful representation of that section of the original model. Figure 6.8 shows the map of this model for a better representation of the internal details.

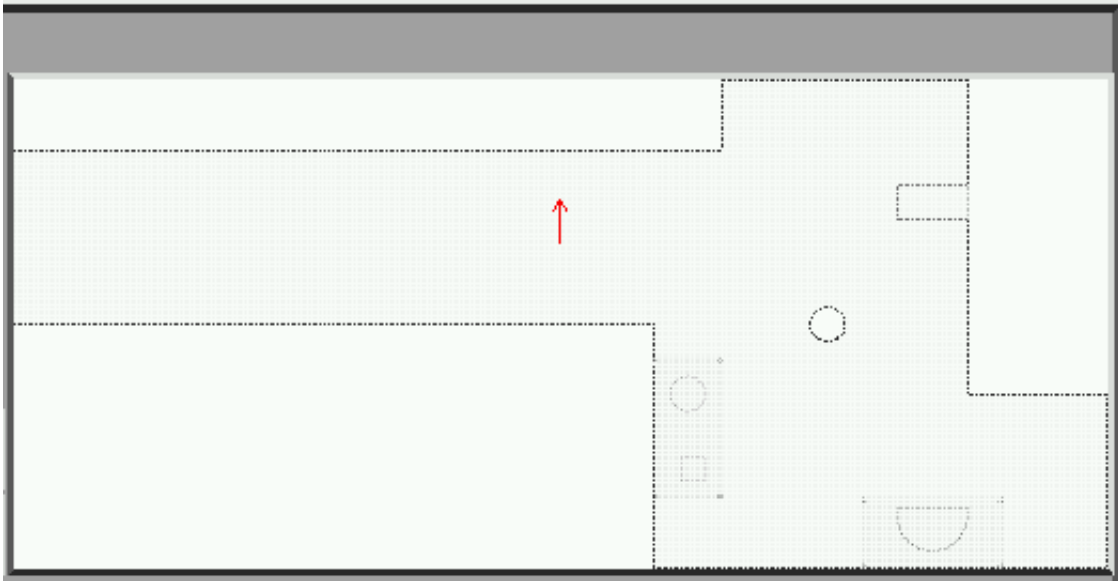


FIGURE 6.8: Laboratory sub-model map.

6.2 Setup parameters

For all the upcoming experiments we will use the same simulated sensor and robot dimensions shown on Table 6.5 to have equal conditions between each experiment. The platform used will be a simulator based on the work by [6] that is programmed with C which uses GCC 4.2.4 on a computer running Ubuntu 8.04.

TABLE 6.5: Setup parameters.

Parameter	Value
Sensor characteristics	
Sensor view angle	80°
Minimum sensor distance	0.5 m
Maximum sensor distance	30 m
Sensor noise deviation	0.012
Sensor horizontal aperture	135°
Sensor vertical aperture	70°
Sensor horizontal points	350
Sensor vertical points	80
Robot dimensions	
Minimum sensor height	0.5 m
Maximum sensor height	2.0 m
Sensor safe distance	0.3 m
Voxel map parameters	
Voxel size	0.15 m

6.3 Objective function experiments

In this section only the objective function will be changed while keeping the global optimization tasks and local optimization method as constant due to the goal of this section being to obtain the best objective function possible which will then be fixed regardless of the target environment or the optimization method, that is why we can manipulate first the objective function as it is independent. The optimization method that will be used for this section is Nelder-Mead for position and exhaustive search for orientation as it has been used before and proven to be functional on [6] and [7].

6.3.1 Area factor experiments

The fixed parameters for all the experiments regarding the area factor presented on section 3.1 are shown on Table 6.6. These parameters contain the objective function factors, global optimization tasks and local optimization methods presented on Chapters 3 - 5.

TABLE 6.6: Area factor experiment simulation parameters.

Parameter	Value
General considerations	
Environment	laboratory model
Number of views	30
Safe cylinder	yes
Objective function	
Area factor	Equation 3.20 with variable α
Distance factor	none
Energy factor	none
Motion factor	none
Quality factor	none
Total objective function	$f = f_{area}$
Global optimization	
Minimum backstepping distance	no backstepping
Backstepping register	no backstepping
Dynamic change in area factor	no
Local optimization	
Method	Nelder-Mead with exhaustive search

The environment that will be used is the laboratory model which can be seen on Figure 6.5 with the information shown in Table 6.3. The reason why we chose only to take 30 views is because we do not want to retrieve the whole model yet, we are just interested in understanding the impact of α within the area factor, more views would consume more time without any crucial benefit for this analysis.

The proposed experiments are presented on Table 6.7, where each experiment is represented with a code and has its corresponding value of α . It was decided that the value of α would have a 0.1 interval between each experiment from 0.4 to 0.9 while adding an initial experiment at 0.34 which is the first possible value of α according to the restrictions from Equation 3.20.

The results from each experiment are summarized on Table 6.8. For this particular set of experiment our only result is the total percentage retrieved from the original model, represented

TABLE 6.7: Area factor experiment variable parameters.

ID	α
1.1	0.34
1.2	0.4
1.3	0.5
1.4	0.6
1.5	0.7
1.6	0.8
1.7	0.9

by the variable $\%retrieved$, which is calculated as:

$$\%retrieved = 100 * \frac{triangles_{retr}}{triangles_{total}} \quad (6.1)$$

where $triangles_{retr}$ is the number of retrieved triangles which are determined by their overlapping with the occupied voxels obtained from the sensing, and $triangles_{total}$ is the total number of triangles in the VRML file and is known at the beginning of the process.

TABLE 6.8: Area factor experiment results.

ID	$\%retrieved$
1.1	51.17
1.2	53.04
1.3	50.88
1.4	46.93
1.5	41.28
1.6	17.86
1.7	17.86

The retrieved area from each experiment is presented on Figure 6.9 where we only show the occupied voxels in blue, and we can see that there is a noticeable difference between the ones with α from 0.34 to 0.7 and the ones with α from 0.8 to 0.9, this last two represented on Figure 6.9(f) and Figure 6.9(g).

On Table 6.8 we can see that the value of α which maximizes the $\%retrieved$ is 0.4 on experiment "1.2", and as we are dealing with numerical values of α on ascending order, we can plot them to understand better their behaviour. Such plot is represented on Figure 6.10, having a slight increment from $\alpha = 0.34$ to $\alpha = 0.40$ which is its maximum, and decreasing from $\alpha = 0.40$ to $\alpha = 0.90$ with a huge drop after $\alpha = 0.70$. As "1.2" was the best performing experiment, we will use $\alpha = 0.40$ on all future experiments. This value was the same as the one proposed on [6] meaning that we were not able to improve it although by doing this analysis we could sustain

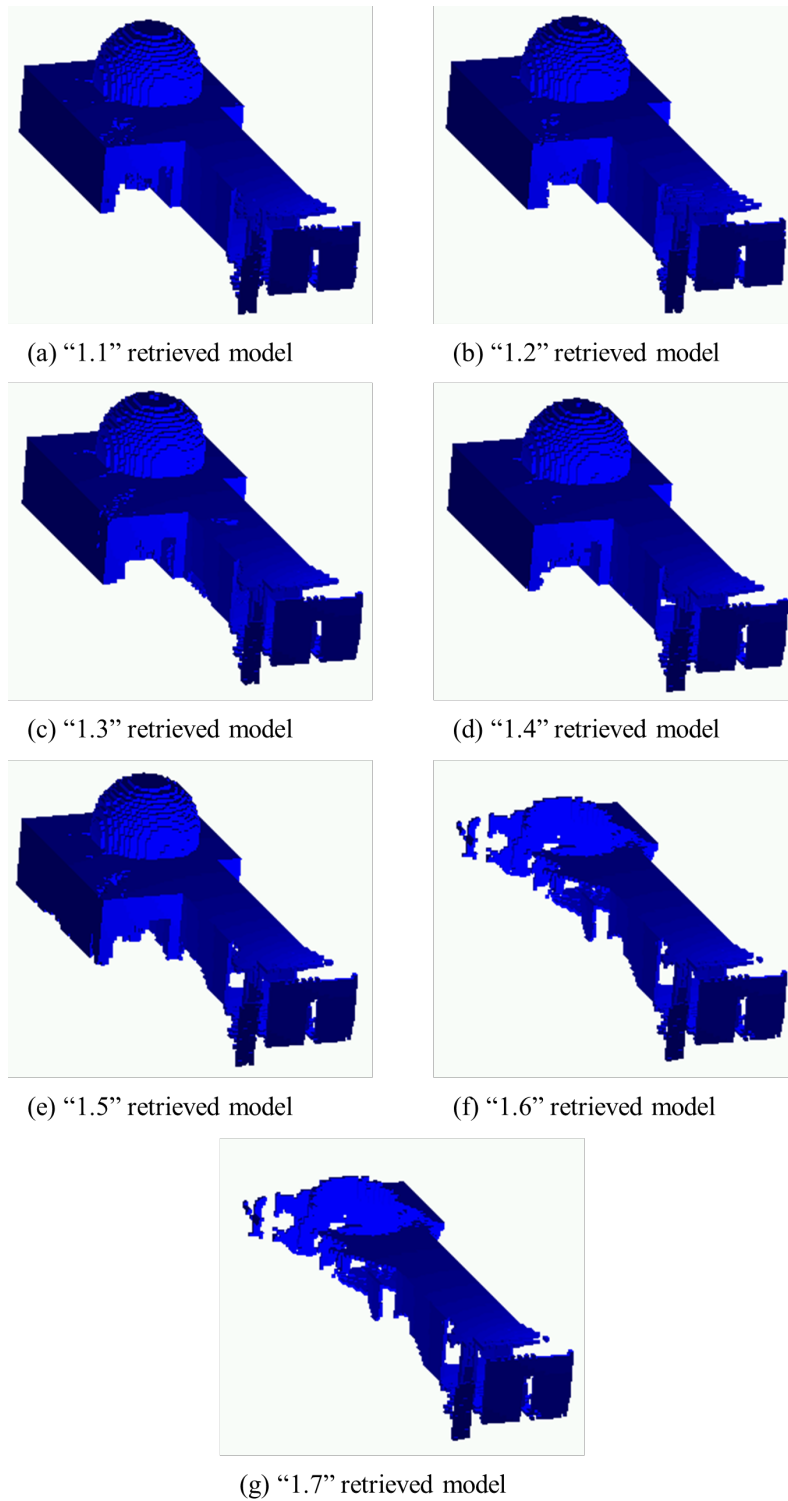
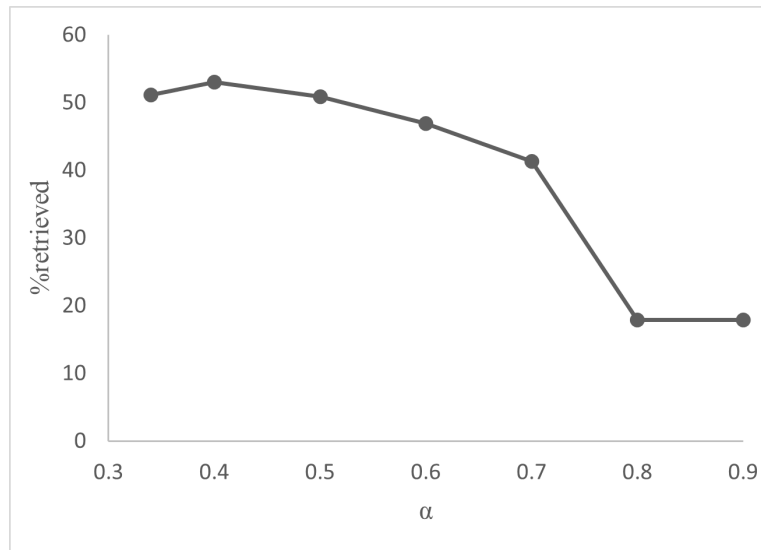


FIGURE 6.9: Area factor experiment retrieved models.

this choice and see why it was made. This also causes that the graph from the resulting area factor function from Equation 3.20 with $\alpha = 0.40$ is the same as the one seen on Figure 3.1.

FIGURE 6.10: α vs %retrieved.

6.3.2 Motion factor experiments

The fixed parameters for all the experiments regarding the motion factor presented on section 3.2 are shown on Table 6.9. This parameters contain the objective function factors, global optimization tasks and local optimization methods presented on Chapters 3 - 5.

TABLE 6.9: Motion factor experiment simulation parameters.

Parameter	Value
General considerations	
Environment	laboratory model
Number of views	50
Safe cylinder	yes
Objective function	
Area factor	Equation 3.20, $\alpha = 0.4$
Distance factor	variable
Energy factor	variable
Motion factor	variable
Quality factor	none
Total objective function	$f_{area} * f_{motion}$
Global optimization	
Minimum backstepping distance	no backstepping
Backstepping register	no backstepping
Dynamic change in area factor	no
Local optimization	
Method	Nelder-Mead with exhaustive search

The environment that will be used is the laboratory model which can be seen on Figure 6.5 with the information shown in Table 6.3. Again, the goal is not to retrieve the whole model yet,

we are just trying to analyse which motion factor is better, however, we decided to increase the number of views from 30 to 50 to give the motion factor more settling time while still keeping the number of views far from extremely big values.

The proposed experiments are presented on Table 6.10, where each experiment is represented with a code and has its corresponding equations for distance factor f_{dist} and energy factor f_{energ} , plus a column with the combination of this two factors which will represent the entire motion factor f_{motion} . The whole motion factor needs to be from 0 to 1, this means that the sum of the contributions from the distance and energy factors needs to be on this range. We propose multiple combinations, where each distance and energy factors will be tested with both of versions:

- Distance factor has more weight: the distance factor represents 70% of the motion factor and the energy factor 30%.
- Energy factor has more weight: the distance factor represents 30% of the motion factor and the energy factor 70%.
- Distance and energy factors have the same weight: the distance factor represents 50% of the motion factor and the energy factor 50%.
- Only use distance factor: the distance factor represents 100% of the motion factor and the energy factor 0%.
- Only use energy factor: the distance factor represents 0% of the motion factor and the energy factor 100%.
- Do not use motion factor: $f_{motion} = 1$ so it has zero impact when multiplying it with f_{area} .

The results for all the experiments are shown on Table 6.11, and new resulting variables are added such as the accumulated distance $dist_{2d}$ and the accumulated normalized energy e .

The first filter for the selection of the best experiment is to take into account only the ones with $\%retrieved > 45$ as the main priority is the model retrieval. After such filter is applied, we will rank each variable, where each rank will represent how far is the current value from the best value, and will be in percentage because not all the variables have the same range. We will have two types of rankings, decreasing ranking $ranking_{decr}$ from Equation 6.3 (the lower the variable

TABLE 6.10: Motion factor experiment variable parameters.

ID	f_{dist}	f_{energ}	f_{motion}
2.1	Equation 3.31	Equation 3.37	$0.7f_{dist} + 0.3f_{energ}$
2.2	Equation 3.31	Equation 3.38	$0.7f_{dist} + 0.3f_{energ}$
2.3	Equation 3.32	Equation 3.37	$0.7f_{dist} + 0.3f_{energ}$
2.4	Equation 3.32	Equation 3.38	$0.7f_{dist} + 0.3f_{energ}$
2.5	Equation 3.31	Equation 3.37	$0.3f_{dist} + 0.7f_{energ}$
2.6	Equation 3.31	Equation 3.38	$0.3f_{dist} + 0.7f_{energ}$
2.7	Equation 3.32	Equation 3.37	$0.3f_{dist} + 0.7f_{energ}$
2.8	Equation 3.32	Equation 3.38	$0.3f_{dist} + 0.7f_{energ}$
2.9	Equation 3.31	Equation 3.37	$0.5f_{dist} + 0.5f_{energ}$
2.10	Equation 3.31	Equation 3.38	$0.5f_{dist} + 0.5f_{energ}$
2.11	Equation 3.32	Equation 3.37	$0.5f_{dist} + 0.5f_{energ}$
2.12	Equation 3.32	Equation 3.38	$0.5f_{dist} + 0.5f_{energ}$
2.13	Equation 3.31	none	f_{dist}
2.14	Equation 3.32	none	f_{dist}
2.15	none	Equation 3.37	f_{energ}
2.16	none	Equation 3.38	f_{energ}
2.17	none	none	1

TABLE 6.11: Motion factor experiment results.

ID	Accumulated $dist_{2d}$ (m)	Accumulated e	%retrieved
2.1	14.495	5.923195	47.09
2.2	12.739	2.594932	39.09
2.3	11.564	5.468051	48.65
2.4	9.867	2.833164	36.81
2.5	16.432	4.947397	48.48
2.6	19.953	2.226102	42.39
2.7	14.419	4.766268	47.5
2.8	15.941	2.378528	38.56
2.9	14.425	5.04215	46.61
2.10	21.608	3.242325	44.42
2.11	12.716	5.517846	48.5
2.12	10.821	2.833164	36.81
2.13	19.889	9.564862	52.71
2.14	11.704	8.473659	50.39
2.15	27.481	3.757335	45.39
2.16	15.231	1.985302	39.96
2.17	33.626	11.962955	53.09

the better), and increasing ranking $ranking_{incr}$ from Equation 6.2 (the higher the variable the better), where $value_{max}$ and $value_{min}$ are the maximum and minimum values of the analyzed variable, and $value_{rank}$ is the current value that will be ranked.

$$ranking_{incr} = 100 * \left| \frac{value_{rank} - value_{max}}{value_{max}} \right| \quad (6.2)$$

$$ranking_{decr} = 100 * \left| \frac{value_{rank} - value_{min}}{value_{min}} \right| \quad (6.3)$$

The ranks that will have $ranking_{decr}$ are the distance d_{rank} for accumulated d , energy e_{rank} for accumulated e . $retr_{rank}$ for $\%retrieved$ will have $ranking_{incr}$. Finally, the total ranking will be calculated by adding all the rankings.

Table 6.12 shows all the experiments that passed the filtering with their corresponding rankings.

TABLE 6.12: Motion factor experiment results after filter.

ID	d_{rank}	e_{rank}	$retr_{rank}$	Total rank
2.1	25.346	57.644	11.302	94.291
2.3	0.000	45.530	8.363	53.893
2.5	42.096	31.673	8.683	82.453
2.7	24.689	26.852	10.529	62.070
2.9	24.741	34.195	12.206	71.141
2.11	9.962	46.855	8.646	65.463
2.13	71.991	154.565	0.716	227.271
2.14	1.211	125.523	5.086	131.819
2.15	137.643	0.000	14.504	152.146
2.17	190.782	218.389	0.000	409.171

On Figure 6.11 the retrieved models from the experiment that passed the filter can be shown, while the retrieved models from the experiment that did not pass the filter are on Figure 6.12, and it can be seen that although the difference between both sets of experiments is not exaggerate, there are some differences specially on the lower area where the ones from Figure 6.11 are better.

On Table 6.12 we can see that the best experiment is "2.3" which had the lowest total ranking, meaning that it was the closest to the best values, plus we can see that it reduced considerably the distance and energy compared to experiment "a" which is the one without motion factor, meaning that this factor fulfills its purpose. This indicates that the final motion factor f_{motion} that will be used on future experiments is will be:

$$f_{motion} = 0.7f_{dist2} + 0.3f_{energ1} \quad (6.4)$$

with f_{dist2} being Equation 3.32 and f_{energ1} being Equation 3.37. This f_{motion} can be seen on Figure 6.13, where we notice that the maximum of f_{motion} is when $d_{norm} = 0$ and $e = 0$, and its minimum is set at 0.1 when $d_{norm} = 1$ and $e = 1$, plus the surface is larger on the d_{norm} axis as it was given more weight.

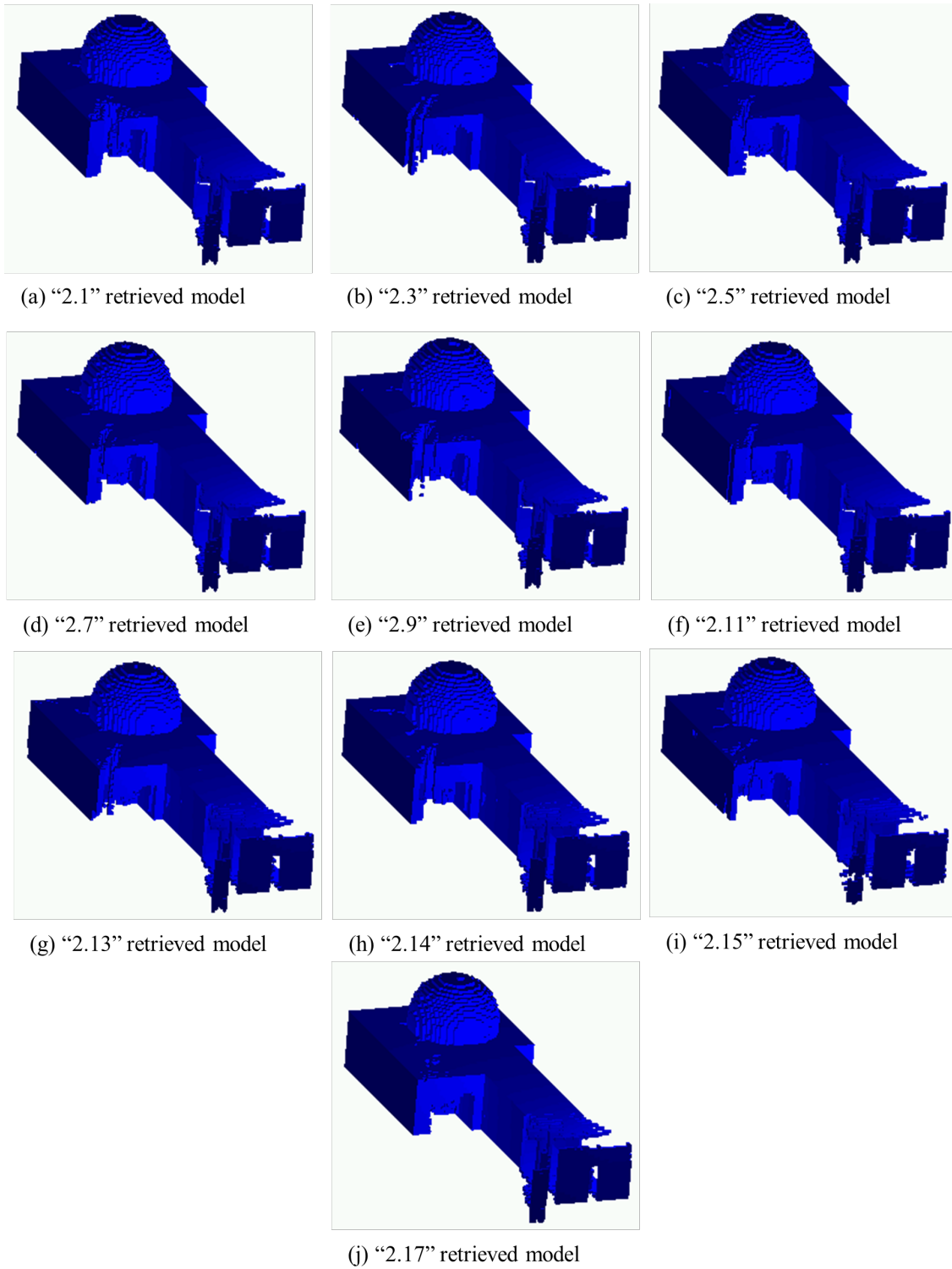


FIGURE 6.11: Motion factor experiment retrieved models greater than filter.

6.3.3 Quality factor experiments

The fixed parameters for all the experiments regarding the quality factor presented on section 3.3 are shown on Table 6.13. This parameters contain the objective function factors, global

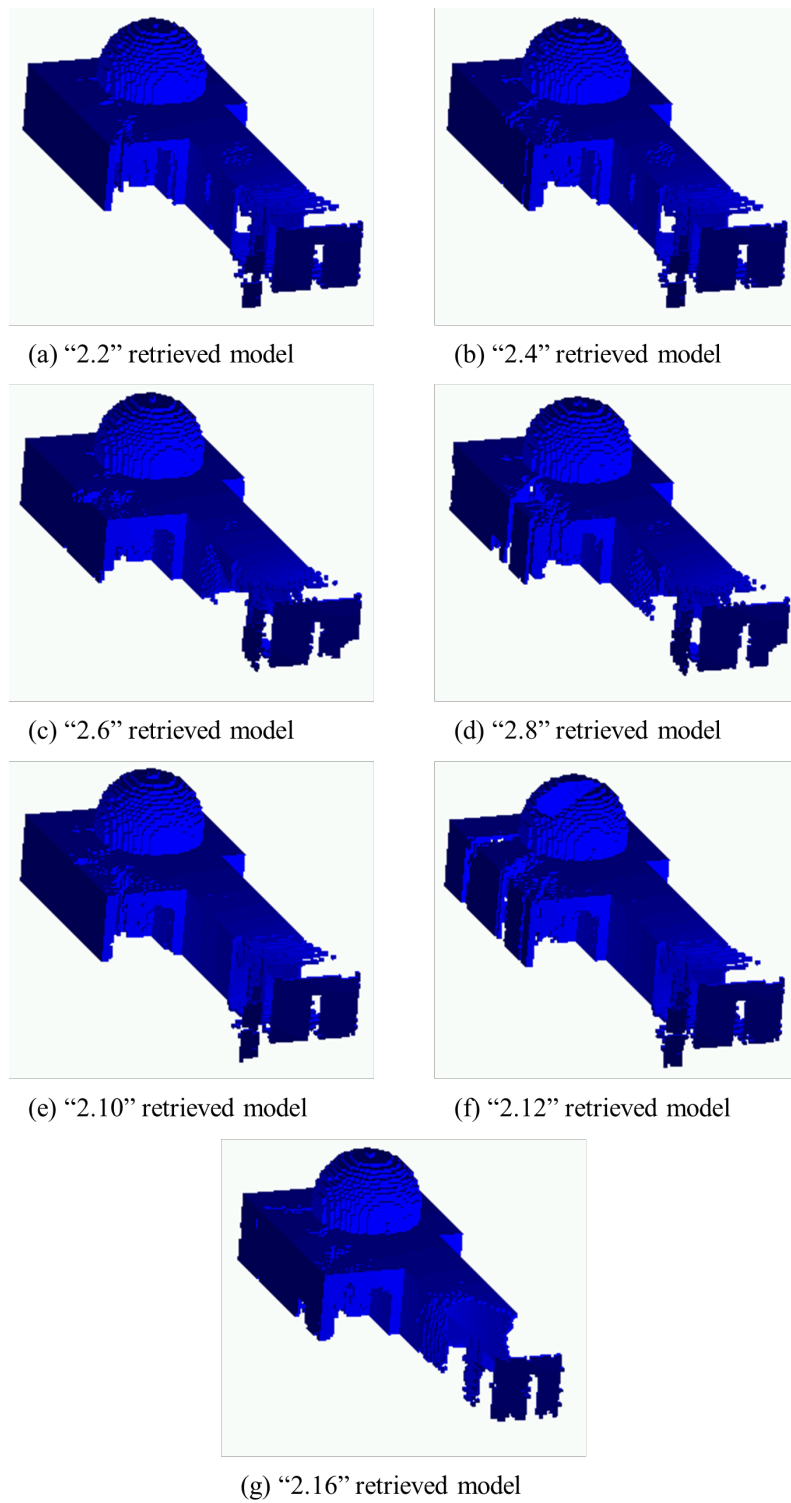


FIGURE 6.12: Motion factor experiment retrieved models smaller than filter.

optimization tasks and local optimization methods presented on Chapters 3 - 5.

The environment that will be used is the laboratory model which can be seen on Figure 6.5 with the information shown in Table 6.3.

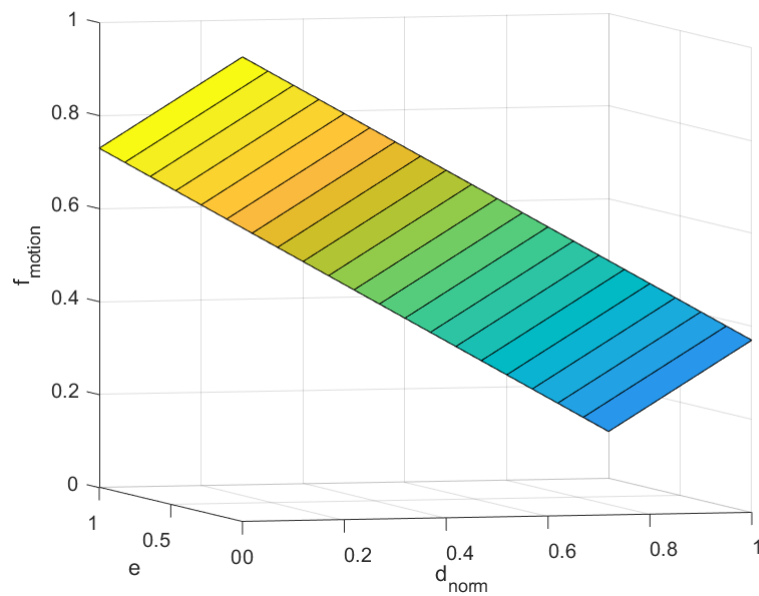


FIGURE 6.13: Motion factor function.

TABLE 6.13: Quality factor experiment simulation parameters.

Parameter	Value
General considerations	
Environment	laboratory model
Number of views	50
Safe cylinder	yes
Objective function	
Area factor	Equation 3.20, $\alpha = 0.4$
Distance factor	Equation 3.32
Energy factor	Equation 3.37
Motion factor	Equation 6.4
Quality factor	Equation 3.40
Total objective function	variable
Global optimization	
Minimum backstepping distance	no backstepping
Backstepping register	no backstepping
Dynamic change in area factor	no
Local optimization	
Method	Nelder-Mead with exhaustive search

The proposed experiments are presented on Table 6.14, where each experiment is represented with a code and has its corresponding combinations of f_{area} , f_{motion} and $f_{quality}$ that will form the total objective function f . We propose multiple combinations:

- No $f_{quality}$ at all, with f_{area} and f_{motion} having the same weight.

- f_{area} having the most weight, then f_{motion} and finally $f_{quality}$ with the least weight.
- f_{area} having the most weight, then $f_{quality}$ and f_{motion} with the same weight.
- f_{area} having the most weight, then $f_{quality}$ and finally f_{motion} with the least weight.
- All three f_{area} , $f_{quality}$ and f_{motion} with the same weight.
- f_{motion} having the most weight, then f_{area} and finally $f_{quality}$ with the least weight.
- f_{motion} having the most weight, then $f_{quality}$ and f_{area} with the same weight.
- f_{motion} having the most weight, then $f_{quality}$ and finally f_{area} with the least weight.

TABLE 6.14: Quality factor experiment variable parameters.

ID	Total function
3.1	$f = f_{area} * f_{motion}$
3.2	$f = f_{area}(0.7f_{motion} + 0.3f_{quality})$
3.3	$f = f_{area}(0.5f_{motion} + 0.5f_{quality})$
3.4	$f = f_{area}(0.3f_{motion} + 0.7f_{quality})$
3.5	$f = f_{area} * f_{motion} * f_{quality}$
3.6	$f = (0.7f_{area} + 0.3f_{quality})f_{motion}$
3.7	$f = (0.5f_{area} + 0.5f_{quality})f_{motion}$
3.8	$f = (0.3f_{area} + 0.7f_{quality})f_{motion}$

The results for all the experiments are shown on Table 6.15, and $mean_{quality}$ from Equation 3.39 is added as a new resulting variable.

TABLE 6.15: Quality factor experiment results.

ID	Accumulated $dist_{2d}$ (m)	Accumulated e	%retrieved	$mean_{quality}$
3.1	11.564	5.468051	48.65	0.672498
3.2	12.687	5.841227	46.08	0.680219
3.3	17.9	6.746774	45.83	0.719572
3.4	20.803	6.8769	43.21	0.745188
3.5	17.618	6.386356	45.92	0.700673
3.6	13.072	4.881258	42.96	0.691276
3.7	8.418	3.910273	35.73	0.713059
3.8	9.411	2.413618	26.93	0.660342

The first filter for the selection of the best experiment is to take into account only the ones with $\%retrieved > 45$ as the main priority is the model retrieval. After such filter is applied, we will rank each variable with the same technique as the previous experiment from subsection 6.3.2 although we now include an additional ranking in the form of $ranking_{incr}$ which will be $qual_{rank}$

for the mean quality. The total ranking will be calculated by adding all the rankings including the new quality one.

Table 6.16 shows all the experiments that passed the filtering with their corresponding rankings.

TABLE 6.16: Quality factor experiment results after filter.

ID	d_{rank}	e_{rank}	$retr_{rank}$	$qual_{rank}$	Total rank
3.1	0.000	0.000	0.000	6.542	6.542
3.2	9.711	6.825	5.283	5.469	27.287
3.3	54.791	23.385	5.797	0.000	83.973
3.5	52.352	16.794	5.612	2.626	77.384

On Figure 6.14(a) through Figure 6.14(c) and 6.14(e) we can see the retrieved models from the experiment that did pass the filter, while on Figure 6.14(d) and Figure 6.14(f) through Figure 6.14(h) the retrieved models from the experiment that did not pass the filter are shown, and it can be seen that although the difference between both sets of experiments is not exaggerate, there are some differences specially on the lower area although Figure 6.14(h) has indeed an exaggerate difference making it the worst. On Table 6.16 we can see that the best experiment is "3.1" which had the lowest total ranking, meaning that it was the closest to the best values. This indicates that the final objective function f that will be used on future experiments is $f = f_{area} * f_{motion}$ without any $f_{quality}$, that although we can see that it upgrades the quality fulfilling its purpose, when making the balance with all the other resulting variables it is not convenient.

6.4 Global optimization experiments

In this section only the global optimization tasks will be changed while keeping the local optimization method and objective function as constant.

6.4.1 Backstepping experiments

The fixed parameters for all the experiments regarding the backstepping solution presented on section 4.1 are shown on Table 6.17. This parameters contain the objective function factors, global optimization tasks and local optimization methods presented on Chapters 3 - 5.

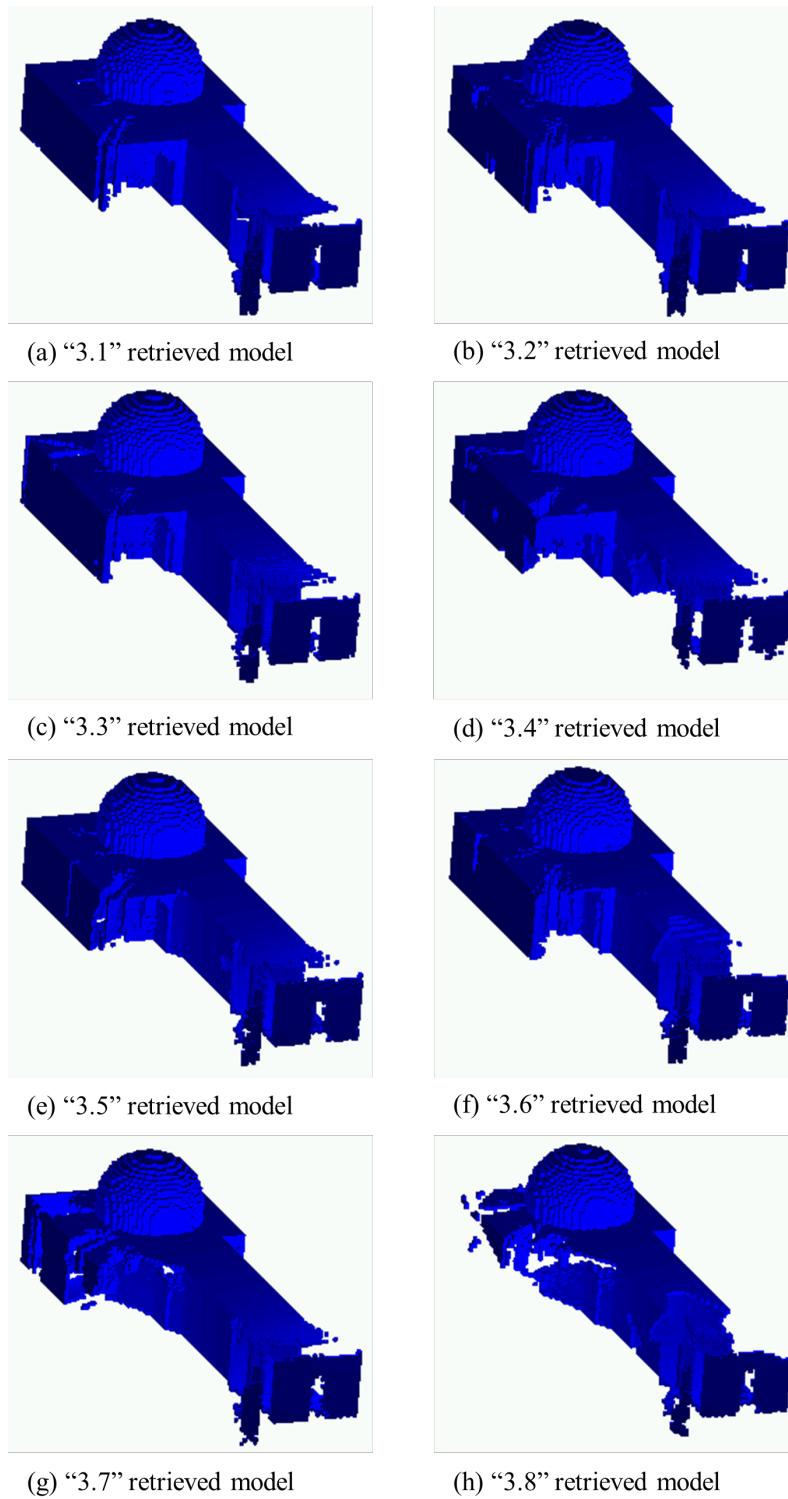


FIGURE 6.14: Quality factor experiment retrieved models.

The environment that will be used is the laboratory sub-model which can be seen on Figure 6.7 with the information shown in Table 6.4. The reason why we will use this sub-model instead of the original laboratory model is because we want a more contained experiment as the goal is to understand the behaviour of the robot with backstepping, we do not need a reconstruction of

TABLE 6.17: Backstepping experiment simulation parameters.

Parameter	Value
General considerations	
Environment	laboratory sub-model
Number of views	50
Safe cylinder	no
Objective function	
Area factor	Equation 3.20, $\alpha = 0.4$
Distance factor	Equation 3.32
Energy factor	Equation 3.37
Motion factor	Equation 6.4
Quality factor	none
Total objective function	$f_{area} * f_{motion}$
Global optimization	
Minimum backstepping distance	variable
Backstepping register	variable
Dynamic change in area factor	no
Local optimization	
Method	Nelder-Mead with exhaustive search

the whole environment from the laboratory model, using the sub-model involves less movement range for the robot as it is smaller. Also, we decided to turn off the Safe cylinder so our robot will be treated with a size of a single voxel regarding collision analysis, and although it may seem as an exaggeratedly ideal condition which will never happen on real life, it is used just for giving it more freedom to get the most out of the backstepping analysis, later it will be turned on again as it was before this experiment.

The proposed experiments are presented on Table 6.18, where each experiment is represented with a code and has its corresponding minimum backstepping distance plus if they include backstepping register or not. It was decided that the distance will go from 1 to 4 meters to avoid extremely big distances, and we will include a distance of 0 meters to represent the experiments that do not consider a minimum distance, and we will also include an experiment in which no backstepping was applied at all.

The results from each experiment are summarized on Table 6.19.

The retrieved model from each experiment is presented on Figure 6.15, and we can see that the ones from Figure 6.15(a) through Figure 6.15(d) are considerably worst, meaning that at small backstepping distances it behaves badly.

TABLE 6.18: Backstepping experiment variable parameters.

ID	Minimum backstepping distance	Backstepping register
4.1	no	no
4.2	0m	no
4.3	0m	yes
4.4	1m	no
4.5	1m	yes
4.6	2m	no
4.7	2m	yes
4.8	3m	no
4.9	3m	yes
4.10	4m	no
4.11	5m	yes

TABLE 6.19: Backstepping experiment results.

ID	Accumulated $dist_{2d}$ (m)	Accumulated e	%retrieved	$mean_{quality}$
4.1	9.510	2.380	41.470	61.152
4.2	24.974	5.717	43.340	0.620
4.3	19.187	5.578	44.260	0.624
4.4	39.619	6.602	49.800	0.649
4.5	42.550	8.807	74.630	0.716
4.6	82.597	7.103	70.100	0.713
4.7	82.597	7.103	70.100	0.713
4.8	131.075	6.060	71.570	0.707
4.9	82.981	6.475	79.260	0.743
4.10	139.870	9.388	64.160	0.655
4.11	78.107	6.487	66.280	0.679

For choosing the best experiment we will only consider *%retrieved*, as now our goal is to get the robot out of positions where it gets stuck, meaning that it will retrieve more sections of the model. According to this criteria, we can see on Table 6.19 that the experiment which recovered the most of the model was "4.9", meaning that a minimum backstepping distance of 3 meters and implementing backstepping register are the optimal parameters that will be used on all upcoming experiments.

6.4.2 Dynamic change in objective function experiments

The fixed parameters for all the experiments regarding the dynamic change in objective function presented on section 4.2 are shown on Table 6.20. This parameters contain the objective function factors, global optimization tasks and local optimization methods presented on Chapters 3 - 5.

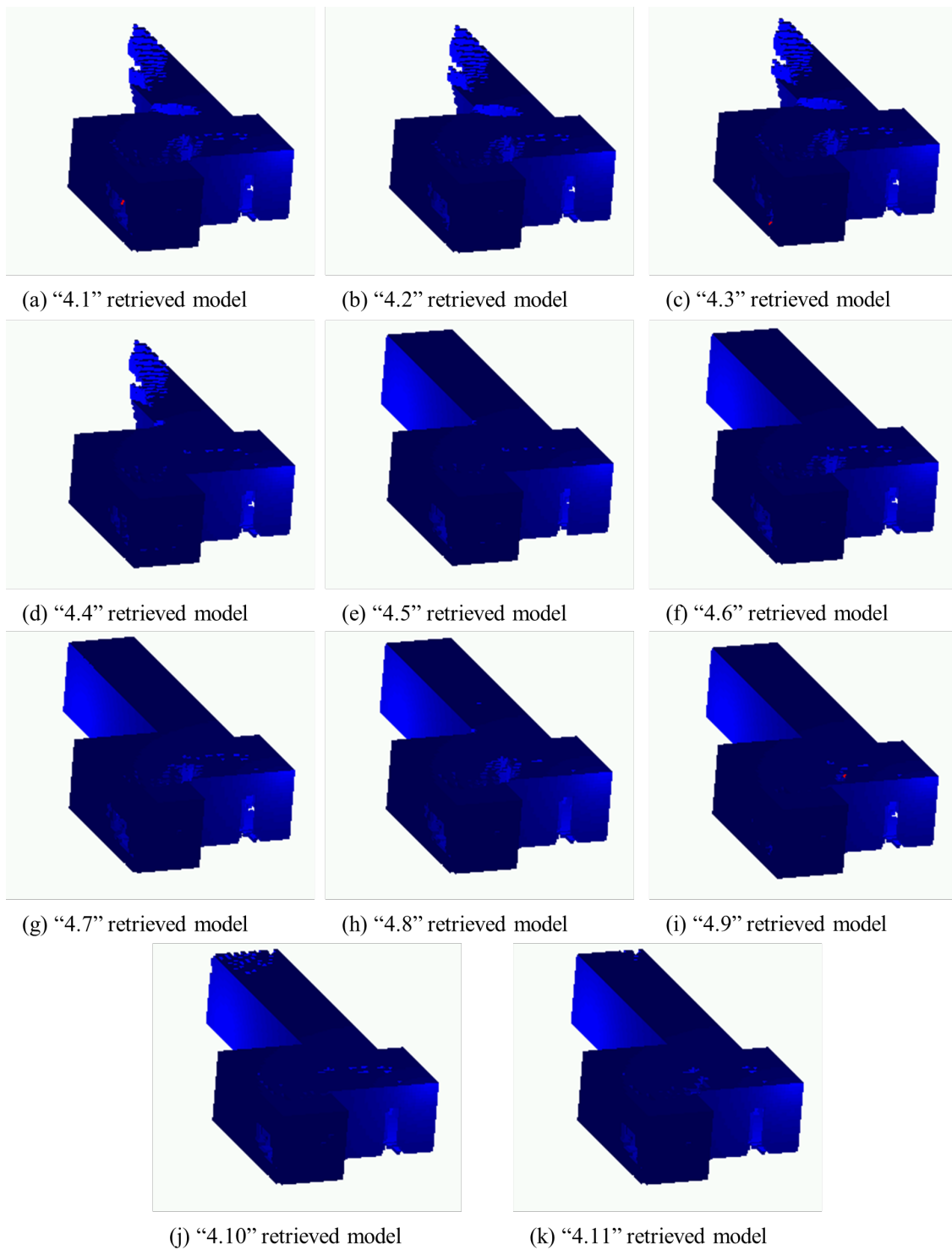


FIGURE 6.15: Backstepping experiment retrieved models.

The environment that will be used is the laboratory model which can be seen on Figure 6.5 with the information shown in Table 6.3. At this point the goal is now to retrieve the whole model, we decided to increase the number of views from 50 to 100 to give the dynamic change in objective function more settling time, as we noticed on previous experiments that the change

TABLE 6.20: Dynamic change in objective function experiment simulation parameters.

Parameter	Value
General considerations	
Environment	laboratory model
Number of views	100
Safe cylinder	yes
Objective function	
Area factor	Equation 3.20, $\alpha = 0.4$
Distance factor	Equation 3.32
Energy factor	Equation 3.37
Motion factor	Equation 6.4
Quality factor	none
Total objective function	$f_{area} * f_{motion}$
Global optimization	
Minimum backstepping distance	3 m
Backstepping register	yes
Dynamic change in area factor	variable
Local optimization	
Method	Nelder-Mead with exhaustive search

in objective function is required on views close to the view #50, so if we decided to maintain the 50 view limit the new function would be working just for very few views which would not be enough to observe its true impact.

We will assign the ID "5.1" for the experiment without dynamic change in objective function and "5.2" for the experiment with it, there is no need to create a table for the experimental matrix as there are no more parameters. The results from each experiment are summarized on Table 6.21.

TABLE 6.21: Dynamic change in objective function experiment results.

ID	Accumulated $dist_{2d}$	Accumulated e	%retrieved	$mean_{quality}$
5.1	60.471	9.595315	70.3	0.748172
5.2	353.478	20.043933	88.28	0.813102

The retrieved model from each experiment is presented on Figure 6.16, and we can see that the difference is on the lower left corner. A zoomed comparison is shown on Figure 6.17 where we notice on Figure 6.17(b) that when applying the dynamic change in objective function that specific corner is retrieved better.

On Table 6.21 we can see that when the dynamic change in objective function was applied the distance and energy increased, but we will still keep considering just %retrieved as our only criteria for this experiment as this auxiliary aid was designed to improve the retrieved model,

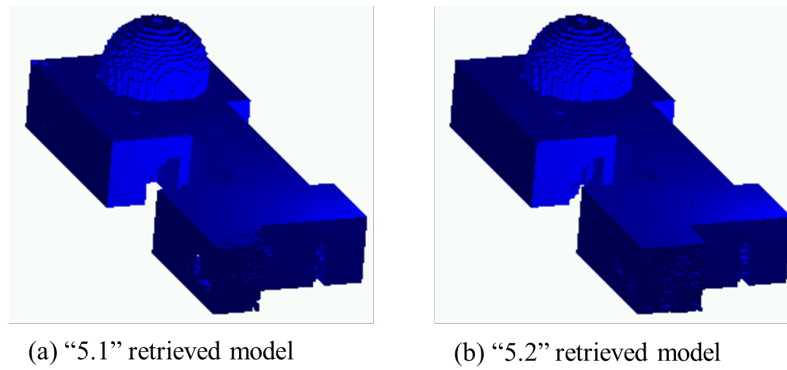


FIGURE 6.16: Dynamic change in objective function experiment retrieved models.

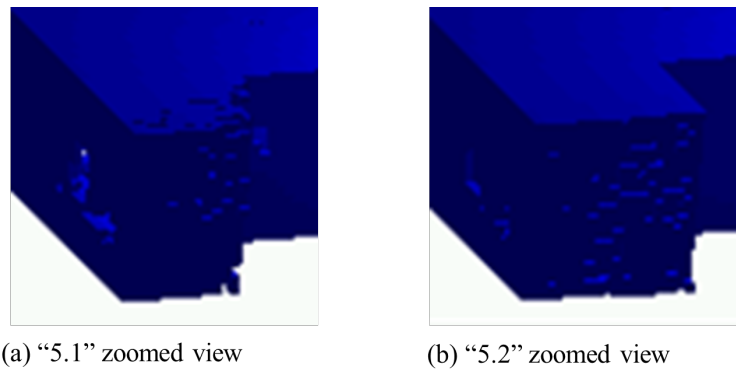


FIGURE 6.17: Dynamic change in objective function zoomed comparison.

and the best *%retrieved* was achieved on experiment "5.2" when applying the dynamic change in objective function making it the best experiment.

This was the last auxiliary aid we defined, meaning that now we have all the best parameters except for the optimization methods which will be dealt in the upcoming section 6.5.

6.5 Local optimization experiments

Now that we have found the best objective function configuration and global optimization tasks using the Nelder-Mead method with exhaustive search, we will test all the other proposed optimization algorithms from Chapter 5 to see if changing the method has an impact on the final result and computation time. Table 6.22 shows the parameters that will apply to every optimization experiment with all of them being fixed except for the variable optimization method.

This set of experiments will be on each method to find their best parameters or modifications, with all of them having the laboratory model as their environment, which can be seen on Figure 6.5 with the information shown in Table 6.3.

TABLE 6.22: Local optimization method simulation parameters.

Parameter	Value
General considerations	
Environment	laboratory model
Number of views	100
Safe cylinder	yes
Objective function	
Area factor	Equation 3.20, $\alpha = 0.4$
Distance factor	Equation 3.32
Energy factor	Equation 3.37
Motion factor	Equation 6.4
Quality factor	none
Total objective function	$f_{area} * f_{motion}$
Global optimization	
Minimum backstepping distance	3 m
Backstepping register	yes
Dynamic change in area factor	to Equation 4.1
Local optimization	
Method	variable

6.5.1 Nelder-Mead

This first method is the one used in all the previous experiments although it was Nelder-Mead for the position variables and exhaustive search for the orientation. In this subsection we will keep that two-phase method as an experiment and include another one in which no exhaustive search is used and all 4 variables (NBV_x , NBV_y , NBV_z and v) are included in a pure Nelder-Mead algorithm.

We will assign the ID "6.1" for the experiment with exhaustive search and "6.2" for the experiment without it, there is no need to create a table for the experimental matrix as there are no more parameters. The results from each experiment are summarized on Table 6.21. The results are shown on Table 6.23 where a new column for the total time is added as it is one key element of optimization methods.

TABLE 6.23: Nelder-Mead experiment results.

ID	Accumulated $dist_{2d}$ (m)	Accumulated e	%retrieved	$mean_{quality}$	Total time (s)
6.1	353.478	20.043933	88.28	0.813102	3594.346667
6.2	327.351	19.115758	89.51	0.806914	297.066667

On Figure 6.18(a) and Figure 6.18(c) we can see the retrieved models and now we include a negative of the models on 6.18(b) and 6.18(d) as it is more useful to notice the interior details.

This negative consists of all the voxels that are not yet defined as occupied or empty, which are occluded voxels in yellow, occplane voxels on cyan and unmarked voxels on purple. It can be seen that there are zero unmarked voxels, while the occluded and occplane voxels correspond to the outside of the model and specific areas such as some of the furniture which are unfeasible or extremely difficult to reach due to the physical limitations of the robot, we consider this as good results on both experiments given the limitations and complexity of the model, and experiment "6.2" seems to have less occplane voxels which is coherent as it has a bigger %retrieved.

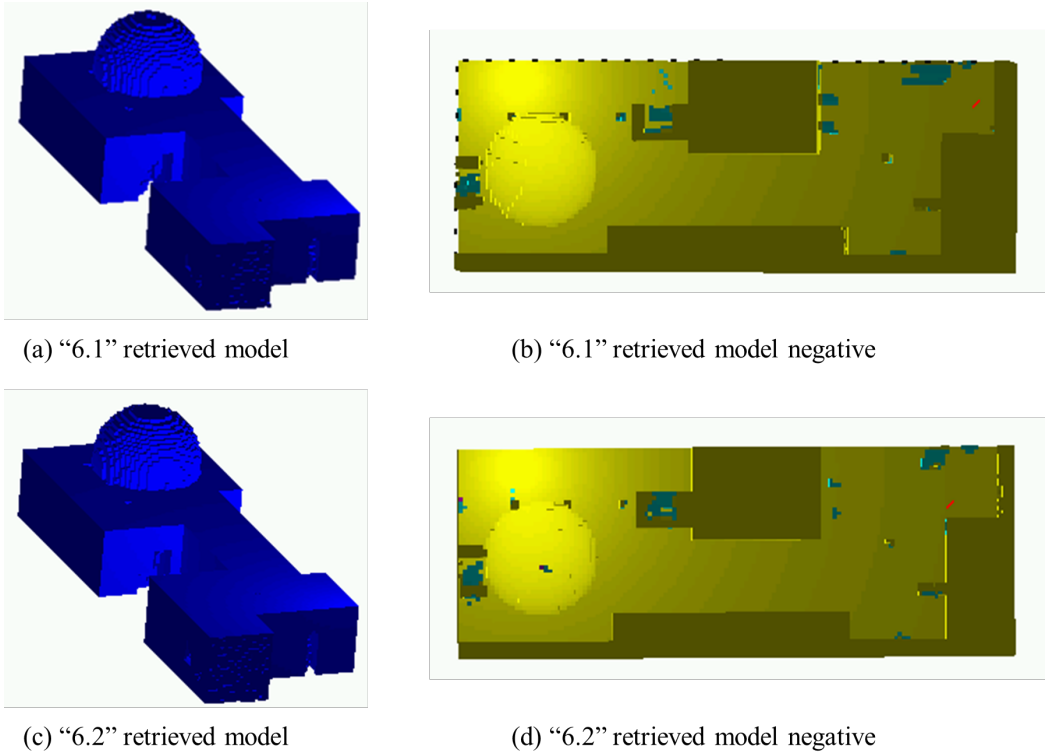


FIGURE 6.18: Nelder-Mead experiment retrieved models.

Seems obvious that the best experiment is "6.2" as it has the best %retrieved and computational time. However, we will still make a deeper comparison including ranks proposed on previous experiments (distance, energy and %retrieved ranks) plus a new time rank $time_{rank}$ that will be in the form of $ranking_{decr}$. A global rank for the objective function which assigns weights for each factor will be computed as:

$$f_{rank} = 0.5retr_{rank} + 0.5(0.7d_{rank} + 0.3e_{rank}) \quad (6.5)$$

and the new total rank $total_{rank}$ will add the function rank plus the time rank having both the same weight, being computed as $total_{rank} = f_{rank} + time_{rank}$. This ranking technique will be the one used on all optimization method experiments as it provides a balance of the objective

function and computational time which are the target and main elements of comparison among all optimization methods. The ranked results are shown on Table 6.24.

TABLE 6.24: Nelder-Mead experiment ranked results.

ID	d_{rank}	e_{rank}	$retr_{rank}$	f_{rank}	$time_{rank}$	$total_{rank}$
6.1	7.981	4.856	1.374	4.209	1109.946	1114.155
6.2	0	0.000	0.000	0.000	0.000	0.000

We confirm that the best experiment is indeed "6.2" as it has the lowest total rank, and we notice that all its ranks are 0.000 meaning that it is the best experiment in each criterion. This gets us to the conclusion that exhaustive search for orientation on this specific problem is not necessary and it has a negative impact where the increase on the computational time is the most drastic. We should use the pure Nelder-Mead algorithm without exhaustive search.

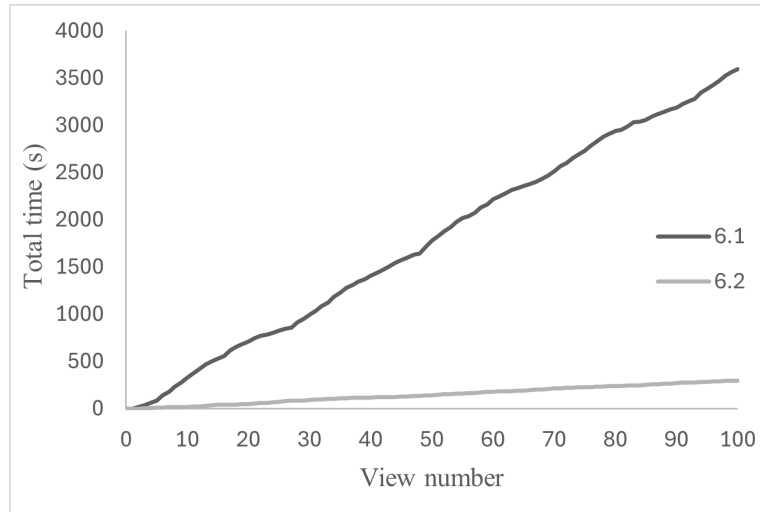


FIGURE 6.19: Nelder-Mead experiment time comparison.

6.5.2 (1+1)-ES

The first set of experiments of this algorithm will be for obtaining the best value of the parameter $\sigma_{initial}$ with $gen_{max} = 500$ as discussed on subsection 5.3.2. A common value of $\sigma_{initial}$ is 1 although we have the hypothesis that it is small compared to the range of our variables and it may provide a small change making the algorithm explore less possibilities, that is why we will propose bigger numbers on other experiments starting on 5 with 2.5 steps until 10. Table 6.25 shows all IDs with their corresponding parameters.

The results of each experiment are shown on Table 6.26 with their corresponding retrieved models on Figure 6.20.

TABLE 6.25: (1+1)-ES $\sigma_{initial}$ experiment variable parameters.

ID	c	gen_{max}	$\sigma_{initial}$
7.1	0.817	500	1
7.2	0.817	500	5
7.3	0.817	500	7.5
7.4	0.817	500	10

TABLE 6.26: (1+1)-ES $\sigma_{initial}$ experiment results.

ID	Accumulated $dist_{2d}(m)$	Accumulated e	%retrieved	$mean_{quality}$	Total time (s)
7.1	253.761	14.50639	56.42	0.751124	913.133333
7.2	255.649	17.796031	83.06	0.771461	508.706667
7.3	331.917	21.594847	84.55	0.779341	435.706667
7.4	283.684	18.766537	85.03	0.774688	378.906667

We immediately notice that the first experiment provides a considerably low value of %retrieved and the model looks pretty incomplete on Figure 6.20(a) and Figure 6.20(b).

In order to be able to apply the ranking for decision making we will filter the results by allowing only the ones with %retrieved greater than 80% and for those we will apply the same ranking technique as the previous method, with Table 6.27 showing the ranked filtered results.

TABLE 6.27: (1+1)-ES $\sigma_{initial}$ experiment ranked results.

ID	d_{rank}	e_{rank}	$retr_{rank}$	f_{rank}	$time_{rank}$	$total_{rank}$
7.2	0.000	0.000	2.317	1.158	34.256	35.415
7.3	29.833	21.346	0.565	13.926	14.990	28.916
7.4	10.966	5.453	0.000	4.656	0.000	4.656

The experiment with the smaller total rank is "7.4", meaning that the best value for $\sigma_{initial}$ is 10. Choosing that parameter value we end up with a total time of 378.9s which can be further reduced by modifying the value of gen_{max} that currently is the only stopping mechanism.

We will make an analysis by repeating the experiment "7.4" that had $gen_{max} = 500$ and store information of the first generation gen that has a relative error of 10% or less compared to the actual value at $gen = 500$. The information stored will be the value of such gen with its corresponding values of m and σ at that specific generation as we want to see which of those three variables represents a better stopping mechanism due to them being the only changing variables from the algorithm.

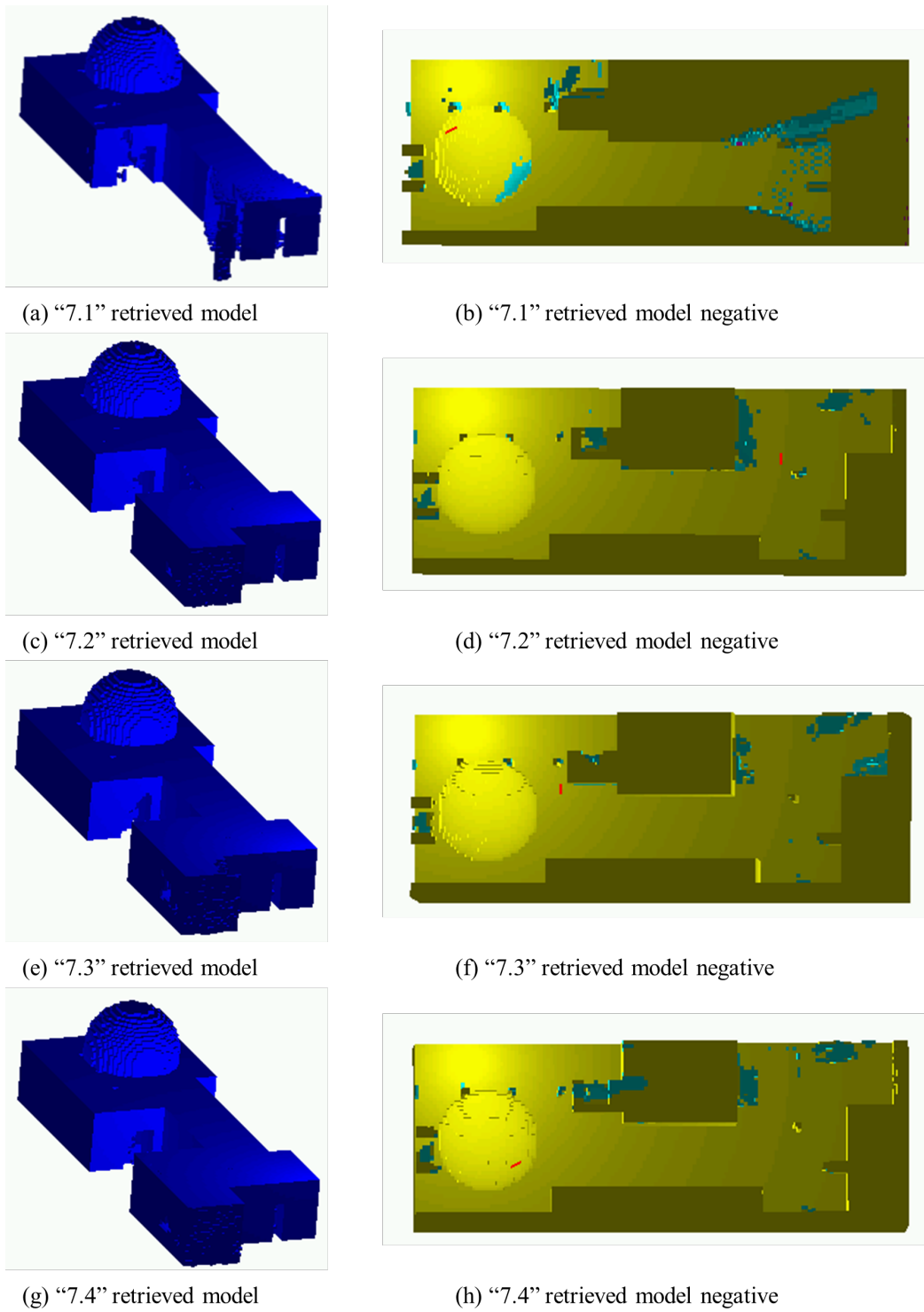
FIGURE 6.20: (1+1)-ES $\sigma_{initial}$ experiment retrieved models.

Table 6.28 shows a summary of the stored results from all 100 views where we have their limits and range plus their mean and standard deviation in order to compute the coefficient of variation as $\frac{100 \times \text{standard deviation}}{\text{mean}}$ that will help us to decide how constant each variable is.

The most constant variable is *gen* which has the smallest coefficient of variation. Although

TABLE 6.28: (1+1)-ES stopping mechanism analysis.

	gen	m	σ
Upper limit	442	15	4.455
Lower limit	88	1	0.117
Range	354	14	4.338
Mean	307.277	4.877	0.748
Standard deviation	86.605	3.389	0.842
Coefficient of variation (%)	28.185	69.488	112.556

28.18% may seem as a big coefficient of variation, it still falls under the acceptable range which is up to 30% as explained on [63]. With this results, we get to the conclusion that no other variable is acceptable as a stopping mechanism, it will still be gen_{max} although now it can be reduced up to $gen_{max} = 308$ which is the mean of gen rounded to the next integer.

To test that by reducing gen_{max} the time is reduced and the final result has not been compromised we will compare it to the original experiment. Table 6.29 shows both IDs with their corresponding gen_{max} .

TABLE 6.29: (1+1)-ES gen_{max} experiment variable parameters.

ID	c	gen_{max}	$\sigma_{initial}$
7.4.1	0.817	500	10
7.4.2	0.817	308	10

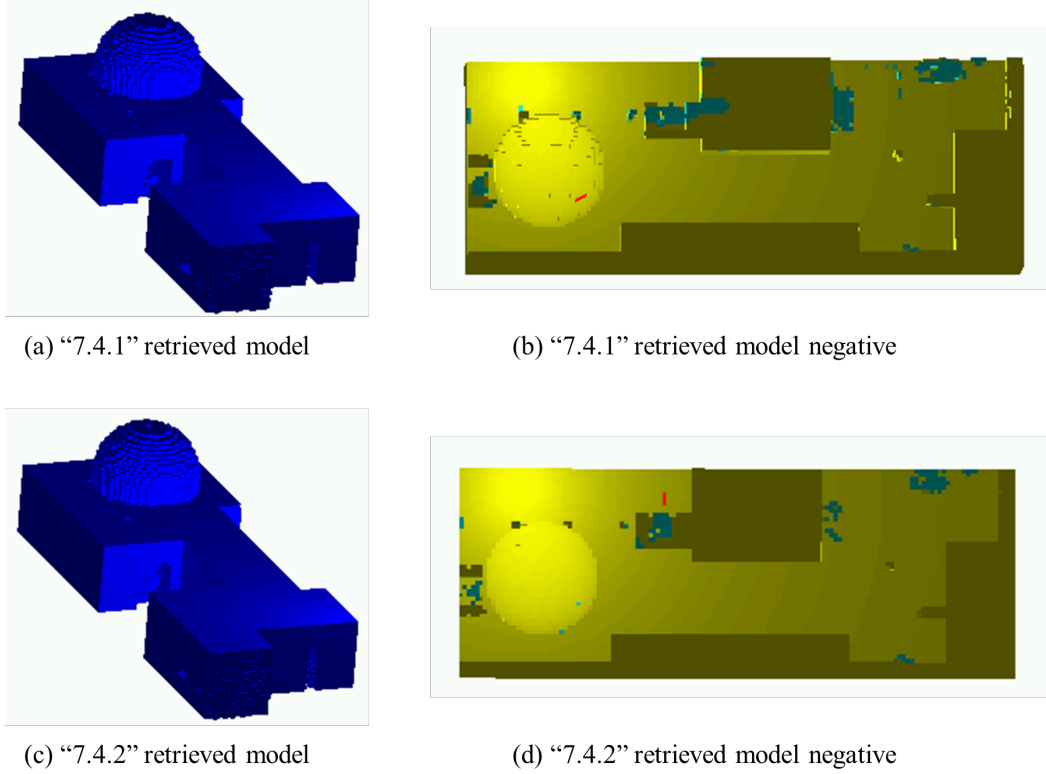
The results of each experiment are shown on Table 6.30 and their retrieved models on Figure 6.21.

TABLE 6.30: (1+1)-ES gen_{max} experiment results.

ID	Accumulated $dist_{2d} (m)$	Accumulated e	%retrieved	$mean_{quality}$	Total time (s)
7.4.1	283.684	18.766537	85.03	0.774688	378.906667
7.4.2	272.077	16.543897	87.78	0.799471	120.493333

We can see that the total time is indeed reduced on "7.4.2" and the difference between both retrieved models is not noticeable at least visually on Figure 6.21 meaning that the final result does not seem to be compromised. Still, we will rank both experiments using the same ranking techniques as before plus we do not need to filter them in this case as both of them have a %retrieved greater than %80. The ranked results are shown on Table 6.31.

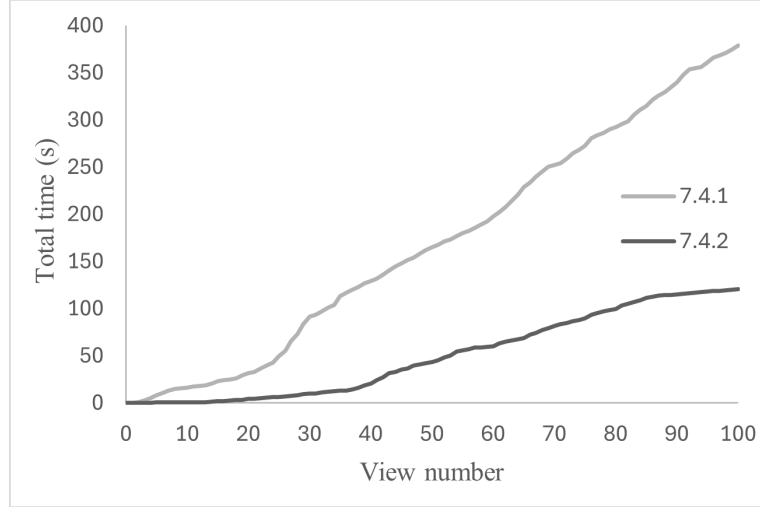
With this results we can confirm that "7.4.2" is the best experiment, as having $gen_{max} = 308$ reduces the computational time as seen on Figure 6.22 and it even upgrades the distance, energy, % retrieved and the mean quality. This upgrade in all the other factors may seem illogical as we

FIGURE 6.21: (1+1)-ES gen_{max} experiment retrieved models.TABLE 6.31: (1+1)-ES gen_{max} experiment ranked results.

ID	d_{rank}	e_{rank}	$retr_{rank}$	f_{rank}	$time_{rank}$	$total_{rank}$
7.4.1	4.266	13.435	3.133	5.075	214.463	219.538
7.4.2	0.000	0.000	0.000	0.000	0.000	0.000

cannot get a better value for the objective function with less generations and in fact we did not, the results from the objective function were worst on some iterations, but our upgrade is not on individual views but on the final result. This could be caused by the sensor falling on a worst view on some iteration and therefore the conditions for the future iterations change completely making the robot follow a different path, causing this downgrade on a certain iteration to cause a future upgrade on the global result. This may not always happen, but the important thing is that we did not see any substantial negative impact and we will take the little upgrade as circumstantial but the big impact on the time is indeed considerable.

Now that we know that we should use $\sigma_{initial} = 10$ and $gen_{max} = 308$, we will analyse if we should use the tabu search technique on this (1+1)-ES algorithm, and if so which parameter values to use. Table 6.32 shows all IDs with their parameters, where we keep the values of c , gen_{max} and $\sigma_{initial}$ as the best values from the previous experiments and we just add $tabu_{span}$ that is the number of banned generations from all tabu elements. We will start by 1 which is

FIGURE 6.22: (1+1)-ES gen_{max} experiment time comparison.

the smallest possible and we propose 5, 10 and 15 as the rest of the values as any other bigger number would cause trouble by providing the possibility that all tabu elements get banned on some iteration and the algorithm gets stuck. We will also include the experiment that we already exposed without tabu search as a point of comparison to see if this modifications is for the best or for the worst.

TABLE 6.32: (1+1)-ES with tabu search experiment variable parameters.

ID	c	gen_{max}	$\sigma_{initial}$	$tabu_{span}$
7.4.2.1	0.817	308	10	1
7.4.2.2	0.817	308	10	5
7.4.2.3	0.817	308	10	10
7.4.2.4	0.817	308	10	15
7.4.2.5	0.817	308	10	no tabu search

The results of each experiment are shown on Table 6.33 with their retrieved models being shown on Figure 6.23.

TABLE 6.33: (1+1)-ES with tabu search experiment results.

ID	Accumulated $dist_{2d}$ (m)	Accumulated e	%retrieved	$mean_{quality}$	Total time (s)
7.4.2.1	371.142	23.300107	85.75	0.793532	160.173333
7.4.2.2	367.3	23.11972	88.6	0.813989	151.146667
7.4.2.3	391.974	20.915059	88.72	0.809264	137.853333
7.4.2.4	328.328	22.608159	87.44	0.806478	154.293333
7.4.2.5	272.077	16.543897	87.78	0.799471	120.493333

We can see that the %retrieved increases when $tabu_{span}$ increases until $tabu_{span} = 10$ which is its peak, then it decreases. The difference between all retrieved models is not noticeable at least

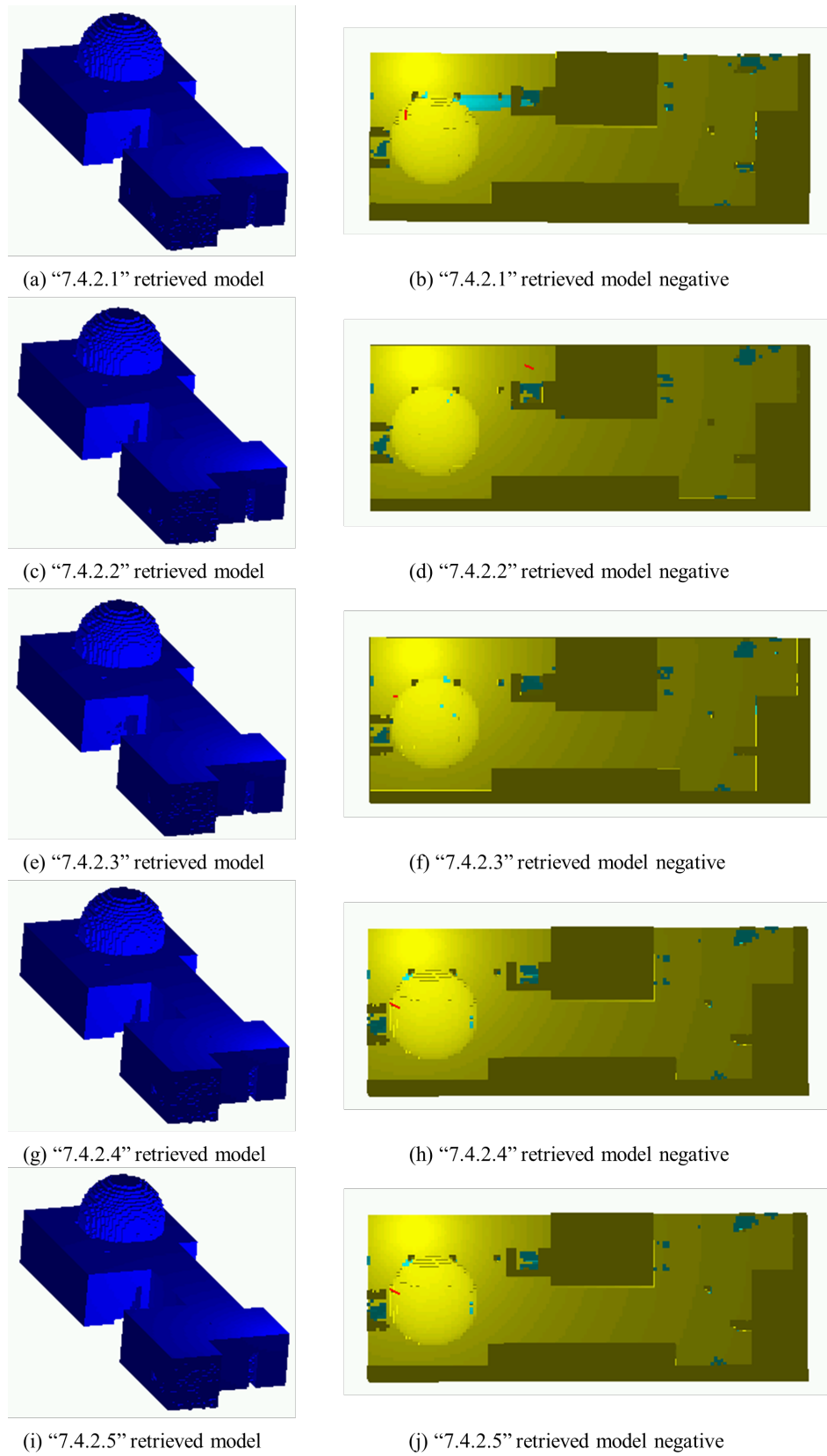


FIGURE 6.23: (1+1)-ES with tabu search experiment retrieved models.

visually on Figure 6.23. We will rank the experiments using the same ranking techniques as before plus we do not need to filter them in this case as all of them have acceptable %retrieved greater than %85, there is no outlier. The ranked results are shown on Table 6.34.

TABLE 6.34: (1+1)-ES with tabu search experiment ranked results.

ID	d_{rank}	e_{rank}	$retr_{rank}$	f_{rank}	$time_{rank}$	$total_{rank}$
7.4.2.1	36.411	40.838	3.348	20.543	32.931	53.475
7.4.2.2	34.999	39.748	0.135	18.279	25.440	43.719
7.4.2.3	44.067	26.422	0.000	19.387	14.407	33.794
7.4.2.4	20.675	36.656	1.443	13.456	28.051	41.507
7.4.2.5	0.000	0.000	1.060	0.530	0.000	0.530

We can see that the best experiment with tabu search was "7.4.2.3" when $tabu_{span} = 10$ as it is the one with the lowest total rank besides "7.4.2.5" that has no tabu search although the best global experiment was indeed "7.4.2.5", meaning that the implementation of tabu search is not recommended. Also, we kept track of the number of time the robot needed to perform backstepping in the experiments having tabu search and they were in the range of 21 – 24 even bigger than the 19 times it was done without it on experiment 7.4.2.5 indicating that even if tabu search is designed to escape local minima it does not adapt well to our specific problem and we still cannot solve it without our implemented backstepping technique. Our conclusion regarding this algorithm is that the best parameters should be $gen_{max} = 308$ and $\sigma_{initial} = 10$ and no tabu search should be used.

6.5.3 Simulated annealing

This last method requires only one parameter variation which will be L . Table 6.35 shows all IDs with their parameters, where we keep the values of T_0 and $cool_{ratio}$ constant with L being the only changing parameter. As it refers to the number of iterations, we will go for change of 100 within each experiment starting at 100 up to 500, exceeding this last value would cause the algorithm to have a significant execution time which we desire to decrease.

TABLE 6.35: Simulated annealing experiment variable parameters.

ID	T_0	$cool_{ratio}$	L
8.1	19.5	0.85	100
8.2	19.5	0.85	200
8.3	19.5	0.85	300
8.4	19.5	0.85	400
8.5	19.5	0.85	500

The results of each experiment are shown on Table 6.36 with their retrieved models being shown on Figure 6.24.

TABLE 6.36: Simulated annealing experiment results.

ID	Accumulated $dist_{2d} (m)$	Accumulated e	%retrieved	$mean_{quality}$	Total time (s)
8.1	236.371	18.628175	77.91	0.77563	54.333333
8.2	272.426	23.731783	81.04	0.788388	133.413333
8.3	317.389	22.771662	87.69	0.803419	176.306667
8.4	311.744	28.826643	89.38	0.808672	228.68
8.5	389.03	22.685377	89.43	0.815767	254.666667

We can see that both the %retrieved and total time increase when L increases. The difference between all retrieved models is not noticeable at least visually on Figure 6.24(e) through Figure 6.24(j) where %retrieved is greater than %85, but on Figure 6.24(a) through Figure 6.24(d) we can see a downgrade on the model especially on the negative representation where we have more occplane voxels on the top right corner. We will filter the experiments so than only the ones with %retrieved greater than %85 are kept as the others are considered outliers, and we will apply the same ranking techniques as before only for the filtered experiments. The ranked results are shown on Table 6.37.

TABLE 6.37: Simulated annealing experiment ranked results.

ID	d_{rank}	e_{rank}	$retr_{rank}$	f_{rank}	$time_{rank}$	$total_{rank}$
8.3	1.811	0.380	1.946	1.664	0.000	1.664
8.4	0.000	27.071	0.056	4.089	29.706	33.794
8.5	24.791	0.000	0.000	8.677	44.445	53.122

We confirm that "8.3" is the most balanced experiment as is the one with lowest total rank, meaning that the best value for the parameter L is 300, even if on other experiments the %retrieved was greater but the downgrade on all the other factors including time was greater in proportion.

6.6 Final comparisons

Now that we obtained the best objective function, global optimization tasks and best versions of our main local optimization methods with their best parameters, we will test this three methods on all of our models excluding the laboratory sub-model from subsection 6.1.4 as this is a section of another model and the aim of this section is to test our algorithm on different scenarios.

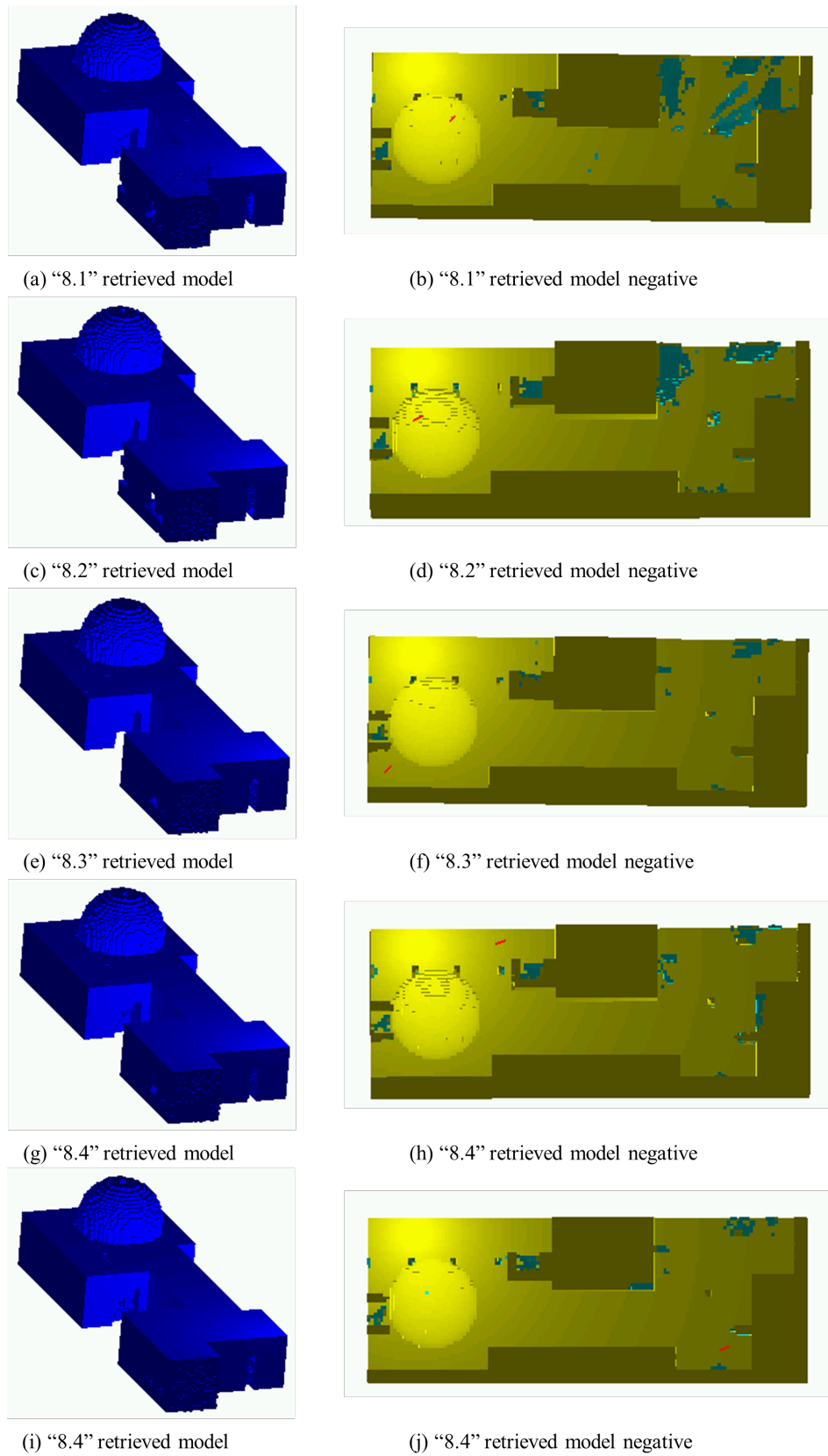


FIGURE 6.24: Simulated annealing experiment retrieved models.

6.6.1 Room with objects model comparisons

This first environment is the model from Figure 6.1 with the information shown in Table 6.1. The simulation parameters for this experiments are shown on Table 6.38 and we will choose the number of views as 50 because it is a relatively simple model. Still, it was tested with 200 views and we noticed that the improvement on the retrieved model was negligible so we kept the experiments with 50 views.

TABLE 6.38: Room with objects model experiment simulation parameters.

Parameter	Value
General considerations	
Environment	room with objects
Number of views	50
Safe cylinder	yes
Objective function	
Area factor	Equation 3.20, $\alpha = 0.4$
Distance factor	Equation 3.32
Energy factor	Equation 3.37
Motion factor	Equation 6.4
Quality factor	none
Total objective function	$f_{area} * f_{motion}$
Global optimization	
Minimum backstepping distance	3 m
Backstepping register	yes
Dynamic change in area factor	to Equation 4.1
Local optimization	
Method	variable

Figure 6.25(a), 6.25(d) and 6.25(g) show us the roof and we can see that there are some differences but in general we can say the three methods obtain similar results. On Figure 6.25(b), 6.25(e) and 6.25(h) we can see the bottom of the model and clearly the (1+1)-ES method represented the worst recovery of the floor. Finally, on On Figure 6.25(c), 6.25(f) and 6.25(i) a side view is shown where we can see some of the interior recovery, and on all three experiments the wall, cube and pyramid were retrieved successfully although the pyramid seems to be pixelated but this is due to the voxel size. One last observation in these side views is that on the (1+1)-ES experiment it seems that a little section of a wall was not recovered.

The results for each method can be seen on Table 6.39 where we notice at first glance that the Nelder-Mead has a considerably higher time compared to the two other methods and the (1+1)-ES method has the highest % retrieved.

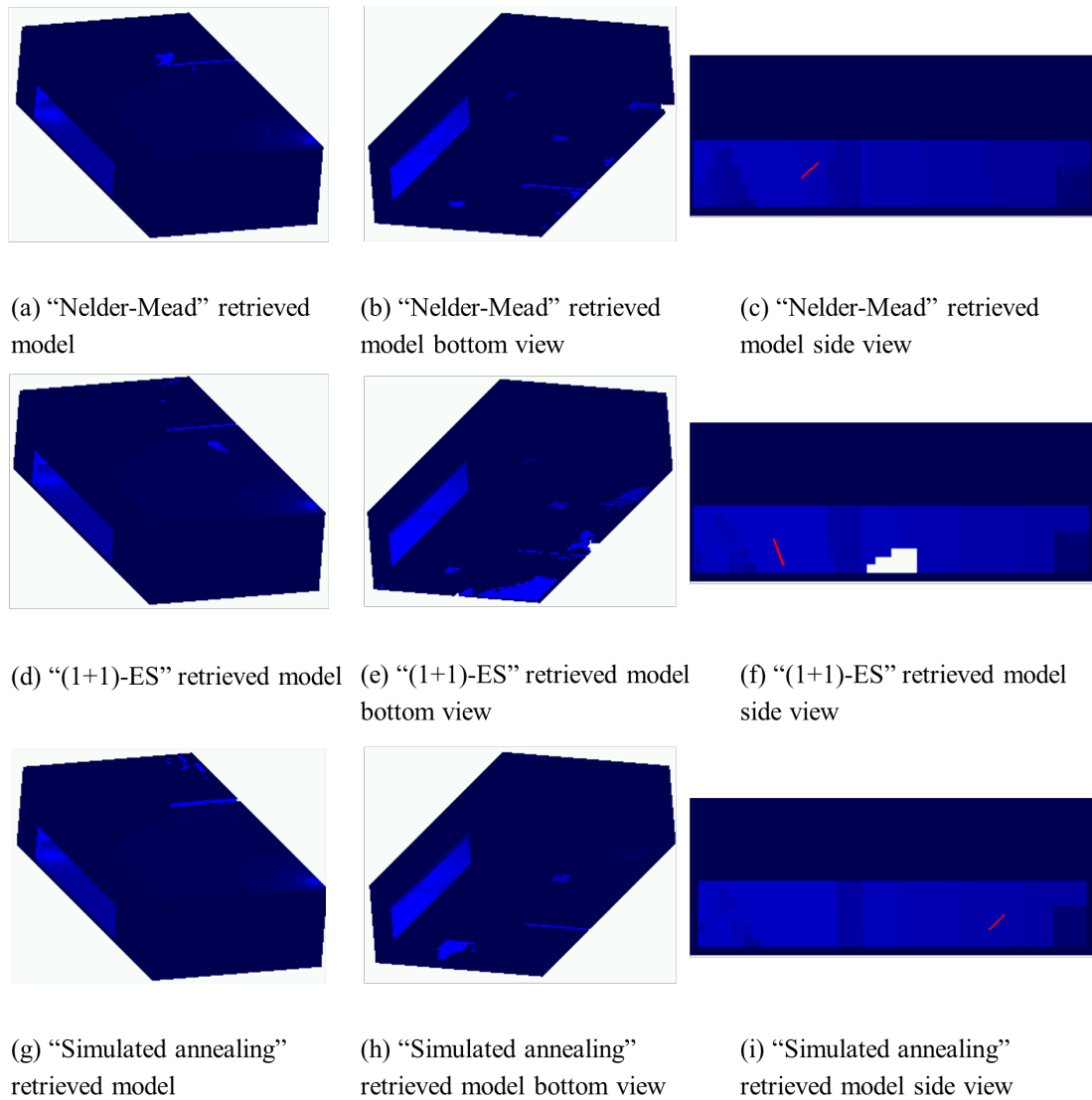


FIGURE 6.25: Room with objects model experiment retrieved models.

TABLE 6.39: Room with objects model experiment results.

Method	Accumulated $dist_{2d} (m)$	Accumulated e	%retrieved	$mean_{quality}$	Total time (s)
Nelder-Mead	98.435	5.23088	83.47	0.699634	131.053333
(1+1)-ES	119.858	6.012989	84.03	0.731505	34.733333
Simulated annealing	146.451	6.218211	83.54	0.727047	35.72

On Figure 6.26(a) we can see that the %retrieved is very close among the three methods and that on different view numbers the best and worst method were changing, so on this variable there is no clear winner but all of them seem to behave correctly. On Figure 6.26(b) the Nelder-Mead experiment was clearly the best as from view #20 it was the one with the smallest distance, plus it was also the case on the energy comparison on Figure 6.26(c). Figure 6.26(d) represents

the mean quality and the three methods seem to fall around views 5-10 to then recover and increase the quality until the end where the (1+1)-ES method wins only by a extremely little difference. Finally, on Figure 6.26(e) we can see the total time and the Nelder-Mead method is the worst one with the biggest time on all the views by a huge difference, where (1+1)-ES and simulated annealing seem to be tied on all the view numbers and we know from the tables that the best was the (1+1)-ES experiment but on the plot that small difference is not noticeable.

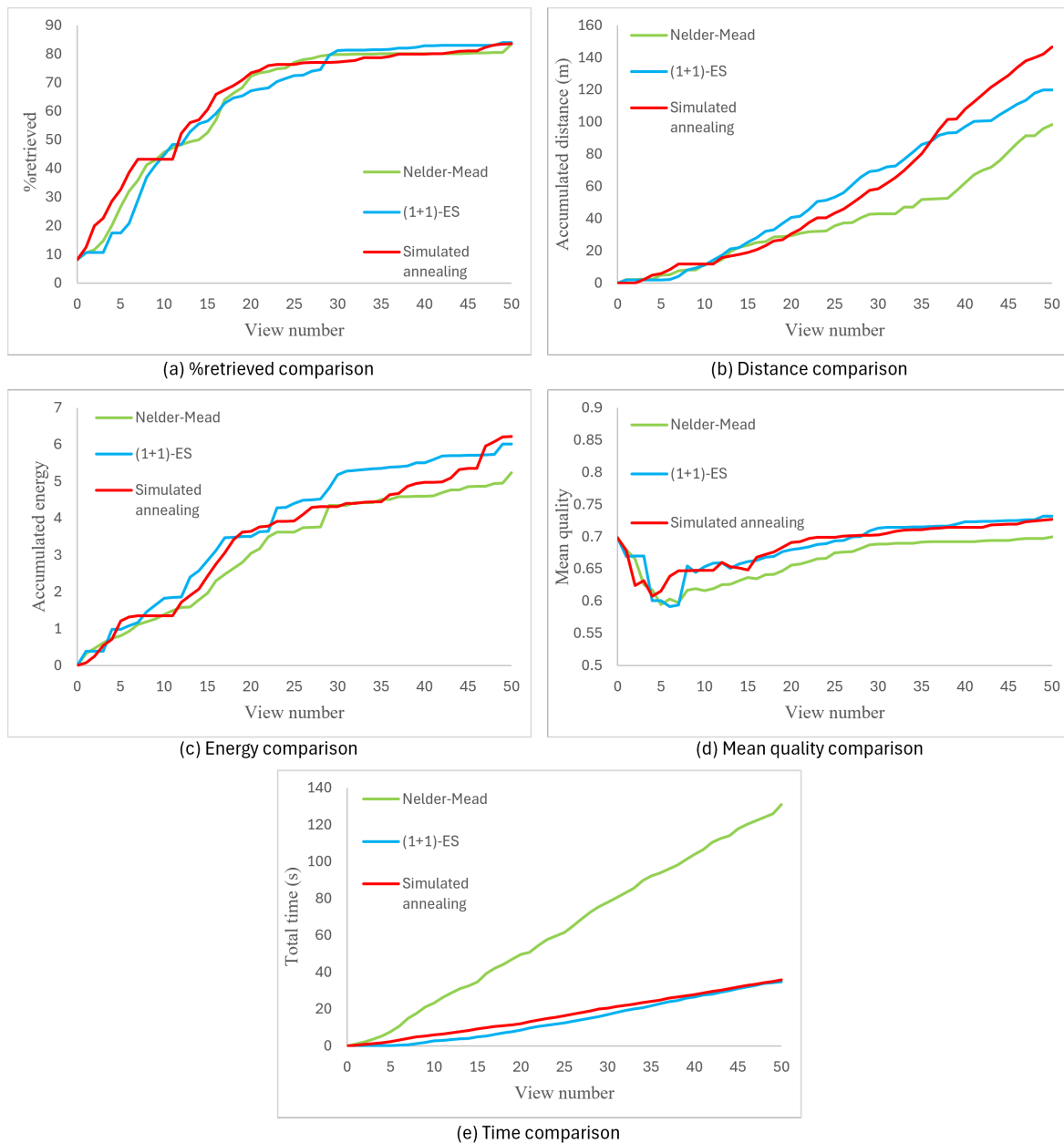


FIGURE 6.26: Room with objects model experiment graphs.

As we established that the decision should be made considering all the factors of the objective

function and the total time, we will use the ranking technique from section 6.5 where we computed function, time and total ranks and can be seen on Table 6.40. With this criteria, the best

TABLE 6.40: Room with objects model experiment ranked results.

Method	d_{rank}	e_{rank}	$retr_{rank}$	f_{rank}	$time_{rank}$	$total_{rank}$
Nelder-Mead	0.000	0.000	0.666	0.333	277.313	277.646
(1+1)-ES	21.764	14.952	0.000	9.860	0.000	9.860
Simulated annealing	48.779	18.875	0.583	20.196	2.841	23.036

method on this environment is the (1+1)-ES algorithm and it was the best on all the individual criteria except for the distance and energy, meaning that it uses more resources, but the balance with the other criteria means that it is worth it. Overall, all three methods provide good results with little differences between them except for the huge difference in time of the Nelder-Mead method.

6.6.2 Study room model comparisons

This second environment is the study room which can be seen on Figure 6.3 with the information shown in Table 6.2. The simulation parameters for this experiments are shown on Table 6.41 and we increase the number of views to 100 as this model is more complex than the previous one. Still, it was tested with 50 views and the difference on the retrieved models is indeed significant. Then, it was tested with 200 views and we noticed that the improvement on the retrieved model was now negligible so we kept the experiments with 100 views.

On Figure 6.27(a), 6.27(d) and 6.27(g) we can see that there are no noticeable differences on the general shape of the model. However, on Figure 6.27(b), 6.27(e) and 6.27(h) we can see the bottom of the models is clearly where the missing % retrieved is as they have a lot of holes, with the (1+1)-ES experiment having the biggest holes and the other two methods being similar between them. Finally, on On Figure 6.27(c), 6.27(f) and 6.27(i) we have a view with more interior detail to see the furniture and we notice that the sofa in front of the top wall is the one with more issues as it has holes although the simulated annealing one seems to be the best regarding the sofa as it only has a tiny hole on the middle of the sofa while the others have bigger holes.

TABLE 6.41: Study room model experiment simulation parameters.

Parameter	Value
General considerations	
Environment	study room
Number of views	100
Safe cylinder	yes
Objective function	
Area factor	Equation 3.20, $\alpha = 0.4$
Distance factor	Equation 3.32
Energy factor	Equation 3.37
Motion factor	Equation 6.4
Quality factor	none
Total objective function	$f_{area} * f_{motion}$
Global optimization	
Minimum backstepping distance	3 m
Backstepping register	yes
Dynamic change in area factor	to Equation 4.1
Local optimization	
Method	variable

The results for each method can be seen on Table 6.42 where we notice at first glance that the Nelder-Mead has a considerably higher time compared to the two other methods besides it having slightly more %retrieved.

TABLE 6.42: Study room model experiment results.

Method	Accumulated $dist_{2d} (m)$	Accumulated e	%retrieved	$mean_{quality}$	Total time (s)
Nelder-Mead	316.973	21.902109	92.41	0.793342	183.333333
(1+1)-ES	283.049	23.259942	89.17	0.799405	25.026667
Simulated annealing	287.862	28.269608	90.26	0.781581	22.813333

On Figure 6.28(a) we can see that the simulated annealing method provided less %retrieved on most of the views although at the end all three methods seem to have similar results. On Figure 6.28(b) the simulated annealing algorithm was the one that needed less distance on most of the views although at the end the (1+1)-ES won by having the smallest distance on the last view. On Figure 6.28(c) it is quite the opposite as the Nelder-Mead experiment which was the one with the highest distance now is the one with the smallest energy consumption on all the views after view #25 approximately. Figure 6.28(d) represents the mean quality and the simulated annealing experiment is clearly the worst on this criteria, where the Nelder-Mead one seems to be have the highest quality on most of the views but at the end on the last views the (1+1)-ES

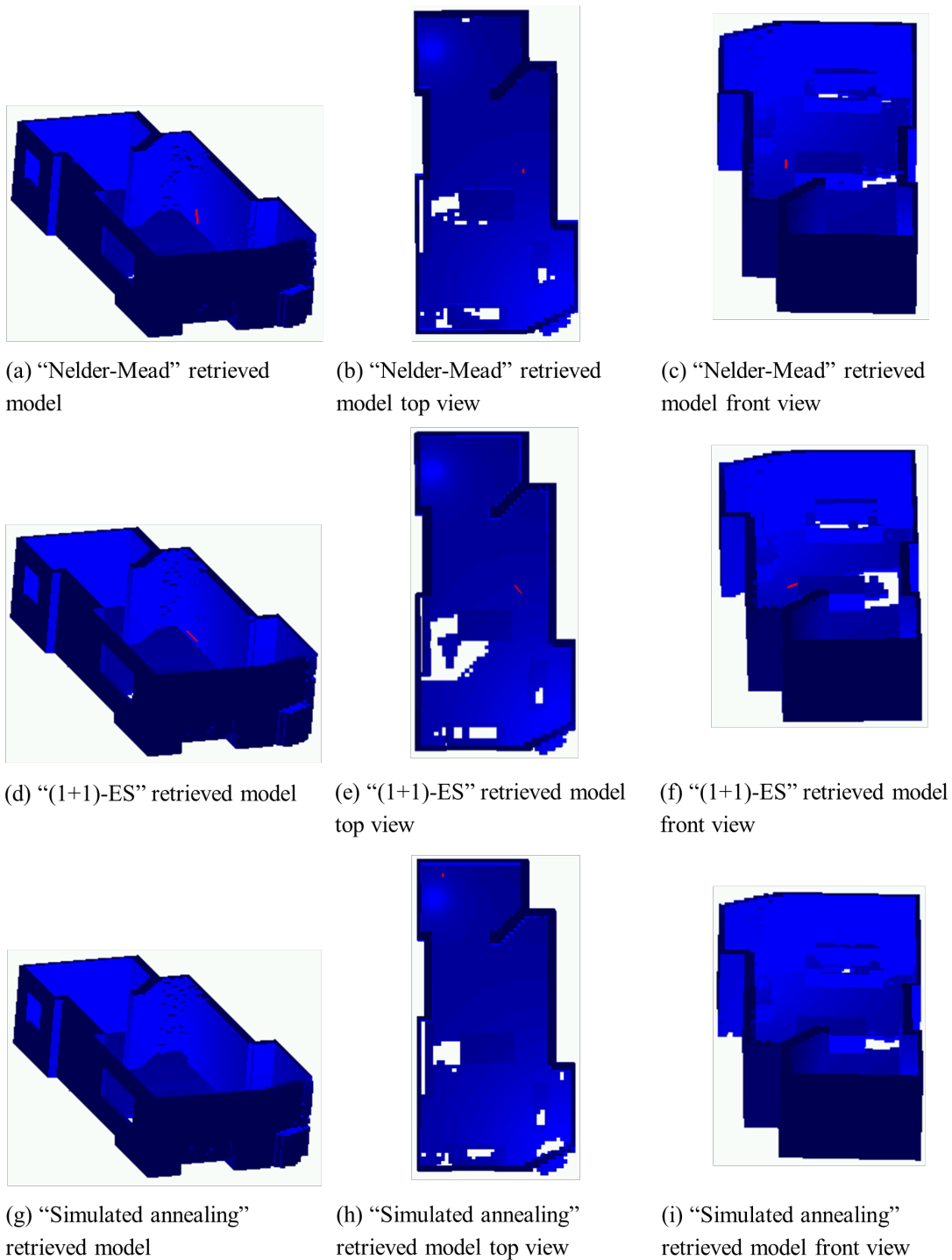


FIGURE 6.27: Study room model experiment retrieved models.

algorithm takes the win. Finally, on Figure 6.28(e) we can see the total time and the Nelder-Mead method is the worst one with the biggest time on all the views by a huge difference, where (1+1)-ES and simulated annealing seem to be tied on all the view numbers until the last ones where the simulated annealing wins with the smallest time although it is by a little difference.

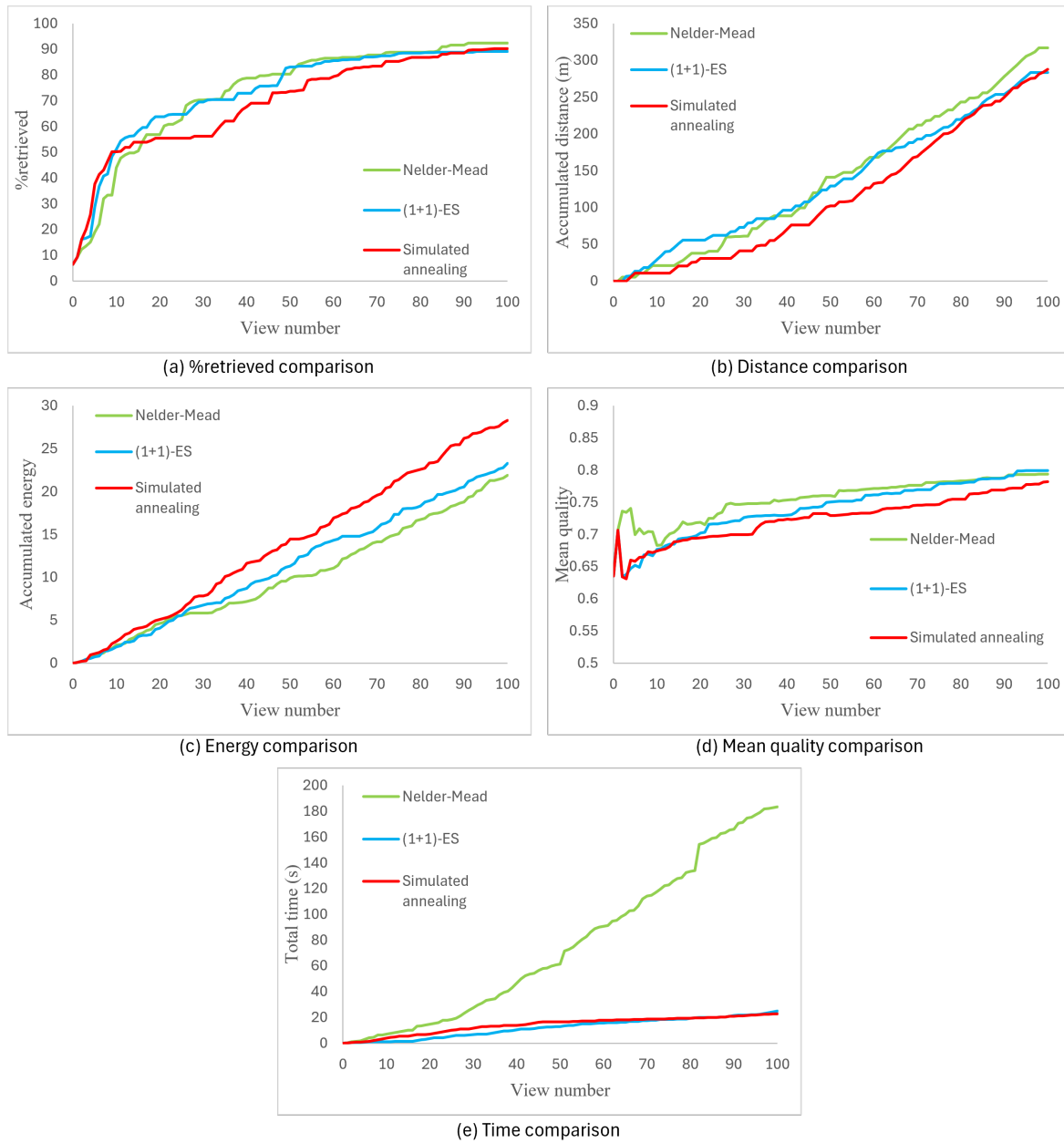


FIGURE 6.28: Study room model experiment graphs.

As we established that the decision should be made considering all the factors of the objective function and the total time, we will use the ranking technique from section 6.5 where we computed function, time and total ranks and can be seen on Table 6.43. With this criteria, the best

TABLE 6.43: Study room model experiment ranked results.

Method	d_{rank}	e_{rank}	$retr_{rank}$	f_{rank}	$time_{rank}$	$total_{rank}$
Nelder-Mead	11.985	0.000	0.000	4.195	703.624	707.818
(1+1)-ES	0.000	6.200	3.506	2.683	9.702	12.385
Simulated annealing	1.700	29.073	2.327	6.119	0.000	6.119

method on this environment is the simulated annealing even if it was the best only in the time rank. This means that even if it was not the best on the other individual criteria, it had more balance than the other methods. Overall, all three methods provide good results with little differences between them except for the huge difference in time of the Nelder-Mead method.

6.6.3 Laboratory model comparisons

This last environment is the laboratory model which can be seen on Figure 6.5 with the information shown in Table 6.3 and is the model used to do the tuning. We will collect the best versions of the three methods on a single experimental matrix for a more straight forward comparison and we will provide new graphs of their performance. The simulation parameters for this experiments are shown on Table 6.44 and we keep the number of views as 100 which was used on the previous experiments mainly because we consider this environment a complex one and we want to provide the algorithm enough opportunity to retrieve the model. Still, it was tested with 200 views and we noticed that the improvement on the retrieved model was negligible so we kept the experiments with 100 views.

TABLE 6.44: Laboratory model experiment simulation parameters.

Parameter	Value
General considerations	
Environment	laboratory model
Number of views	100
Safe cylinder	yes
Objective function	
Area factor	Equation 3.20, $\alpha = 0.4$
Distance factor	Equation 3.32
Energy factor	Equation 3.37
Motion factor	Equation 6.4
Quality factor	none
Total objective function	$f_{area} * f_{motion}$
Global optimization	
Minimum backstepping distance	3 m
Backstepping register	yes
Dynamic change in area factor	to Equation 4.1
Local optimization	
Method	variable

On Figure 6.29 we notice that visually the difference is almost unnoticeable although the (1+1)-ES experiment seems to have a few more occplane voxels but we will rely more on the numerical information.

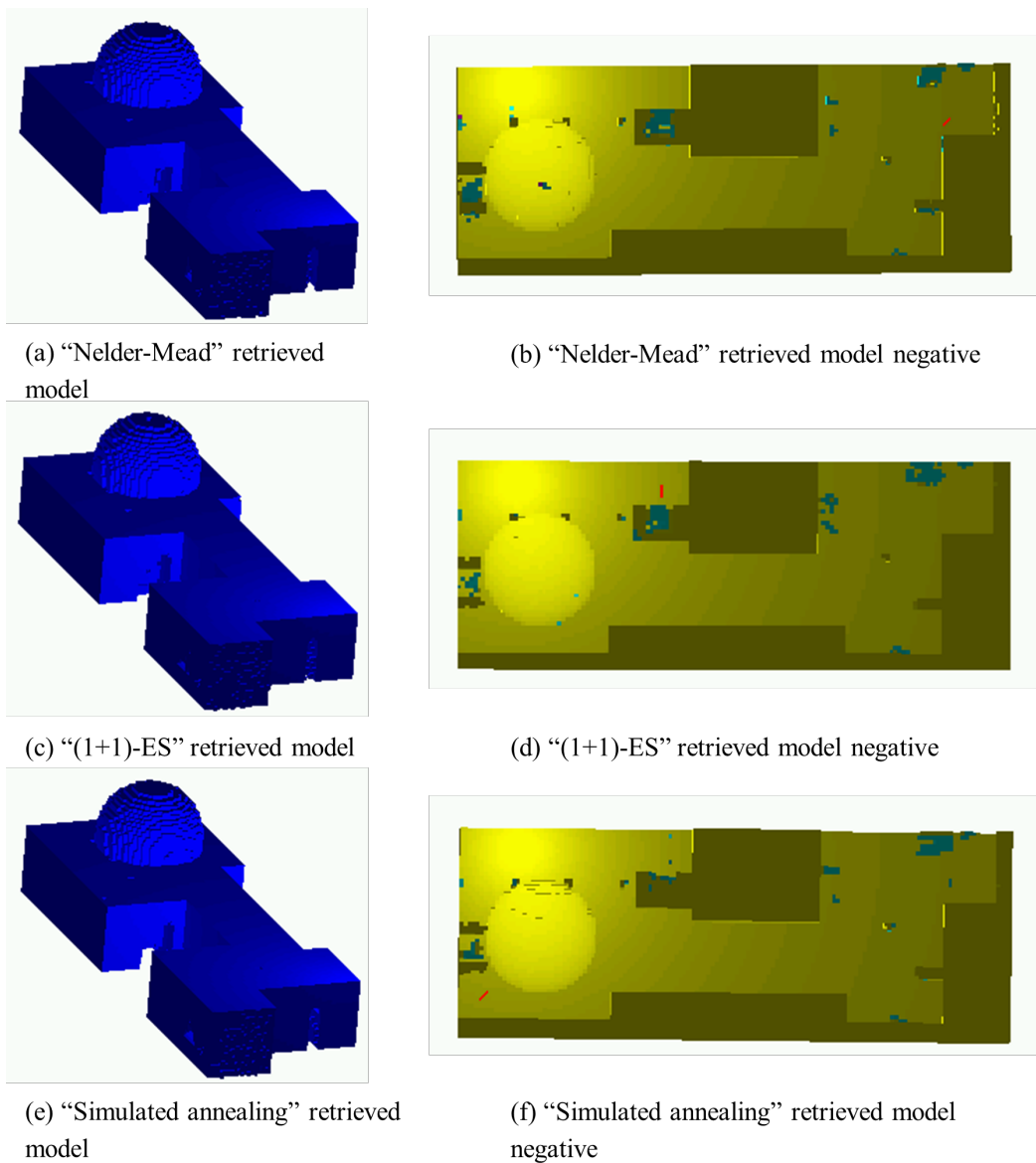


FIGURE 6.29: Laboratory model experiment retrieved models.

The results for each method can be seen on Table 6.45 where we notice at first glance that the Nelder-Mead has a considerably higher time compared to the two other methods but having the most %retrieved.

TABLE 6.45: Laboratory model experiment results.

Method	Accumulated $dist_{2d} (m)$	Accumulated e	%retrieved	$mean_{quality}$	Total time (s)
Nelder-Mead	327.351	19.115758	89.51	0.806914	297.066667
(1+1)-ES	272.077	16.543897	87.78	0.799471	120.493333
Simulated annealing	317.389	22.771662	87.69	0.803419	176.306667

On Figure 6.30(a) we can see that the simulated annealing method provided more %retrieved on most of the views although at the end in the last views the best method turned out to be Nelder-Mead by a little difference. On Figure 6.30(b) the (1+1)-ES algorithm was the one that needed less distance from all methods on all the views and is the same case for the energy on Figure 6.30(c). Figure 6.30(d) represents the mean quality and the three methods seem to fall around views 10-20 to then recover and increase the quality until the end where the Nelder-Mead method wins only by a extremely little difference. Finally, on Figure 6.30(e) we can see the total time and the Nelder-Mead method is the one with the biggest time on all the views, where (1+1)-ES and simulated annealing seem to be tied on the first views and around view 60 although at the end (1+1)-ES wins having a considerable less time around the last views.

As we established that the decision should be made considering all the factors of the objective function and the total time, we will use the ranking technique from section 6.5 where we computed function, time and total ranks and can be seen on Table 6.46. With this criteria, the

TABLE 6.46: Laboratory model experiment ranked results.

Method	d_{rank}	e_{rank}	$retr_{rank}$	f_{rank}	$time_{rank}$	$total_{rank}$
Nelder-Mead	20.316	15.546	0.000	9.442	146.542	155.984
(1+1)-ES	0.000	0.000	1.933	0.966	0.000	0.966
Simulated annealing	16.654	37.644	2.033	12.492	46.321	58.813

best method on this environment is the (1+1)-ES algorithm and it was the best on all the individual criteria except for the %retrieved and $mean_{quality}$. The $mean_{quality}$ does not represent a problem as the difference is extremely little compared to the other two methods, plus we do not take into consideration this factor on the rankings as our objective functions does not consider it. On the other hand, %retrieved is indeed considered on our rankings but also the difference was not that significant. Overall, all three methods provide good results with little differences between them except for the difference in time of the Nelder-Mead method although it is less extreme than on the previous two models.

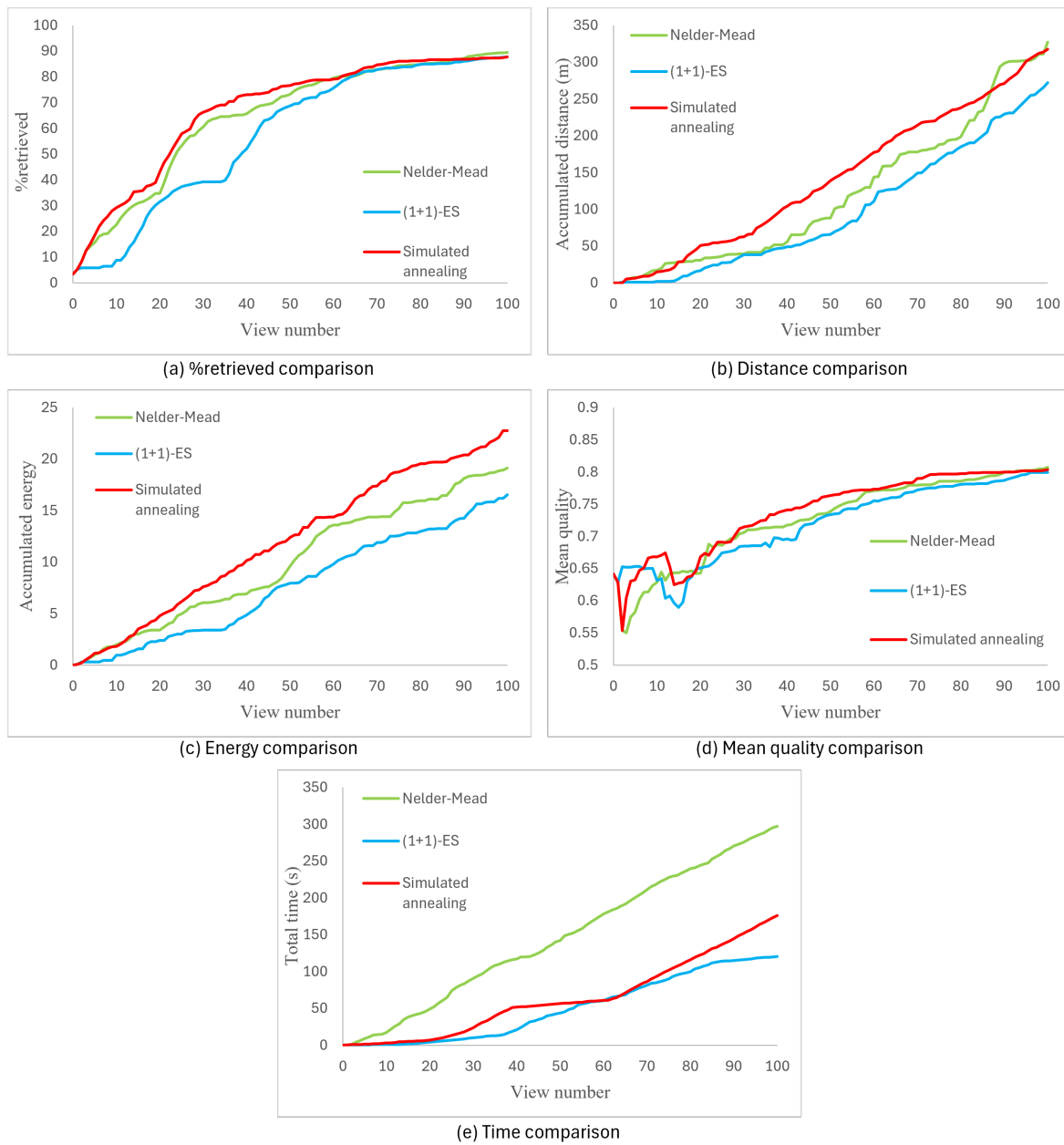


FIGURE 6.30: Laboratory model experiment graphs.

Chapter 7

Conclusions and Future Work

7.1 Conclusions

The principal area factor was defined successfully using as a base the work from [6] where we got to the conclusion that the variable α indeed has an effect on the behaviour of the function, so its best value was chosen. We formulated extra factors such as distance and energy and we noticed that the inclusion of a quality factor was not convenient when making an analysis of the overall balance among our measurement variables. Our first hypothesis that by adding more factor to the objective function we could upgrade the results of the retrieved model was correct.

Probably the most crucial part was defining the global optimization tasks, as we noticed that without them the robot was getting stuck on local minima and the process ended with poor results regarding the retrieval of the model. The principal task was the backstepping technique which helped the robot to return to previously visited positions every time it was stuck on local minima, and the reason why we chose to deal with this local minima problem by adding an external algorithm such as our backstepping technique is because if we wanted to deal with it purely with the optimization method, we would need to choose either a full exhaustive search which would not be feasible due to its immense possible points, or to use some optimization method designed for escaping local minima like tabu search which we indeed took into consideration in our experiments but noticed that even with such type of methods our backstepping algorithm was still being used making our local minima problem extreme enough to be handled without this technique. The other task was a dynamic change in the objective function when the unmarked voxels were less than 1% to change priorities for the function as there was no need

to keep looking for unmarked area. This two tasks provided overall good results by increasing significantly the percentage of retrieval of the model even if the resources consumption was also increased, but we decided that the retrieval had more priority in that sense.

As for the optimization methods tested, an adequate tuning of their individual parameters was done at the best of our abilities because if we decided to make a full experimental matrix it would not been optimal due to the nature of the parameters being continuous and the endless combinations of all of them. Even if no full experimental matrix was made, we still got the opportunity to notice that by changing the parameters we could end up with huge differences in our results. On the Nelder-Mead we had no parameters but instead we chose to compare two versions of the method, but on the (1+1)-ES we now included parameter variations and we managed to upgrade the $total_{rank}$ when tuning its first parameter from 35.415 being the highest to 4.656 being the lowest and the best, and then from 219.538 to even 0.000 on its second parameter meaning that the final version was the best in all the criteria. For the simulated annealing we only had one parameter tuning where we got to improve the $total_{rank}$ from 53.122 being the highest to 1.664 being the best. Our second hypothesis that the changes in the parameters when tuning each optimization method have an impact on their results is correct.

During the final comparison we noticed that even if on two models the best algorithm was the (1+1)-ES, on the remaining model the simulated annealing was the best, this means that there is no clear superior algorithm between those two for all possible environments but it varies depending on the scene, although the overall results on both of them were similar so there should be no problem on using either of those two methods on any environment. The main observation was with the Nelder-Mead method as it was the worst on all three models on its $total_{rank}$ with a significant difference from the other two methods, with the most affected criterion being its high computational time even if it was considerably reduced against its two-phase variant method, but still not enough to compete against (1+1)-ES and simulated annealing on that aspect even if it had overall good results on all the other criteria. Our third and final hypothesis that we can obtain significant differences among our proposed methods is correct when comparing (1+1)-ES and simulated annealing against the Nelder-Mead method which we consider as the worst.

7.2 Future Work

One main assumption of this work was the simplification of the movement of the robot as it was decided that due to being on a simulation the sensor could move from one point to another without any problem by just checking if the space around it was empty. However, it would be ideal that a deeper analysis on the robot kinematics and path planning was done on a navigation module to make easier its future implementation on a real robot inside a real environment.

During the final experiments done on all three environments, we noticed that even if the experiments had a good retrieval percentage, they seemed to look incomplete visually. We identified that the few existing holes were concentrated between them making those missing parts more noticeable. For this, we suggest to modify the area factor from the objective function so that even if two candidate images have the same values on their individual area proportions, different weights or priorities are assigned if they are concentrated or scattered. This is only a proposal but a deeper analysis and experiments should be made to test if indeed it provides better results or not.

Finally, after those two suggestions are dealt with, we could then start a physical implementation as future work where new considerations and steps should be considered to adapt our solution with real world tools.

Appendix A

Articles Published

The article "Tuning and comparison of optimization algorithms for the Next Best View problematic" by Everardo Shain-Ruvalcaba and Efraín López-Damian was accepted for publication on IEEE Access with a DOI of "10.1109/ACCESS.2024.3513154". When submitting this thesis document, the article is still in an initial production stage with IEEE Publishing Operations, meaning that we cannot provide an adequate reference. However, we include proof of its status as "Accepted" in Figure A.1, a proof of its production state including the DOI in Figure A.2, and its first page in Figure A.3.

IEEE Access
Regular Manuscript

Tuning and comparison of optimization algorithms for the Next Best View problematic

Submission Status	Accepted
Manuscript ID	Access-2024-38741

FIGURE A.1: Accepted status of the article.

Tuning and comparison of optimization algorithms for the Next Best View problematic

IEEE Access

Manuscript Number	Digital Object identifier	Published to IEEE Xplore®	Production State	Alert
Access-2024-38741	10.1109/ACCESS.2024.3513154	Article has not published	In initial production stage with IEEE Publishing Operations	

FIGURE A.2: Production state of the article.

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.8429080

Tuning and comparison of optimization algorithms for the Next Best View problematic

EVERARDO SHAIN-RUVALCABA AND EFRAÍN LÓPEZ-DAMIAN

Tecnológico de Monterrey, School of Engineering and Sciences, Av. Eugenio Garza Sada 2501, Monterrey, N.L., México, 64849

Corresponding author: Efraín López-Damian (e-mail: eldamian@tec.mx)

The APC was funded by Tecnológico de Monterrey.

ABSTRACT The aim of this paper is to tune and compare different optimization algorithms on the Next Best View (NBV) problem, which consists in finding the next position that the sensor or camera needs to take to scan an object or scenery in its totality. A simulated 5 Degree-of-Freedom (DOF) mobile robot with a mounted simulated range sensor was used on a Virtual Reality Modeling Language (VRML) environment, and the space discretization was made using a voxel map. For the objective function, two main factors were included: an area factor to make sure that the image taken by the sensor provides the best possible information, and a motion factor made up of distance and energy sub-factors to reduce the resources used by the robot. Tasks such as a backstepping technique to escape local minima and a dynamic change in the objective function were implemented. The retrieval of the scene was made on an iterative process, and three different optimization methods were tuned and tested: Nelder-Mead, an Evolution Strategy, and Simulated Annealing. A set of experiments comparing the three methods in computational time and retrieval efficiency were made on three different environments with increasing difficulty to test their repeatability, with them being a laboratory model, a room with a cube and a pyramid inside it, and a study room with multiple furniture and windows.

INDEX TERMS Next Best View, objective function, optimization, voxel map.

I. INTRODUCTION

GETTING true digital representations of a physical object or environment can be a challenge when it comes to autonomous methods. Nowadays, Computer-Aided Design (CAD) technologies are raising on importance among world-wide fabricators, so many objects produced in the industry are brought to reality by converting a 3D digital object to a physical piece. However, there is the opposite situation in which the object exists without a CAD model like the ones made by artisans, or even pieces that indeed were fabricated using digital models or blueprints that were kept by the fabricator, so it is not possible to access them. If we want to obtain the digital object, we need to have some scanning strategy to save time and resources that would be bigger if no strategy was used resulting in a random exhaustive scanning. This strategy is called Next Best View, which is defined as the process to select the view that would provide more information on the object or environment following some criteria, and repeating this process until the scan is complete.

Some applications where this technique can be used are object digitization for reverse engineering such as a core die of the inlet of a diesel engine [1]. It can even be used for

non-engineering applications such as criminology providing a reconstruction of a crime scene [2]. A medical approach can also be set in which a whole scene is scanned, although instead of objects or terrain, a set of chronic wounds are reconstructed with the human body being the environment [3]. For industrial applications involving robotics, the NBV is used for environment exploration on autonomous mobile robots required to find and retrieve task-related objects to accomplish their tasks [4], and a more concrete example can be found on industrial facilities in which robots perform an inspection of the facility so that workers can avoid hazardous places when performing activities such as removing toxic substances, and repairing damaged machinery [5].

NBV can be considered as an optimization problem as we seek to find the best view given the available conditions; therefore, two key elements are the objective function and the optimization method used. The objective function sets the conditions and criteria to define how good a view, while the optimization method is an algorithm to reach the best view according to the requirements of the objective function. Both have a huge importance when designing the solution as the first defines the nature of the problem and the second provides

FIGURE A.3: First page of the article.

Bibliography

- [1] Yu Zhang. Research into the engineering application of reverse engineering technology. *Journal of Materials Processing Technology*, 139(1-3):472–475, 2003.
- [2] Jinming Wang, Zhengdong Li, Wenhua Hu, Yu Shao, Liyang Wang, Rongqi Wu, Kaijun Ma, Donghua Zou, and Yijiu Chen. Virtual reality and integrated crime scene scanning for immersive and heterogeneous crime scene reconstruction. *Forensic science international*, 303:109943, 2019.
- [3] Damir Filko, Domagoj Marijanović, and Emmanuel Karlo Nyarko. Automatic robot-driven 3d reconstruction system for chronic wounds. *Sensors*, 21(24):8308, 2021.
- [4] Christopher McGreavy, Lars Kunze, and Nick Hawes. Next best view planning for object recognition in mobile robotics. In *CEUR Workshop Proceedings*, volume 1782. CEUR Workshop Proceedings, 2017.
- [5] Menaka Naazare, Francisco Garcia Rosas, and Dirk Schulz. Online next-best-view planner for 3d-exploration and inspection with a mobile manipulator robot. *IEEE Robotics and Automation Letters*, 7(2):3779–3786, 2022.
- [6] Robert Fisher and José Miguel Sanchiz. A next-best-view algorithm for 3d scene recovery with 5 degrees of freedom. In *Proc. British Machine Vision Conference BMVC99*, 1999.
- [7] MT Lozano Albalade, Michel Devy, J Miguel, and S Marti. Perception planning for an exploration task of a 3d environment. In *2002 International Conference on Pattern Recognition*, volume 3, pages 704–707. IEEE, 2002.
- [8] Richard Pito. A sensor-based solution to the” next best view” problem. In *Proceedings of 13th International Conference on Pattern Recognition*, volume 1, pages 941–945. IEEE, 1996.

- [9] Tomas Olsson. Vision guided force control in robotics. *MSc Theses*, 2001.
- [10] Liila Torabi and Kamal Gupta. An autonomous six-dof eye-in-hand system for in situ 3d object modeling. *The International Journal of Robotics Research*, 31(1):82–100, 2012.
- [11] Torea Foissotte, Olivier Stasse, Adrien Escande, and Abderrahmane Kheddar. A next-best-view algorithm for autonomous 3d object modeling by a humanoid robot. In *Humanoids 2008-8th IEEE-RAS International Conference on Humanoid Robots*, pages 333–338. IEEE, 2008.
- [12] Robots Team. HRP-2, dec 2023.
- [13] Reem Ashour, Tarek Taha, Jorge Manuel Miranda Dias, Lakmal Seneviratne, and Nawaf Almoosa. Exploration for object mapping guided by environmental semantics using uavs. *Remote Sensing*, 12(5):891, 2020.
- [14] Guillaume Hardouin, Julien Moras, Fabio Morbidi, Julien Marzat, and El Mustapha Mouaddib. Next-best-view planning for surface reconstruction of large-scale 3d environments with multiple uavs. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1567–1574. IEEE, 2020.
- [15] Andreas Bircher, Mina Kamel, Kostas Alexis, Helen Oleynikova, and Roland Siegwart. Receding horizon” next-best-view” planner for 3d exploration. In *2016 IEEE international conference on robotics and automation (ICRA)*, pages 1462–1468. IEEE, 2016.
- [16] Mateja Hržica, Robert Cupec, and Ivan Petrović. Active vision for 3d indoor scene reconstruction using a 3d camera on a pan-tilt mechanism. *Advanced Robotics*, 35(3-4):153–167, 2021.
- [17] Laurana M Wong, Christophe Dumont, and Mongi A Abidi. Determining optimal sensor poses in 3-d object inspection. In *Conference on Quality Control By Artificial Vision, Takamatsu (Japan)*, pages 371–377. Citeseer, 1998.
- [18] Nikolaos A Massios, Robert B Fisher, et al. *A best next view selection algorithm incorporating a quality criterion*, volume 2. Citeseer, 1998.
- [19] Torea Foissotte, Olivier Stasse, Pierre-Brice Wieber, and Abderrahmane Kheddar. Using newuoa to drive the autonomous visual modeling of an object by a humanoid robot. In *2009 International Conference on Information and Automation*, pages 78–83. IEEE, 2009.

- [20] Juan Irving Vásquez-Gómez, Efraín López-Damian, and Luis Enrique Sucar. View planning for 3d object reconstruction. In *2009 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 4015–4020. IEEE, 2009.
- [21] Joseph E Banta, LR Wong, Christophe Dumont, and Mongi A Abidi. A next-best-view system for autonomous 3-d object reconstruction. *IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans*, 30(5):589–598, 2000.
- [22] Yanzi Kong, Feng Zhu, Haibo Sun, Zhiyuan Lin, and Qun Wang. A generic view planning system based on formal expression of perception tasks. *Entropy*, 24(5):578, 2022.
- [23] J Irving Vasquez-Gomez, David Troncso, Israel Becerra, Enrique Sucar, and Rafael Murrieta-Cid. Next-best-view regression using a 3d convolutional neural network. *Machine Vision and Applications*, 32:1–14, 2021.
- [24] Randa Almadhoun, Abdullah Abduldayem, Tarek Taha, Lakmal Seneviratne, and Yahya Zweiri. Guided next best view for 3d reconstruction of large complex structures. *Remote Sensing*, 11(20):2440, 2019.
- [25] Cl Connolly. The determination of next best views. In *Proceedings. 1985 IEEE international conference on robotics and automation*, volume 2, pages 432–435. IEEE, 1985.
- [26] Fredrik Bissmarck, Martin Svensson, and Gustav Tolt. Efficient algorithms for next best view evaluation. In *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5876–5883. IEEE, 2015.
- [27] Christian Potthast and Gaurav S Sukhatme. A probabilistic framework for next best view estimation in a cluttered environment. *Journal of Visual Communication and Image Representation*, 25(1):148–164, 2014.
- [28] J Irving Vasquez-Gomez, L Enrique Sucar, and Rafael Murrieta-Cid. Hierarchical ray tracing for fast volumetric next-best-view planning. In *2013 International conference on computer and robot vision*, pages 181–187. IEEE, 2013.
- [29] J Irving Vasquez-Gomez, L Enrique Sucar, Rafael Murrieta-Cid, and Juan-Carlos Herrera-Lozada. Tree-based search of the next best view/state for three-dimensional object reconstruction. *International Journal of Advanced Robotic Systems*, 15(1):1729881418754575, 2018.

- [30] Abderrahim Saaïdi, Khalid Satori, et al. Multi-view passive 3d reconstruction: Comparison and evaluation of three techniques and a new method for 3d object reconstruction. In *2014 International Conference on Next Generation Networks and Services (NGNS)*, pages 194–201. IEEE, 2014.
- [31] Luis Miguel González-deSantos, Joaquín Martínez-Sánchez, Higinio González-Jorge, Lucía Díaz-Vilariño, and B Riveiro. New discretization method applied to nbv problem: Semioc-tree. *PloS one*, 13(11):e0206259, 2018.
- [32] Günter Hetzel, Bastian Leibe, Paul Levi, and Bernt Schiele. 3d object recognition from range images using local feature histograms. In *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001*, volume 2, pages II–II. IEEE, 2001.
- [33] J Irving Vasquez-Gomez, L Enrique Sucar, Rafael Murrieta-Cid, and Efrain Lopez-Damian. Volumetric next-best-view planning for 3d object reconstruction with positioning error. *International Journal of Advanced Robotic Systems*, 11(10):159, 2014.
- [34] Marco Steinbrink. Sampling-based exploration strategies for mobile robot autonomy. 2023.
- [35] Yao Zhao, Zhi Xiong, Shuailin Zhou, Jingqi Wang, Ling Zhang, and Pascual Campoy. Perception-aware planning for active slam in dynamic environments. *Remote Sensing*, 14(11):2584, 2022.
- [36] Ana Batinovic, Antun Ivanovic, Tamara Petrovic, and Stjepan Bogdan. A shadowcasting-based next-best-view planner for autonomous 3d exploration. *IEEE Robotics and Automation Letters*, 7(2):2969–2976, 2022.
- [37] Hongbiao Zhu, Chao Cao, Yukun Xia, Sebastian Scherer, Ji Zhang, and Weidong Wang. Dsvp: Dual-stage viewpoint planner for rapid exploration by dynamic expansion. in 2021 iee. In *RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7623–7630.
- [38] Sertac Karaman and Emilio Frazzoli. Incremental sampling-based algorithms for optimal motion planning. 2011.
- [39] Marco Steinbrink, Philipp Koch, Bernhard Jung, and Stefan May. Rapidly-exploring random graph next-best view exploration for ground vehicles. In *2021 European Conference on Mobile Robots (ECMR)*, pages 1–7. IEEE, 2021.

- [40] Jack E Bresenham. Algorithm for computer control of a digital plotter. In *Seminal graphics: pioneering efforts that shaped the field*, pages 1–6. 1998.
- [41] Akira Fujimoto, Takayuki Tanaka, and Kansei Iwata. Arts: Accelerated ray-tracing system. *IEEE Computer Graphics and Applications*, 6(4):16–26, 1986.
- [42] Frederick S Hillier. *Introduction to operations research*. McGrawHill, 2001.
- [43] George Dantzig. *Linear programming and extensions*. Princeton university press, 1963.
- [44] Stephen P Boyd and Lieven Vandenberghe. *Convex optimization*. Cambridge university press, 2004.
- [45] Kanchan Rajwar, Kusum Deep, and Swagatam Das. An exhaustive review of the metaheuristic algorithms for search and optimization: Taxonomy, applications, and open challenges. *Artificial Intelligence Review*, 56(11):13187–13257, 2023.
- [46] John A Nelder and Roger Mead. A simplex method for function minimization. *The computer journal*, 7(4):308–313, 1965.
- [47] Margaret H Wright. Direct search methods: Once scorned, now respectable. *Pitman Research Notes in Mathematics Series*, pages 191–208, 1996.
- [48] Michael JD Powell. Direct search algorithms for optimization calculations. *Acta numerica*, 7:287–336, 1998.
- [49] Tamara G Kolda, Robert Michael Lewis, and Virginia Torczon. Optimization by direct search: New perspectives on some classical and modern methods. *SIAM review*, 45(3):385–482, 2003.
- [50] Ilhem Boussaïd, Julien Lepagnot, and Patrick Siarry. A survey on optimization metaheuristics. *Information sciences*, 237:82–117, 2013.
- [51] Thomas Bartz-Beielstein, Jürgen Branke, Jörn Mehnen, and Olaf Mersmann. Evolutionary algorithms. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 4(3):178–195, 2014.
- [52] M^a Zapatero Moreno, Jesús Alegre Martínez, and Joaquín Pacheco Bonrostro. Análisis de algunas metaheurísticas creadas a partir de “optimización gravitatoria”. *Anales de ASEPUMA*, 19, 2011.

-
- [53] Marie Laure Delignette Muller and Christophe Dutang. Which optimization algorithm to choose?, apr 2023.
- [54] William T Vetterling, Saul A Teukolsky, William H Press, and Brian P Flannery. *Numerical recipes in C: the art of scientific computing*. Cambridge university press, 1999.
- [55] Hans-Paul Schwefel. *Evolution and Optimum Seeking*. 01 1995.
- [56] Carlos A Coello Coello and Col San Pedro Zacatenco. Introducción a la computación evolutiva. *México*, 2004.
- [57] Hans-Paul Schwefel. *Numerical optimization of computer models*. John Wiley & Sons, Inc., 1981.
- [58] Fred Glover. Tabu search: A tutorial. *Interfaces*, 20(4):74–94, 1990.
- [59] Dimitris Bertsimas and John Tsitsiklis. Simulated annealing. *Statistical science*, 8(1):10–15, 1993.
- [60] Darrall Henderson, Sheldon H Jacobson, and Alan W Johnson. The theory and practice of simulated annealing. *Handbook of metaheuristics*, pages 287–319, 2003.
- [61] Moon-Won Park and Yeong-Dae Kim. A systematic procedure for setting parameters in simulated annealing algorithms. *Computers & Operations Research*, 25(3):207–217, 1998.
- [62] Konrad Klein and Vitor Sequeira. View planning for the 3d modelling of real world scenes. In *Proceedings. 2000 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2000)(Cat. No. 00CH37113)*, volume 2, pages 943–948. IEEE, 2000.
- [63] AG Mangrio, M Asif, E Ahmed, MW Sabir, T Khan, I Jahangir, et al. Hydraulic performance evaluation of pressure compensating (pc) emitters and micro-tubing for drip irrigation system. *Science Technology and Development, Islamabad*, 32(4):290–298, 2013.